ENGINEERING HANDBOOK · REVISION 1

Takashi Dungeons

How the procedural dungeon engine works, and why every part of it works that way

A complete walkthrough of Phases 1, 2 and 3 — the generation core, the instance lifecycle, and the mob system — written for someone who wants to be able to change the code, not just run it. Every rule in here is traced back to the measurement or the failure that produced it.

PAPER 1.21.8 · JAVA 21 · MAVEN · FAWE / WORLDEDIT · GPLv3

com.takashi:TakashiDungeons · package com.takashi.dungeons

Onur Sessiz · 2026

Contents

PART ZERO

Orientation

What this plugin is, what it deliberately refuses to be, and how its packages are arranged. Three short chapters that make the rest of the book readable in any order.

CHAPTER 1

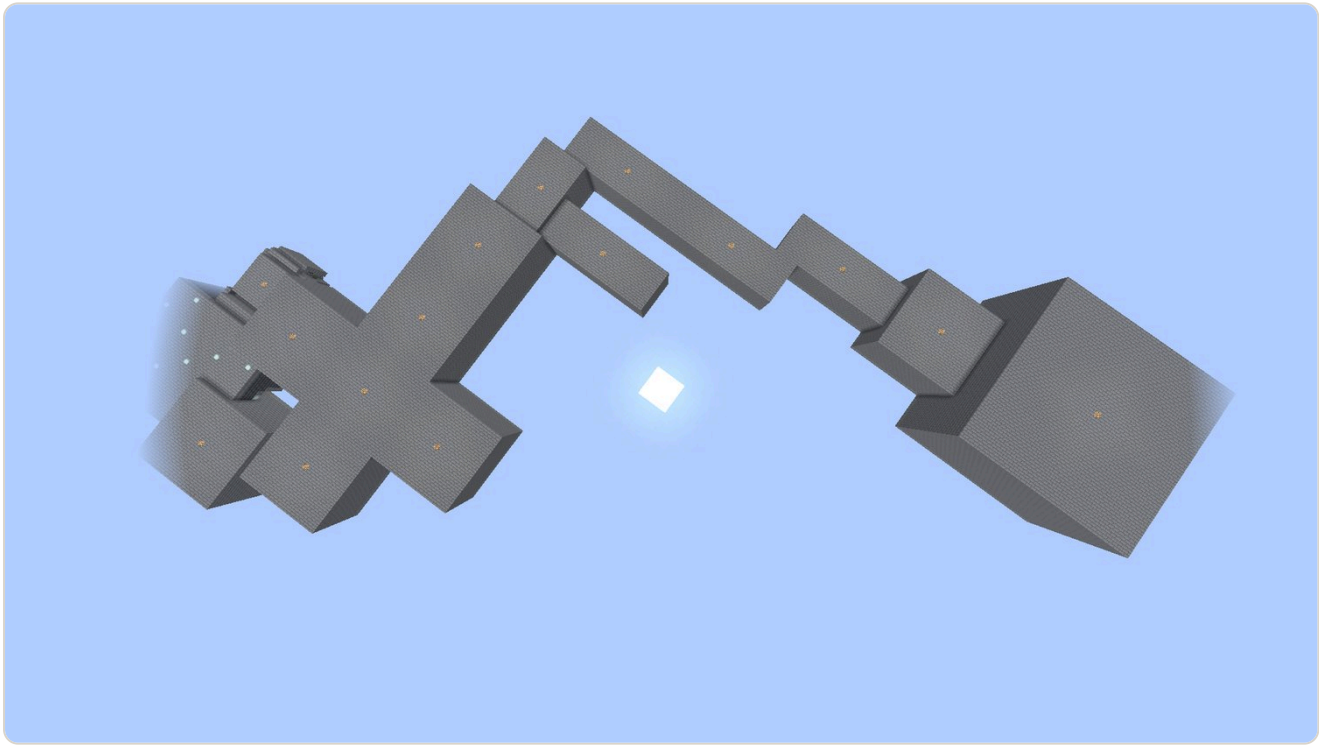
What the plugin is, and the rules it will not break

TakashiDungeons is a procedural dungeon system for Paper 1.21.8. It generates an instanced dungeon out of hand-built room schematics, gives that dungeon a life with a countdown and an entrance in the world, fills it with mobs whose statistics are configurable, and then deletes it cleanly. It is free and open source under GPLv3, and it is not written for one specific server — which turns out to be the single most important fact about its design.

1.1 The one sentence version

Most "random dungeon" plugins generate the *rooms* procedurally, using cellular automata or binary space partitioning. Both approaches produce something, but not what a dungeon needs: cellular automata produce caves, and BSP splits produce rectangles. Neither can accommodate a room a human being drew by hand.

This engine does not generate rooms at all. Rooms are **schematics** a map team draws in a world editor and exports. The engine's entire job is to decide **which room, where, and at what angle**. That is the socket/jigsaw approach — the same method Mojang uses for villages, bastions, ancient cities and trial chambers — and it is the only one that allows a 9×25 corridor, a 17×17 hall and a 33×33 boss arena to coexist inside one dungeon.



One `medium` dungeon, viewed from outside the void world. Every box is a hand-built schematic; only their *choice*, *position* and *rotation* are generated.

1.2 Free changes the requirements — upward

The project was originally going to be sold. That idea was deliberately abandoned in August 2026, and the reason was not technical: collecting foreign marketplace income as a sole trader in Turkey (registration, VAT, withholding, currency handling) creates an administrative burden larger than the plugin's price would return. Free distribution removes that burden entirely and serves the actual goal — visibility — better.

> **THE REASONING THAT MATTERS**
>
> Being free does **not** lower the engineering standard, it raises it. A person who pays for a plugin installs it by reading the documentation. A person who downloads a free one installs it without reading anything, and if it breaks, the thing that gets dragged through the mud is the author's name. Every feature therefore has to work **out of the box**, on a server whose owner has read nothing.

That single constraint explains an enormous amount of the code you are about to read. It is why the plugin enables with zero optional dependencies installed. It is why the room set is bundled inside the jar and extracted on first start. It is why a missing MythicMobs does not disable the mob system but only the entries that named MythicMobs. It is why a dungeon generates even when no boss room template exists.

1.3 Why GPLv3 and not MIT

What is being protected is not revenue — the plugin is free. It is *attribution*. The failure scenario is concrete: somebody forks it, renames the package, deletes the README and uploads it under their own name. MIT does not prevent that; it asks for the licence text to remain in the source, and nobody who downloads a jar looks at the source. GPL forces anyone distributing a derivative to open *their* source too. It does not make theft impossible, it makes it **visible and unprofitable** — and that is what deters it.

Bukkit and Spigot's own API is GPL, so this is not an eccentric choice in this ecosystem. It is also a **one-way door**: a version published under GPL cannot later be closed, which is why "relicense to something more permissive" appears on the never-do list.

> **ATTRIBUTION ACTUALLY LIVES IN THE JAR, NOT THE LICENCE**
>
> The licence is legal leverage. In practice the author's name travels in three places: `plugin.yml`'s `author:` field, which appears in the server's `/plugins` listing; the signature line printed to the console on every `onEnable`; and `/tdungeons version`. Deleting a README is free; deleting those three is deliberate effort, and at that point the other party cannot claim ignorance. New systems must preserve all three.

1.4 The product architecture

JAR	CONTENTS	STATUS
TakashiDungeons (core)	Generation, mob spawning, loot, parties, instancing	This project
TakashiRanks (addon)	XP and rank system	Separate jar, separate repo, free — Phase 11
TakashiMarket (addon)	Coin and market system	Separate jar, separate repo, free — Phase 12

Addons are separate jars in separate repositories and talk to the core exclusively through the public API the core exposes in Phase 8. The core has to be fully functional on its own, with no addon installed. This is why the sidebar HUD already has `coin`, `xp` and `rank` placeholder rows that render as `-`: the seams exist, the systems that will fill them do not yet.

1.5 Decisions that are closed

These are recorded as unchangeable in the project's design charter. They are listed here not as trivia but because several chapters later will look like arbitrary choices unless you know they are constraints.

Build tool: Maven

A single `pom.xml`, with `maven-shade-plugin` relocating the SQLite driver into `com.takashi.dungeons.libs.sqlite`.

Platform: Paper API, kept Spigot-compatible

Paper-only features are used where they are clearly better (Adventure components, display entities), but the plugin does not require a Paper fork feature to function.

Database: SQLite by default, MySQL optional. Player data is *never* in YAML.

This is why the HUD's on/off preference is currently session-only. Persisting it to a YAML file would have been three lines and would have broken a rule.

Schematics: FAWE/WorldEdit `.schem` with async paste

WorldEdit is a *soft* dependency. Without it the plugin still enables; generation reports itself as disabled.

Room variety comes from *rotation*, never from duplicated door variants

Chapter 14 works through the arithmetic that makes this the difference between 40 and 200 schematics.

Instanced only — never permanent world writes

Supporting both would double every lifecycle question. One model, chosen once.

YAML first, GUI editors later

Every system must be fully configurable in YAML before any GUI is written for it. The GUI is a convenience over a working file format, never the only way to configure something.

1.6 Rules that are never broken

> **THE NEVER-DO LIST**
>
> - Never store player data in YAML or a flat file.
> - Never hard-depend the core on a mob plugin. MythicMobs and friends are `softdepend`; with none of them installed, the vanilla fallback must work.
> - Never make a breaking change to the public API — addons break.
> - Never support instanced dungeons *and* permanent world writing at once.
> - Never try to bolt custom currency onto vanilla villager trading. It does not work; the answer is cancelling `PlayerInteractEntityEvent` and opening a custom GUI.
> - `/tp` and `/tpa` must not work inside a dungeon — **administrators excepted** (a decision revised on 2 September 2026; the earlier decision had included admins).
> - Never relicense away from GPLv3.
> - Never remove the signature line from the enable log.
> - Every mob spawn carries a `statOverride: true/false` flag, always.

1.7 One phase at a time, finished

The roadmap has thirteen phases, and the governing rule is that **a phase is not left until it works**. Not "until it compiles" and not "until the happy path runs once" — until it has been verified, usually both headlessly and on a live server, and the verification has been written down.

PHASE	SUBJECT	STATE
0	Project setup, build → deploy loop	Complete
1	**Generation core** — 1A infrastructure, 1B geometry, 1C graph placement, 1D graph generation	Complete, verified
2	**Instance lifecycle** — 2A ownership and teardown, 2B duration and tracking, 2C the entrance portal	Complete, verified in game
3	**Mob system** — 3A providers and registry, 3B room population, 3C boss and ownership	3A and 3B complete; 3C in progress
4	Loot: weighted rarity, chests, mob drop tables	Not started
5	Parties	Not started
6	Supply mob (optional module)	Not started
7	Database: SQLite/MySQL, async access, migrations	Not started
8	Public API and custom events	Not started
9	GUI editors over the existing YAML	Not started
10	Map building: 4 biomes × 10 room types (runs in parallel)	Not started
11–12	Addons: Ranks, Market	Not started
13	Release preparation	Not started

This handbook covers phases 1, 2 and 3 in full. Everything described here exists and has run.

CHAPTER SUMMARY

- Rooms are *arranged* procedurally, never generated. Rooms are hand-drawn schematics; the engine chooses which, where and at what angle.
- Free distribution raises the engineering bar because a free plugin is installed without being read. Out-of-the-box operation is a hard requirement, not a nicety.
- GPLv3 protects attribution, not revenue, and it is a one-way door.
- Phases end only when they demonstrably work. Phases 1 and 2 are done; phase 3 is at 3C.

CHAPTER 2

The build, the test server, and how to run any of this

Short chapter, but skipping it costs an afternoon. The toolchain has one hard constraint and the console test loop has four traps, all four of which were hit for real during phase 1B.

2.1 JDK 21 is mandatory, and it is pinned

Paper 1.21.8 does not accept Java 26, and the development machine has JDK 26 on the system `PATH`. Both build scripts therefore hard-code the Temurin JDK 21 path rather than resolving `mvn` or `java` from `PATH`. If you invoke either from the path, the build or the server breaks — and the error message will not point at the cause.

```
powershell -ExecutionPolicy Bypass -File scripts\build.ps1          # compile -> run\plugins\
powershell -ExecutionPolicy Bypass -File scripts\server.ps1         # start Paper 1.21.8
powershell -ExecutionPolicy Bypass -File scripts\geo-probe\run.ps1  # geometry, no server needed
```

The three entry points. Everything else is a variation on these.

`build.ps1` pins `JAVA_HOME`, runs `mvnw.cmd clean package`, lets `maven-shade-plugin` embed and relocate the SQLite driver, lets resource filtering substitute `${project.version}` into `plugin.yml`, and copies the resulting jar into `run/plugins/` — deleting the old jar first, because two versions sitting side by side means the server loads both.

> **A NOTE FOR PHASE 7**
>
> Because the SQLite driver is relocated, the class to load in phase 7 is `com.takashi.dungeons.libs.sqlite.JDBC`, not `org.sqlite.JDBC`. This is the kind of detail that produces a twenty-minute `ClassNotFoundException` hunt years later.

2.2 The headless geometry probe

The `generation` package is deliberately pure Java — no Bukkit types, no WorldEdit types. That is not stylistic. It means the placement mathematics can be executed on a bare JVM, which is what `scripts/geo-probe` does. There are five probe programs:

PROBE	WHAT IT CHECKS	ASSERTIONS
`GeoProbe`	Rotation round-trips, all 16 `align` combinations, wall derivation on square, rectangular and asymmetric rooms, collision rules, 5 rooms × every door × every direction = 48 placements	53
`GenProbe`	Weight distribution over 200,000 draws (deviation < 0.3%), pool filtering, draw-without-replacement, 500 seeds with zero inconsistency, slot overflow, DEAD marking, reproducibility	29
`DungeonProbe`	Path length formula, critical path guarantee across 3×1000 generations, boss type and depth, size ranges, plug target coverage, consecutive-seed independence, out-of-the-box fallbacks	31
`SpawnProbe`	L-shaped hall, narrow corridor, room with a closed centre, two-storey room, sealed alcove, minimum spacing, same seed → same result, floorless room	22
`RotProbe`	The rotation sign, measured against WorldEdit's own `AffineTransform`	—

WHY THIS MATTERS MORE THAN IT LOOKS

A flood fill that is wrong on an L-shaped hall **will not be caught on a server**. Every room still receives mobs; they are simply in the wrong places. The same is true of the wall-derivation formula: in a square room the naive rule and the correct rule agree, so the bug is invisible until the first rectangular room arrives. Headless probes catch the class of error that in-game testing structurally cannot.

2.3 The test server is not in the repository

`run/` is git-ignored. Anyone setting up on another machine supplies two things themselves: `run/paper.jar` (Paper 1.21.8) and FastAsyncWorldEdit in `run/plugins/`. Without FAWE the plugin still enables — generation reports itself disabled and everything else, including the HUD and the mob registry, keeps working.

The room schematics *are* in the repository, bundled inside the jar under `src/main/resources/schematics/` and extracted on first enable. That is the out-of-the-box guarantee: downloading the jar and installing FAWE should be enough. As of this writing the bundled set contains three entrance rooms and no normal or boss rooms, so a clean installation still cannot generate a dungeon. The missing piece is *rooms*, not code — that is Phase 10.

2.4 Driving the server from the console: four traps

Commands can be fed to the server on standard input, which makes automated verification possible. All four of the following were hit during phase 1B, and all four produce *false passes* rather than errors — which is what makes them worth a section.

TRAP 1 — FORCELOAD BEFORE ANY BLOCK CHECK

FAWE does not leave the chunks it pasted into loaded. `execute if block` in an unloaded chunk silently produces no result — not a failure, *no result*. Every block verification must be preceded by `forceload add`. The dungeon world's dimension key is `minecraft:takashi_dungeons`.

TRAP 2 — DELETE LATEST.LOG FIRST

The old log still contains `Done (`. A script that waits for that string will think the server is ready and start sending commands during startup.

TRAP 3 — POWERSHELL 5.1 WRITES A UTF-8 BOM TO STDIN

`StandardInputEncoding` does not exist in .NET Framework, so the first write carries a byte-order mark that corrupts the first command. Send one sacrificial line and let it be eaten.

TRAP 4 — DELETE THE TEST WORLD

`run/takashi_dungeons` must be removed between runs. The slot index restarts at zero on every boot, old blocks remain underneath (`release` does not clear them — see Chapter 21), and the result is a *passing* test on stale geometry.

2.5 Where the documentation lives

The project keeps its memory in a folder that is **not** in the repository — `AI Yönergeleri/`, git-ignored, present only on the author's machine. Five files: the design charter, the session log, the roadmap, the generation specification, and the systems reference. In a clean clone the equivalents are `README.md` and `docs/generation.md`.

THE MOST EXPENSIVE MISTAKE IN THIS PROJECT

`docs/generation.md` is the public English counterpart of the private generation specification. **If the algorithm changes, both are updated.** A document that is not updated does not become merely stale — it starts to lie, and it lies with the full authority of a document that was once correct.

CHAPTER SUMMARY

- JDK 21 is pinned in both scripts because the system PATH has JDK 26 and Paper rejects it.
- Touch the geometry, run `geo-probe` before starting a server. 135 headless assertions across four probes.
- The four console traps all cause false passes, not errors.
- SQLite is relocated: phase 7 loads `com.takashi.dungeons.libs.sqlite.JDBC`.

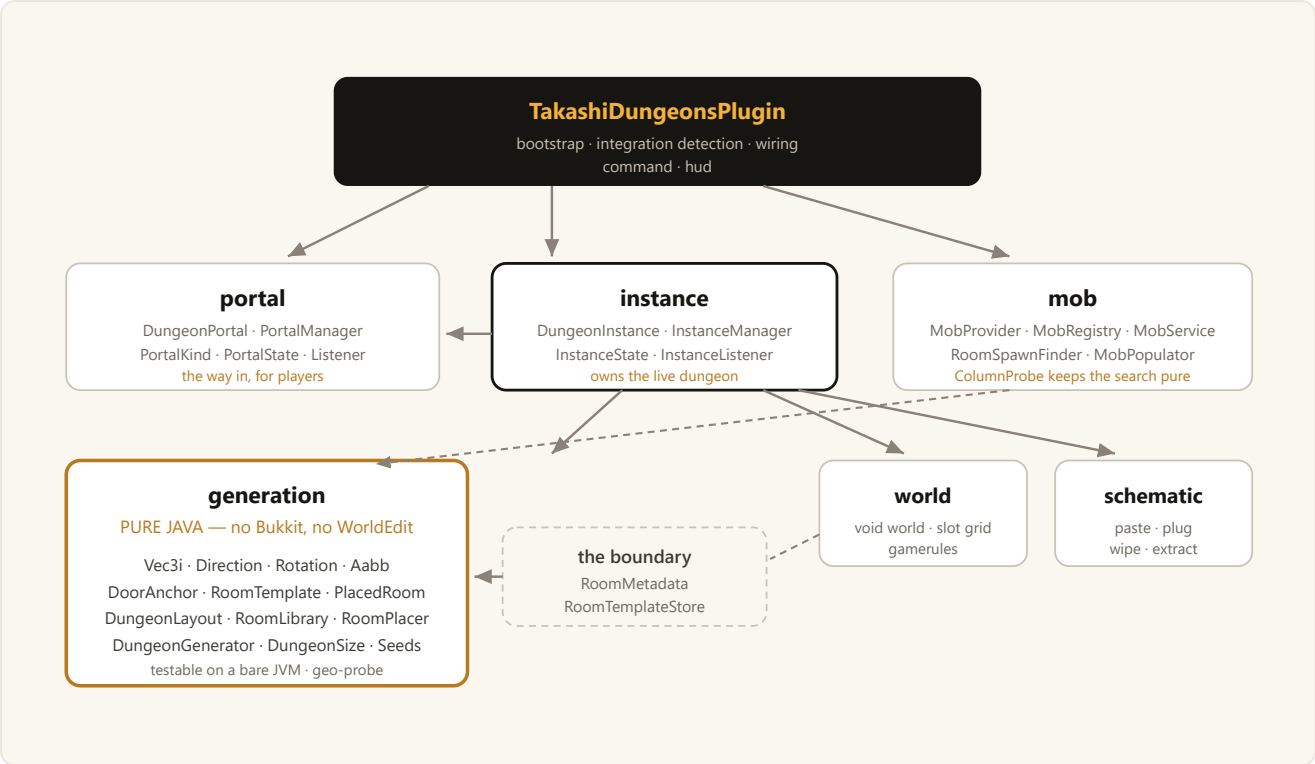
CHAPTER 3

Architecture map: packages, layers and the purity boundary

Eight packages, roughly ten thousand lines of Java. This chapter is the map you should be able to come back to whenever a later chapter mentions a class you have forgotten. The single most important structural fact is the *purity boundary*: which packages are allowed to know that Minecraft exists.

3.1 The packages

PACKAGE	FILES	RESPONSIBILITY	DEPENDS ON
`generation`	18	Geometry, rotation, door anchors, collision, the room graph, the generator itself	**Nothing** (two boundary classes excepted)
`schematic`	5	Loading and pasting `.schem` files, plugging doors, wiping regions, generating test rooms, extracting bundled rooms	WorldEdit/FAWE, Bukkit
`world`	4	The void world, its gamerules, and the slot grid	Bukkit
`instance`	4	Live dungeons: creation, membership, the clock, teardown, the teleport block	All of the above
`portal`	5	Entrances standing in the world, wild spawning, lobby refresh	`instance`
`mob`	14	Providers, the registry, stat application, spawn point search, room population	`generation`, `instance`, Bukkit
`hud`	1	The sidebar scoreboard	Bukkit only
`command`	2	Everything reachable from `/tdungeons` and `/hud`	Everything



Dependencies point *downward and inward*. Nothing points into `generation` from below, and `generation` points at nothing at all.

3.2 The purity boundary, and why it is worth the effort

The `generation` package uses no Bukkit type and no WorldEdit type. It has its own integer vector (`Vec3i`) rather than WorldEdit's `BlockVector3`, and its own box (`Aabb`) rather than anything Bukkit offers. Two classes sit on the boundary and are the exception — `RoomMetadata`, which parses YAML, and `RoomTemplateStore`, which joins a schematic to its metadata.

There are exactly two reasons, and both are load-bearing:

1. The placement mathematics is testable without a server.

Starting a Paper server, generating a dungeon and walking through it is a thirty-second loop at best. Running `GeoProbe` is under a second, and it asserts fifty-three invariants including cases that are physically awkward to construct in game — an asymmetric room, a rotation round-trip, a box that overhangs a slot edge by one block.

2. No third-party type leaks into the API that Phase 8 will publish.

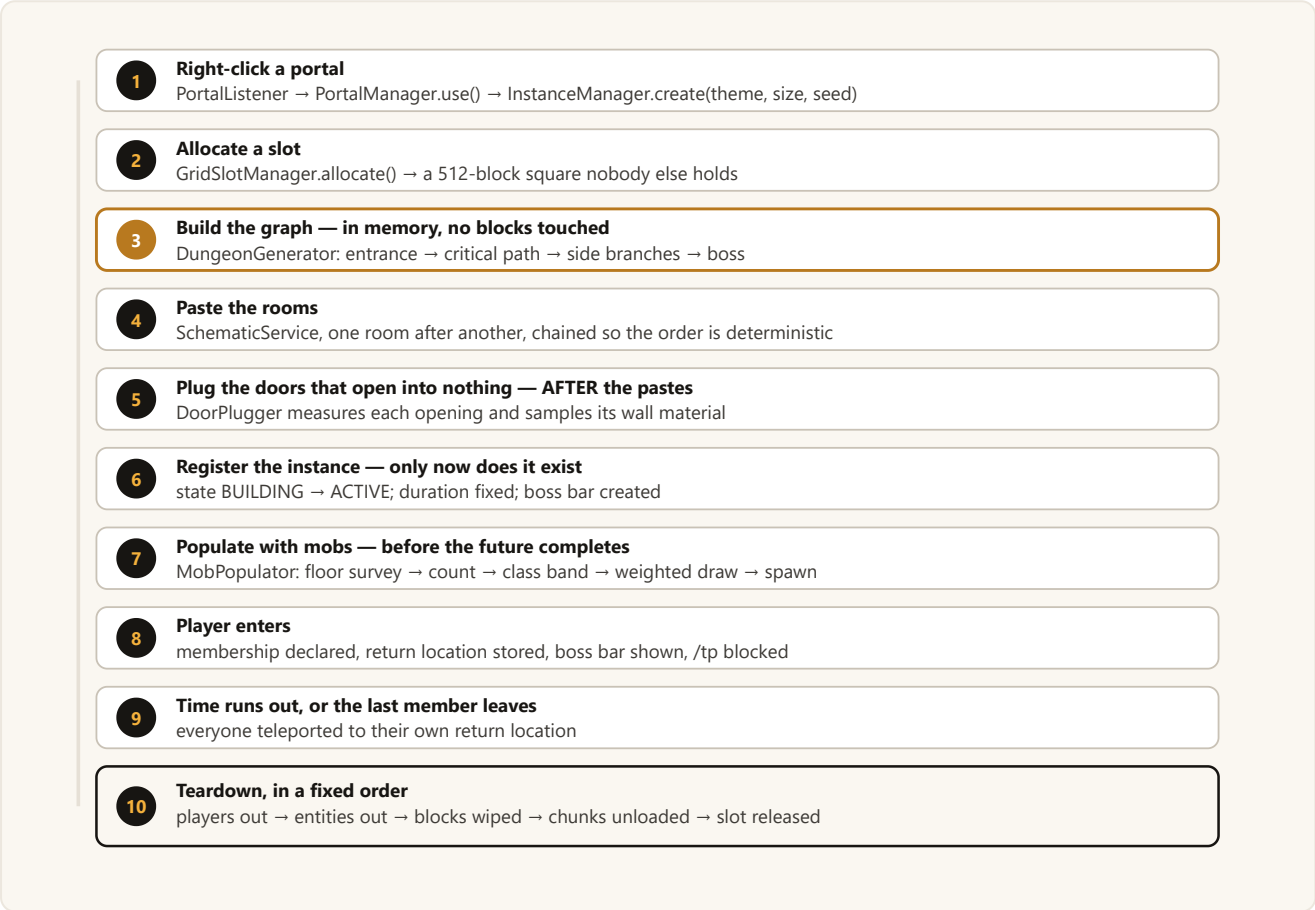
Addons will consume these classes. If `PlacedRoom` exposed a `BlockVector3`, then a WorldEdit major version bump would be a breaking change to *this* plugin's API — and breaking the API is on the never-do list. The conversion to WorldEdit types happens at the paste call, on one line.

THE SAME IDEA APPEARS TWICE MORE

`ColumnProbe` in the mob package does exactly this for the spawn-point search: the flood fill sees the world only through `isFloor(x,y,z)` and `isClear(x,y,z)`. `WorldColumnProbe` is the one-line Bukkit implementation; `SpawnProbe` in `geo-probe` implements the same two methods over hand-drawn ASCII room shapes. Likewise `DungeonLayout` receives the slot boundary as an `Aabb` passed in from outside rather than asking a `World` for its height limits.

3.3 How a dungeon comes into existence, end to end

This is the whole system in one sequence. Every step gets a chapter of its own later; the point here is the shape.



The full lifecycle. Steps 1–5 belong to Phase 1, 6 and 8–10 to Phase 2, step 7 to Phase 3.

3.4 Reading order for the rest of the book

Part I is the longest and the most mathematical; it is also the part everything else assumes. Parts II and III can be read independently of each other, but both assume Part I's vocabulary — *slot*, *layout*, *node*, *anchor*, *plug*. Part IV collects the principles that recur everywhere, plus the command and configuration reference.

CHAPTER SUMMARY

- Eight packages; dependencies point inward, and `generation` depends on nothing.
- The purity boundary buys headless testability and protects the future public API.
- `ColumnProbe` repeats the same trick for the mob spawn search.
- A dungeon's life is ten steps; the ordering of steps 4–7 and of the teardown is itself the design, not an implementation detail.

PART ONE

The generation core

Phase 1. How a list of hand-drawn schematics becomes a connected, sealed, walkable dungeon with a guaranteed critical path — and how every formula in it was measured rather than assumed.

CHAPTER 4

Why arrange rooms instead of generating them

Every procedural dungeon system starts with the same fork in the road: do you generate the space, or do you generate the arrangement of spaces somebody else drew? This project takes the second road, and the reason is not a preference. It is a consequence of who is doing the work.

4.1 The constraint that decides it

The plan for Phase 10 is four biomes × ten room types = **forty hand-built schematics**, produced by a team of two or three people working in parallel with the code. Those rooms are the product. They are what a player actually looks at: pillars, mossy brick, a glowstone panel in the ceiling, a staircase that turns.

Ground level inside a generated dungeon. Nothing you can see here was generated — the room is a schematic. What was generated is that this room is *here*, at *this* angle, connected to *that* corridor.

Given that constraint, "generate the rooms" is not merely a different technique; it discards the entire asset pipeline. The two standard techniques were considered and both were eliminated:

APPROACH	WHY IT WAS REJECTED
Cellular automata	Produces organic, cave-like negative space. Genuinely good at caves. It cannot produce a *room* — a bounded space with deliberate walls, a doorway in a specific place, and decoration a person authored.
Binary space partitioning	Recursively splits a rectangle into rectangles and connects them with corridors. It produces rooms, but only rectangles of sizes the algorithm chose. A hand-drawn 33×33 boss arena does not fit into a cell the splitter invented.
Socket / jigsaw placement ← chosen	Pieces carry connection points; the engine clamps piece to piece at those points. Piece size is unconstrained, so a 9×25 corridor, a 17×17 hall and a 33×33 arena coexist. This is what Mojang uses for villages, bastions, ancient cities and trial chambers.

4.2 What "socket" means here

A socket in this engine is a **door anchor**: the base-centre block of a door opening, stored as a coordinate relative to the room's origin. A room is a schematic plus a list of these anchors. Placement is: take an unused anchor on an already-placed room, take an anchor on a candidate room, rotate the candidate so the two face each other, and translate it so the anchors line up one block apart.

That is the entire idea. Chapters 6 through 8 are the arithmetic that makes it exact, Chapter 9 is the collision test that free placement makes mandatory, and Chapters 10 through 12 are the policy layer that decides *which* room and *which* anchor.

4.3 The price of the choice

Free placement is not free. A fixed cell grid makes overlap *mathematically impossible*; anchor-based placement makes it not merely possible but likely, so a 3D collision test becomes a required step rather than an optimisation. The original design did use a cell grid inside the dungeon and it was abandoned. Chapter 5 is the postmortem of that decision.

CHAPTER SUMMARY

- The rooms are the product and they are hand-built; the algorithm exists to arrange them.
- Cellular automata give caves, BSP gives rectangles the algorithm chose. Neither accepts a hand-drawn room.
- Socket/jigsaw placement accepts pieces of any size, which is what lets a corridor, a hall and an arena live in one dungeon.
- The cost is that collision testing becomes mandatory.

CHAPTER 5

Two grids, and why conflating them breaks everything

There are two things in this project called a grid. They are unrelated, they solve different problems, and one of them was deleted. Getting them confused is the fastest way to misread the whole engine.

5.1 The instance slot grid — alive and well

Two parties must never share geometry. The solution is a dedicated void world divided into fixed 512-block squares; one live dungeon occupies one square. The mapping from slot index to position is deterministic:

```
public GridSlot slotAt(int index) {
    int col = index % columns;
    int row = index / columns;
    return new GridSlot(index, col * slotSize, baseY, row * slotSize, slotSize);
}
```

GridSlotManager: the entire position calculation.

Two properties follow, and both are used later. Overlap between instances becomes impossible by construction, and — because position is a pure function of the index — Phase 7 only ever has to store the *index* in the database, never a coordinate.

5.2 The layout inside a dungeon — deleted

The original design had a second grid *inside* the slot: rooms would sit on cells of a fixed size. It was abandoned for three reasons, in ascending order of importance.

Rooms would have had to be one size.

A cell grid forces every room to be a cell, or an exact multiple of one. That kills the 9×25 corridor and the 33×33 arena in the same breath.

Rotation would have been searched for rather than computed.

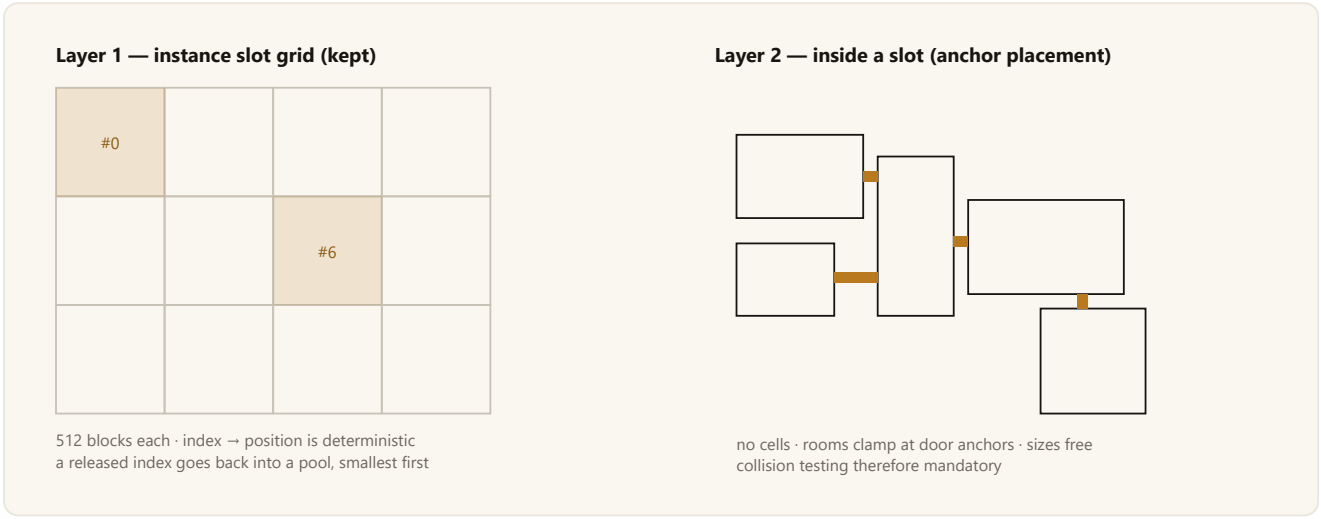
Under a cell scheme the candidate pool gets narrowed — "rooms with a west-facing door" — and then a rotation is looked for. Under anchor placement the required rotation is a single arithmetic expression (Chapter 6), which means **every room is a candidate for every connection**.

Straight lines would have needed a separate fix.

On a cell grid, chaining north repeatedly produces a corridor drawn with a ruler. With anchors, doors sit at arbitrary offsets in their walls, so each connection introduces a lateral shift, and the shifts accumulate into a layout that curves on its own (Chapter 11).

THE RULE THAT VANISHED WITH THE CELL GRID

The old design required room side lengths to be **odd** (17, 33). On an even side there is no true centre block, so the rotation centre shifts by half a block and a rotated room lands one block off its cell. Anchor placement removed the need: placement is computed from the anchor, not from the bounding box, so a room may be asymmetric about its own origin. This was verified with a purpose-built room, `test_even` (10 wide × 16 long, origin at `(5,0,8)`, therefore asymmetric on *both* axes), and all twelve combinations of its three doors against all four directions landed exactly on target. **The map team can be told side lengths are free.**



Two grids. The left one keeps parties apart and still exists. The right one is not a grid at all any more.

5.3 Slot size is a configuration hazard

`dungeon-world.slot-size` defaults to 512 and must exceed the footprint of the largest dungeon the engine can produce. Set it too small and two instances bleed into each other's blocks. Worse, changing it after the world has content shifts the entire slot geometry, so existing structures end up straddling new boundaries. The safe procedure is to change it only on a world that is about to be reset — which, since `reset-on-start` defaults to true, is every restart anyway.

RELEASE() DOES NOT CLEAR BLOCKS

`GridSlotManager.release(index)` is **bookkeeping only**. It returns the index to the free pool; the blocks stay where they are. Clearing is `InstanceManager.close`'s job, and it wipes *then* releases — in that order, for a reason covered in Chapter 21. The raw `/tdungeons free` command still leaves blocks behind, deliberately, because it exists for the manual `paste` and `connect` test commands.

CHAPTER SUMMARY

- Slot grid: 512-block squares, one per instance, index → position deterministic. Kept.
- Cell grid inside a dungeon: deleted. It forced one room size, made rotation a search, and drew straight corridors.
- The odd-side-length rule died with it and was proven unnecessary by `test_even`.
- `release()` is accounting, not cleanup.

CHAPTER 6

Coordinates and the rotation algebra

Four files — `Vec3i`, `Direction`, `Rotation`, `Aabb` — carry the whole coordinate system. They are small, they are pure, and one of them contains a sign that was *measured* before a single line of the placement code was written. This chapter explains why that measurement came first.

6.1 Minecraft's axes and the direction index

```
Minecraft:  +X = East    +Z = South    North = -Z

Direction index:   N=0   E=1   S=2   W=3        (clockwise, viewed from above)

opposite(d)  = (d + 2) mod 4
step(d)      = N:(0,0,-1)   E:(1,0,0)   S:(0,0,1)   W:(-1,0,0)
```

The conventions everything else assumes.

The enum order is **clockwise**, and that is not cosmetic. It reduces the rotation formula `d' = (d + R) mod 4` to plain `ordinal()` arithmetic — which also means that reordering the enum silently breaks every rotation in the engine. The class comment says so explicitly, because "silently" is the operative word.

```java
public enum Direction {

    NORTH(0, 0, -1, "kuzey"),
    EAST(1, 0, 0, "doğu"),
    SOUTH(0, 0, 1, "güney"),
    WEST(-1, 0, 0, "batı");

    /** {@code opposite(d) = (d + 2) mod 4}. */
    public Direction opposite() {
        return VALUES[(ordinal() + 2) & 3];
    }

    public static Direction byIndex(int index) {
        return VALUES[Math.floorMod(index, 4)];
    }
}
```

Direction — the enum body. The clockwise order IS the rotation maths.

6.2 Rotation: four steps, and the sign that had to be measured

`R ∈ {0,1,2,3}` counts 90° clockwise steps. Applying a rotation to a *direction* is index arithmetic; applying it to a *point* is a coordinate swap:

```
Direction:   d' = (d + R) mod 4

Point:       R=0  ->  ( x, y,  z)
```

```
R=1  ->  (-z, y,  x)
R=2  ->  (-x, y, -z)
R=3  ->  ( z, y, -x)
```

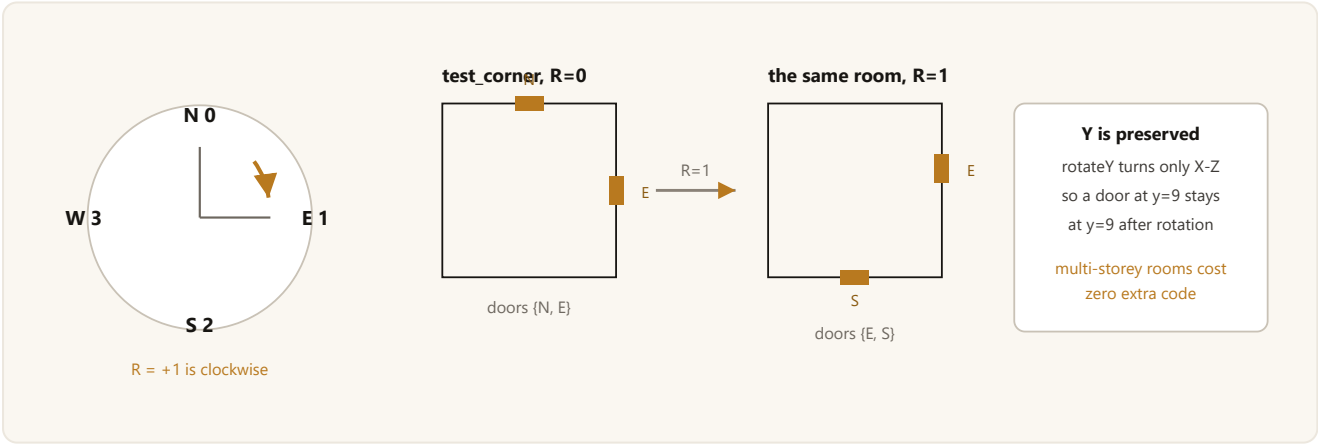
Point rotation. Y is untouched — that is what makes multi-storey rooms free.

Sanity check: the north unit vector is `(0,-1)` in X-Z. Apply `R=1`: `(-(-1), 0) = (1, 0)`, which is East. In the direction index, `0 + 1 = 1`, which is also East. Consistent.

> **WHY THE SIGN WAS MEASURED BEFORE ANY CODE WAS WRITTEN**
>
> `SchematicService` pastes with `new AffineTransform().rotateY(-degrees)` — *negative* degrees, so that the engine's rotation direction matches what WorldEdit's own `//rotate` command does. A mapper who rotates a room in the editor and a dungeon the engine generates must turn the same way.
>
> Had this sign been backwards, **every generated dungeon's doors would have failed to line up, and the failure would have been silent**: rooms still paste, the passage just comes out sealed. So the actual call was executed directly on a bare JVM — no server needed, the transform is pure mathematics — and the results were tabulated.

R	POINT	DIRECTION MAPPING
0	`( x, y, z)`	N→N E→E S→S W→W
1	`(-z, y, x)`	**N→E E→S S→W W→N**
2	`(-x, y, -z)`	N→S E→W S→N W→E
3	`( z, y, -x)`	N→W E→N S→E W→S

Measured 25 August 2026. All four steps matched the point formulas exactly. The competing hypothesis `d' = (d - R) mod 4` was eliminated at R=1 and R=3. The two-door test room `test_corner` (N+E) rotated 90° comes out **E+S** — the expected result.



Clockwise R, and what it does to a two-door room.

6.3 The alignment formula

Here is the payoff of all this bookkeeping. When attaching a child room to a parent's door, the required rotation is **not searched for**. It is one expression:

```java
/**
 * The rotation that brings a child room's door back to back with the parent's door —
 * R = (d_p + 2 - d_c) mod 4.
 *
 * This value is COMPUTED, not searched for. The child's door wall ends up facing the
 * opposite of the parent door's outward direction, so the two walls face each other.
 * That is why the candidate pool never has to be narrowed to "rooms with a door facing
 * this way": every room is a candidate for every connection, and rotation closes the gap.
 */
public static Rotation align(Direction parentOutward, Direction childWall) {
    return ofSteps(parentOutward.index() + 2 - childWall.index());
}
```

Rotation.align — the single most consequential line in the generation package.

Read it as: turn the child until its door wall points at `opposite(d_p)`. An equivalent implementation is a loop that adds 90° up to four times until the condition holds; the closed form is the same answer without the loop.

> **WHY THIS IS THE LOAD-BEARING DECISION**
>
> Because rotation is computed rather than searched, **every template is a candidate for every connection**. Under the abandoned cell design, the pool was filtered down to rooms that happened to have a door facing the right way, which both reduced variety and made the available variety depend on what the map team happened to draw. Here, a room with a single north door can serve a connection facing any of the four directions.

6.4 Aabb: inclusive on both ends, and three-dimensional

`Aabb` is an axis-aligned box with **both ends inclusive** — it describes blocks, not a half-open range. That choice drives the comparison operator in the intersection test:

```java
/**
 * Because the ends are inclusive the comparison is {@code <=}: a box with maxX=10 and one
 * with minX=10 SHARE the x=10 column, so they intersect. This is why rooms connected back
 * to back do not collide — there is exactly one block of clearance between them.
 */
public boolean intersects(Aabb other) {
    return minX <= other.maxX && maxX >= other.minX
            && minY <= other.maxY && maxY >= other.minY
            && minZ <= other.maxZ && maxZ >= other.minZ;
}
```

Aabb.intersects. The <= is a consequence of inclusive ends.

The box is three-dimensional and that is deliberate. A 2D footprint test would be cheaper and would be *wrong*: in a multi-storey dungeon, two rooms can overlap in footprint while their volumes do not — an upper room passing over a lower one. A footprint test would reject precisely the arrangement the architecture was designed to permit.

Three helper operations matter later:

`rotate(Rotation)`
: Rotates the two corners and retakes min/max. That is why an asymmetric room rotates correctly: the asymmetry lands in the box, and the box is computed *after* rotation.

`union(Aabb)`

The smallest box containing both. Phase 2 clears an instance using the union of its placed rooms — the exact volume the generator wrote to. Clearing the whole slot instead would be roughly two orders of magnitude more blocks for the same result.

`clampTo(Aabb)`

Intersection, or `null` if they do not overlap. Used as a second lock on the cleanup volume so that a room which somehow overran its slot cannot cause the wipe to overrun it too.

CHAPTER SUMMARY

- `Direction`'s clockwise enum order is part of the arithmetic; reordering it breaks rotation silently.
- `R=+1` is clockwise. Measured against WorldEdit's own transform before the placement code existed, because the failure mode would have been silent.
- Y is preserved by rotation, which makes multi-storey rooms free.
- `align()` computes the rotation instead of searching for it — that is what makes every room a candidate for every connection.
- `Aabb` is inclusive and 3D. 2D would forbid the multi-storey case on purpose.

CHAPTER 7

The door anchor, and deriving which wall it is in

The door anchor is the whole interface between the map team and the engine. It is also, in the author's own words, "the one place these systems break". This chapter covers what an anchor is, what is deliberately *not* stored alongside it, and the formula that catches the trap hiding in the obvious approach.

7.1 What an anchor is

The anchor is the **base-centre block of the door opening**, written as a local coordinate relative to the room's origin. That is all. Two consequences follow and both are important.

Facing is not stored — it is derived

There is no `facing: NORTH` field in the metadata. The anchor vector is already a delta from the origin, so the wall can be computed from it. This leaves nowhere for the inconsistency "metadata says north but the anchor is in the east wall" to arise.

> **THE PRINCIPLE, STATED ONCE AND APPLIED EVERYWHERE**
>
> **Every field that can be written is a field that can be written wrong.** The map team will hand-author metadata for forty-plus rooms. Anything derivable is derived: the wall from the anchor, the bounding box from the schematic, the door opening's size and material by measurement (Chapter 14), and the mob spawn points at runtime (Chapter 30). What is left for a human to write is one line per door.

The door need not be in the middle of the wall

Because the anchor is written explicitly, a door can sit anywhere along a wall, and one wall can carry several doors. This is not a minor flexibility — it is the engine driving Chapter 11's "how the layout stops being a straight line".

```
public record DoorAnchor(int index, Vec3i local, Direction wall) {

    /** Builds the door by deriving its wall from the anchor. */
    public static DoorAnchor of(int index, Vec3i local, Aabb localBox) {
        return new DoorAnchor(index, local, Direction.ofAnchor(local, localBox));
    }
}
```

DoorAnchor. Note that `wall` is a derived field, filled in by the factory method.

`index` is the door's position in the room's list, and the specification is explicit that it is an **address, not a fill order**. Geometry decides which door gets filled; the index only tracks which one was connected and which one needs plugging.

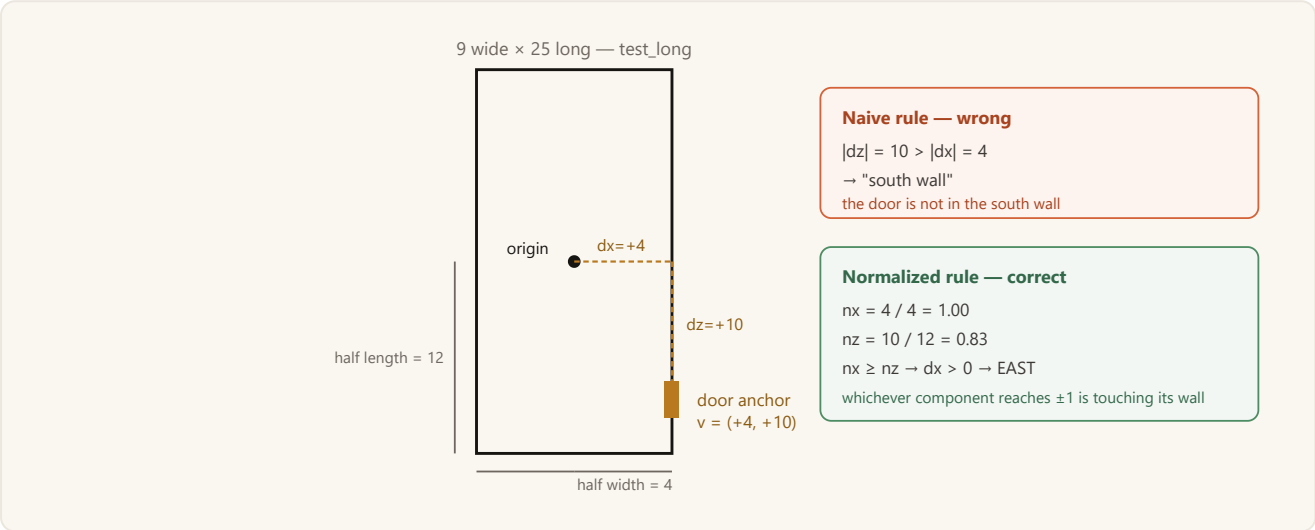
7.2 The trap: the naive wall rule

Given an anchor `v = (dx, dy, dz)`, the obvious rule is "whichever horizontal component is larger in absolute value names the axis". In a square room this is correct. In a rectangular room it is wrong, and the failure is geometric rather than random — it happens reliably in one specific, common situation.

```
v = (dx = +4, dz = +11)

naive:   |dz| > |dx|   ->   "south wall"        <- WRONG, the door is in the east wall
```

A 9-wide x 25-long corridor. Door in the EAST wall, near the south end.



`test_long` exists in the test set purely to catch this. If the formula regresses, this room fails immediately.

7.3 The correct rule: normalize against half-extents

Normalize each component against the room's extent *in the direction that component points*. Whichever normalized value reaches ±1 is the wall the point is touching.

```
public static Direction ofAnchor(Vec3i local, Aabb localBox) {
    double nx = ratio(local.x(), localBox.maxX(), -localBox.minX());
    double nz = ratio(local.z(), localBox.maxZ(), -localBox.minZ());

    if (nx == 0.0 && nz == 0.0) {
        throw new IllegalArgumentException(
                "Door anchor coincides with the room origin — no wall can be derived: " + local);
    }
    // X wins ties. The rule is fixed so that a corner anchor never leaves the choice ambiguous.
    if (nx >= nz) {
        return local.x() > 0 ? EAST : WEST;
    }
    return local.z() > 0 ? SOUTH : NORTH;
}

/**
 * The component's ratio against the extent in the direction it points.
 * If the extent is 0 (the origin sits right on that edge) the ratio counts as infinite:
 * the point is already on that wall.
```

```
     */
    private static double ratio(int delta, int positiveExtent, int negativeExtent) {
        if (delta == 0) {
            return 0.0;
        }
        int extent = delta > 0 ? positiveExtent : negativeExtent;
        if (extent <= 0) {
            return Double.POSITIVE_INFINITY;
        }
        return Math.abs(delta) / (double) extent;
    }
```

Direction.ofAnchor — the §4 calculation.

Three details in that code are worth pausing on.

Normalization is per direction, not per axis.

East and west extents are read separately (`maxX` and `-minX`). Since the odd-side-length rule was dropped, the origin no longer has to sit at the room's exact middle, so the room may be asymmetric about it. A single "half width" value would give the wrong answer for an asymmetric room.

Ties go to X, deliberately.

An anchor in a corner reaches ±1 on both axes. Leaving that ambiguous would make placement depend on floating-point noise. The rule is fixed rather than clever.

A zero extent means infinite ratio.

If the origin sits exactly on an edge, the extent in that direction is 0 and dividing would be meaningless. The point is already on that wall, so the ratio is treated as infinite.

> **WHY THIS BUG CLASS IS DANGEROUS**
>
> In a square room the naive and correct rules give the same answer. The bug is therefore **invisible in test rooms** and only appears when the first rectangular room arrives — which, in a project whose room set is built in a separate phase by a separate team, could be months later. That is why `test_long` is a permanent fixture rather than a one-off check.

7.4 A real edge case that was caught

While building the test rooms, `test_long`'s east door was placed at offset `+11`, which pushed a 3-block opening onto the wall's corner block — in a 25-long wall, the last valid centre for a 3-wide opening is 22. A bounds check inside `carveDoor` made this fail loudly at `/tdungeons gen` time rather than producing a subtly broken room, and the offset was pulled back to `+10`.

CHAPTER SUMMARY

- An anchor is the base-centre block of the opening, in local coordinates. Facing is derived, never stored.
- The door index is an address, not a fill order.
- The naive `|dx| > |dz|` rule is correct only in square rooms; the correct rule normalizes each component against the extent in its own direction.
- Normalization is per direction because rooms may be asymmetric about their origin.
- `test_long` is a permanent regression test for exactly this.

CHAPTER 8

The placement formula: back to back

Everything in the last two chapters exists to serve five lines of arithmetic. Given a parent room's open door, this chapter computes exactly where a candidate room sits and at what angle — and explains the one-block gap that is the difference between debuggable output and output that depends on paste order.

8.1 Inputs

```
A_p  =  the parent door's anchor, WORLD coordinate
d_p  =  that door's outward facing, in the WORLD frame
        (the parent's own rotation has already been applied)
```

Two values, both already in world coordinates.

8.2 The four steps

Step 1 — choose a candidate

A candidate is a **pair**: `(room template × one of its doors)`. The rotation turns on the pair, not on the template alone. A room with three doors can be seated three different ways against the same parent door, and which door it arrives through determines where its *other* doors will point. That is where layout variety comes from.

Step 2 — compute the rotation

```
v_c  =  the child door's LOCAL anchor
d_c  =  wall(v_c)                      // Chapter 7
R    =  (d_p + 2 - d_c) mod 4
```

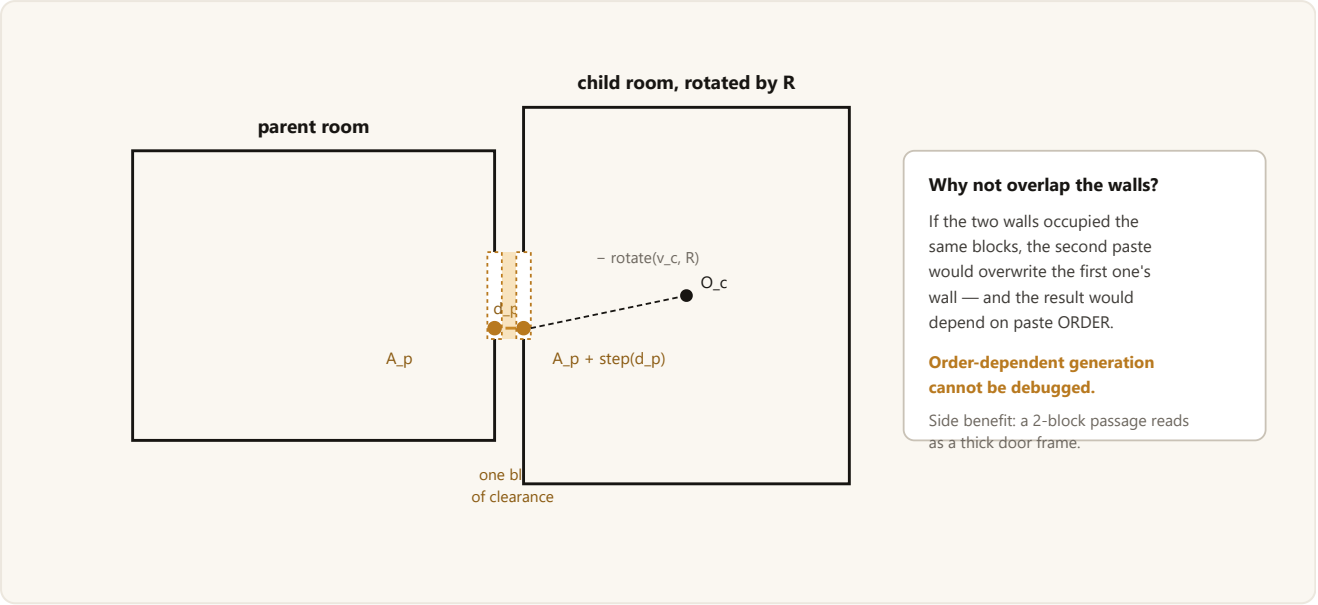
Not searched. One expression.

Step 3 — compute the position

```
O_c  =  A_p + step(d_p) - rotate(v_c, R)
```

The back-to-back convention.

After placement the child's door anchor sits at `A_p + step(d_p)`: exactly one block outside the parent's anchor. The two walls end up back to back, both have a hole in them, and the passage is open.



The back-to-back convention. No room ever writes a block belonging to another room.

Step 4 — Y aligns itself

Because rotation preserves Y, `O_c.y = A_p.y − v_c.y` falls out of the same expression. The child's floor lands wherever it must for its door's Y to coincide with the parent's door's Y. A room entered from below and continued above needs no extra code at all — the anchor's Y does the work.

8.3 The code

```
public PlacedRoom attachTo(int doorIndex, Vec3i parentAnchor, Direction parentOutward) {
    DoorAnchor door = door(doorIndex);
    Rotation rotation = Rotation.align(parentOutward, door.wall());

    // Back to back: the child's door anchor sits exactly one block outside the parent's.
    // If the walls overlapped, the second paste would overwrite the first one's wall and
    // the result would depend on paste ORDER — and order-dependent generation cannot be
    // debugged.
    Vec3i childAnchorWorld = parentAnchor.plus(parentOutward.step());
    Vec3i origin = childAnchorWorld.minus(rotation.apply(door.local()));

    return PlacedRoom.of(this, rotation, origin);
}
```

RoomTemplate.attachTo — steps 2 through 4 in one method.

Note what this method does *not* do: it never asks whether the room fits. It answers "where does it seat", not "may it seat". The separation is what keeps the placement mathematics testable without a world and without a list of already-placed rooms.

8.4 PlacedRoom: geometry, and only geometry

The result is a `PlacedRoom` — template, rotation, world origin, and the world-space bounding box. It is immutable and it deliberately carries no graph state:

```java
public record PlacedRoom(RoomTemplate template, Rotation rotation, Vec3i origin, Aabb bounds) {

    public static PlacedRoom of(RoomTemplate template, Rotation rotation, Vec3i origin) {
        Aabb bounds = template.localBox().rotate(rotation).translate(origin);
        return new PlacedRoom(template, rotation, origin, bounds);
    }

    /** The door anchor's WORLD coordinate. */
    public Vec3i doorAnchor(int index) {
        return origin.plus(rotation.apply(template.door(index).local()));
    }

    /** The door's outward facing in the WORLD frame — d' = (d + R) mod 4. */
    public Direction doorOutward(int index) {
        return rotation.apply(template.door(index).wall());
    }

    /** Where the anchor of the child door attaching here will land. */
    public Vec3i doorMate(int index) {
        return doorAnchor(index).plus(doorOutward(index).step());
    }
}
```

PlacedRoom. The world box is computed by rotating the local box and then translating.

> **WHY DOOR STATE IS NOT IN PLACEDROOM**
>
> Geometry is computed *before* a placement is accepted — the collision test needs the box. Whether a door is connected, open or dead only comes into existence *after* acceptance, and changes over time. Merging them would create a "half-built room" state in which a rejected candidate nevertheless has door states. `LayoutNode` (Chapter 9) is where that state lives.

8.5 How this was verified

Headlessly: 53 assertions in `GeoProbe`, including rotation round-trips, all 16 combinations of `align`, wall derivation on square, rectangular and asymmetric rooms, and 5 rooms × every door × every direction = 48 placements.

On a server: 23 block checks with `execute if block` across two connection scenarios, confirming that the passage is open, the walls stand back to back, the space outside the door stays solid, and rotation moves the room's *other* doors to the computed positions. The values the plugin reported for R, origin and box matched values hand-computed from the specification exactly — for instance `rot=270` giving origin `(273,64,256)`, and `rot=90` giving origin `(771,64,243)` with a 10×8×16 box becoming 16×8×10.

CHAPTER SUMMARY

- A candidate is a `(template × door)` pair, which is where variety comes from.
- `R = (d_p + 2 − d_c) mod 4`, then `O_c = A_p + step(d_p) − rotate(v_c, R)`.
- One block of clearance: no room ever writes another room's blocks, so output does not depend on paste order.
- Y alignment is free because rotation preserves Y.
- `attachTo` answers "where", never "whether" — that split is what keeps the maths testable.

CHAPTER 9

Collision, bounds and self-validation

This is the price of free placement, paid in full. A cell grid made overlap impossible; anchor placement makes it likely. `DungeonLayout` is the class that says no — and, separately, the class that can be asked to prove the finished layout is sound.

9.1 Two rejection reasons, both needed

```
public String rejectReason(Aabb candidate) {
    if (!contains(bounds, candidate)) {
        return "exceeds the slot bounds";
    }
    for (LayoutNode node : nodes) {
        if (node.bounds().intersects(candidate)) {
            return "collides with room#" + node.id() + " (" + node.template().name() + ")";
        }
    }
    return null;
}
```

DungeonLayout.rejectReason — returns null when the candidate fits.

Exceeding the slot bounds

The room would reach into a neighbouring instance's blocks. Since slots are handed out to different parties, that is a correctness failure, not an aesthetic one.

Intersecting a placed room

The dungeon would break against itself — two rooms writing the same blocks, with the result depending on which pasted last.

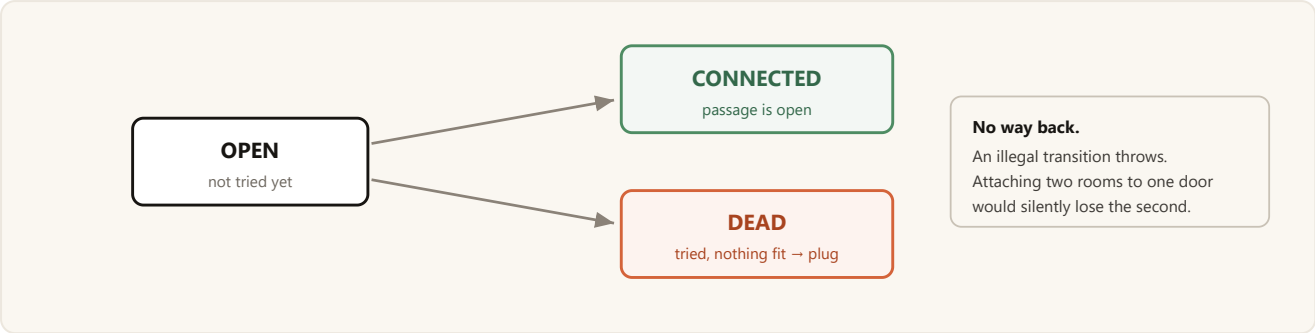
Returning a *reason* rather than a boolean matters more than it looks. When a door ends up DEAD, the last rejection reason is what gets reported, so the operator learns whether the dungeon ran out of space or ran into itself. `fits()` exists as a thin boolean wrapper for call sites that do not care.

9.2 Brute force is the right algorithm here

The collision test is a linear scan over the placed rooms. The largest dungeon is twenty rooms, so a new candidate costs at most twenty box comparisons. A spatial hash or an octree would be pointless complexity — the hash computation alone would cost more than the scan it replaces. The class comment says this outright so that a future reader does not "optimise" it.

9.3 LayoutNode: door state, and why transitions throw

A node is a `PlacedRoom` plus per-door graph state. Three states, and the transitions are strictly one-way.



Door state transitions. Both OPEN doors that were never tried and DEAD doors that were tried and failed get plugged in Chapter 14 — the graph does not distinguish them at plug time, only in the report.

```
/**
 * State transitions are one-way: OPEN → CONNECTED or OPEN → DEAD. There is no way back.
 * Attaching two rooms to the same door would silently lose the second one, so we throw.
 */
private void require(int index, DoorState expected) {
    if (doorStates[index] != expected) {
        throw new IllegalStateException("room#" + id + " door#" + index + " is "
                + doorStates[index] + ", expected " + expected
                + " (" + template().name() + ")");
    }
}
```

LayoutNode: the guard that turns a silent bug into a stack trace.

`LayoutNode` also carries `depth` — how many rooms away from the entrance it is. Phase 1C does not use it. It is stored so that 1D can guarantee critical-path length (Chapter 12) and, later, so that Phase 3B can scale mob difficulty by relative depth (Chapter 31). A field added one phase early, on purpose, because computing it retroactively would mean walking the graph.

9.4 Links are always recorded in both directions

```
/** Links two doors to each other — a link is always recorded in both directions. */
public void link(int nodeA, int doorA, int nodeB, int doorB) {
    nodes.get(nodeA).link(doorA, nodeB, doorB);
    nodes.get(nodeB).link(doorB, nodeA, doorA);
}
```

DungeonLayout.link. A one-sided link would make the graph traversal lie.

9.5 addRoot throws rather than returning false

Placing the entrance room is the one placement where failure cannot be tolerated quietly: if it does not fit, there is no dungeon at all. So `addRoot` throws, with a message that names the room, its size and the slot size — because the realistic cause is a configuration mistake (a boss-sized entrance room, or a `slot-size` that was lowered).

```
throw new IllegalStateException("Entrance room '" + template.name()
        + "' does not fit the slot: " + reason + ". Room is " + template.describeSize()
```

```
        + ", slot is " + bounds.sizeX() + "×" + bounds.sizeZ() + ".");
```

The error message is written for the operator, not for the developer.

9.6 validate(): the invariant checker that never runs in production

`DungeonLayout.validate()` is **not called on the generation path**. It exists to answer a different question: when something is wrong, *which* invariant broke? It returns a list of problems; an empty list means the layout is consistent. Four classes of problem are checked.

INVARIANT	HOW IT IS CHECKED
No room leaves the slot	Each node's box against the layout bounds
No two rooms intersect	Every pair, once (the `j = i+1` loop)
Every connected passage is aligned	For each CONNECTED door, the far room must have a door linking back *and* sitting exactly at `anchor + step(outward)`
The graph is connected	Breadth-first from node 0 following door links; the reached count must equal the node count

```
// Back to back: the two anchors must be exactly one block apart and face EACH OTHER.
LayoutNode other = nodes.get(otherId);
Vec3i mine = a.room().doorAnchor(d);
Vec3i expected = mine.plus(a.room().doorOutward(d).step());
boolean matched = false;
for (int e = 0; e < other.doorCount(); e++) {
    if (other.linkedNode(e) == a.id() && other.room().doorAnchor(e).equals(expected)) {
        matched = true;
        break;
    }
}
if (!matched) {
    problems.add("the passage between room#" + a.id() + " door#" + d
            + " and room#" + otherId + " is misaligned (expected " + expected + ")");
}
```

The passage alignment check. This is the one that catches a broken rotation sign.

> **THE PRINCIPLE BEHIND HAVING THIS AT ALL**
>
> **In procedural generation the most expensive failure is broken output being accepted silently.** A dungeon with one misaligned passage still generates, still pastes, and still looks fine from outside. A player walks into a wall where a door should be, reports it as "the dungeon was broken", and there is nothing to go on. `validate()` converts that into a sentence naming two rooms and a coordinate.

9.7 What 1C verified

- **29 headless assertions** (`GenProbe`): weight distribution over 200,000 draws with deviation under 0.3%, the pool filter, drawing without replacement, 500 seeds with zero inconsistency, zero overflow in a

deliberately narrow slot, DEAD marking, and reproducibility.

- **13 block checks on a server**: passages open in a generated dungeon, walls back to back, a rotated room's ceiling light in the right place, and — importantly — a DEAD door's opening still standing, because plugging was 1D's job.
- The server's `weights` output matched `GenProbe`'s measured distribution exactly.
- Three seeds × 12 rooms generated on a server; all three came back with a clean `validate()`.

CHAPTER SUMMARY

- Two rejection reasons — outside the slot, or intersecting a placed room — and the reason text is carried through to the operator.
- Linear scan is correct at twenty rooms; the comment says so to prevent future "optimisation".
- Door states are one-way and illegal transitions throw.
- `validate()` never runs in production; it exists so that a failure names the broken invariant instead of being a player complaint.

CHAPTER 10

Candidate selection and who owns the weight

Every room template carries a `weight` that a server owner edits in YAML. This chapter is about a decision that looks like a detail and is not: does that weight belong to the *template*, or to the `(template × door)` pair the placer actually draws? Getting it wrong does not merely shift the distribution — it **inverts the ordering the mapper wrote**.

10.1 Two-stage selection

```
1) Draw a TEMPLATE      — weighted random, removed from the pool (no replacement)
2) Choose a DOOR        — that template's doors are ordered, then tried one by one
3) If none fit          — the template is out of the pool; go back to 1
4) If the pool empties  — the door is marked DEAD  ->  it gets plugged
```

The selection procedure, in full.

Stage one is weighted. Stage two is not — it is a shuffle with a bias applied afterwards (Chapter 11). Keeping the weight strictly in stage one is the entire point.

10.2 The measurement that settled it

A candidate is a pair, so a four-door room produces four candidates. If the weight belonged to the pair, that room would count its weight **four times**. Here is what that does to the actual test room set:

ROOM	DOORS	WEIGHT	IF WEIGHT WERE ON THE PAIR	WEIGHT ON THE TEMPLATE (CHOSEN)
`test_corridor`	2	**150**	21.4%	**25.4%**
`test_corner`	2	120	17.1%	20.3%
`test_cross`	4	100	**28.6%**	16.9%
`test_even`	3	80	17.1%	13.6%
`test_long`	2	80	11.4%	13.6%
`test_deadend`	1	60	**4.3%**	10.2%

The mapper gave the corridor 150 and the cross 100. Under pair ownership the cross (28.6%) would *outrank* the corridor (21.4%). The configuration would mean the opposite of what it says.

> **AND THE DISTORTION COMPOUNDS**
>
> The more multi-door rooms get placed, the more open doors appear; every open door again favours multi-door rooms. In a twenty-room dungeon this is not a one-off skew, it is a growing one.

The user-facing argument, which is the decisive one

`weight` is edited by server owners in a YAML file. The expectation "if I write 200 it comes up twice as often as 100" has to hold. Door count silently overriding it would be a bug nobody could diagnose, **because the cause is written down nowhere** — a room's door count is a property of the schematic, not of the configuration file the owner is looking at.

The counter-argument, and why it does not win

A four-door room genuinely *is* more useful to the engine: it answers more geometric situations and it keeps the graph growing. That is true. But it is a **separate concern** and must not be conflated with the mapper's weight. If branching should be encouraged, that gets its own explicit, switchable configuration knob. The turn bias in Chapter 11 follows exactly this pattern: it is a coefficient in the candidate *ordering*, not something buried inside the weight.

10.3 The draw itself

```java
/**
 * Draws a template by weight and REMOVES it from the pool (no replacement).
 *
 * No replacement is essential for backing off: if every door of a template collides, that
 * template must be out, otherwise we would keep drawing the same candidate and retrying
 * forever.
 *
 * @param pool a MUTABLE copy of the pool being worked on
 */
public static RoomTemplate drawWeighted(List<RoomTemplate> pool, RandomGenerator random) {
    if (pool.isEmpty()) {
        return null;
    }
    long total = 0;
    for (RoomTemplate t : pool) {
        total += t.weight();
    }
    // RoomMetadata already guarantees a positive weight; this is defence in depth.
    if (total <= 0) {
        return pool.remove(random.nextInt(pool.size()));
    }
    long roll = random.nextLong(total);
    long cumulative = 0;
    for (int i = 0; i < pool.size(); i++) {
        cumulative += pool.get(i).weight();
        if (roll < cumulative) {
            return pool.remove(i);
        }
    }
    return pool.remove(pool.size() - 1);   // integer maths, so this should be unreachable
}
```

RoomLibrary.drawWeighted — cumulative scan, and removal from the pool.

TEMPLATE ORDER AFFECTS GENERATION

`drawWeighted` scans cumulative weights, so **the same seed with a different template order produces a different dungeon**. On a server the order comes from `SchematicService.list()`, which sorts filenames — so it is alphabetical and stable. The practical consequence: *when the map team adds a new room, every seed's output changes.* That is expected behaviour, but it is also the answer to "why did the same seed give me a different dungeon". `scripts/geo-probe/Rooms.java` keeps its rooms in alphabetical order for the same reason — so the probes can predict what the server will build.

10.4 Three pools, not one

`RoomLibrary` partitions the templates on construction. Rooms with no doors are discarded outright — they cannot be attached to anything, and keeping them would only produce wasted attempts.

POOL	CONTENTS	USED BY
`normalPool()`	Every `NORMAL` template with at least one door	Side branches (Chapter 12)
`branchingPool()`	`normalPool()` minus the single-door rooms	The critical path (Chapter 12)
`entrances()` / `bosses()`	Templates typed `entrance` / `boss`	Assigned, never drawn from the normal pool

WHY ENTRANCE AND BOSS ARE EXCLUDED FROM THE NORMAL POOL

Both are *assigned* during graph generation, not selected. If a boss template stayed in the ordinary pool, a boss room could materialise in the middle of a dungeon — and the room that is supposed to be the destination becomes a room you pass through.

The library also knows how to explain itself when it cannot work, which matters on a fresh installation:

```
public String describeProblem() {
    if (isUsable()) {
        return null;
    }
    if (all.isEmpty()) {
        return "no room templates loaded (use /tdungeons gen to create test rooms)";
    }
    long doorless = all.stream().filter(t -> t.doorCount() == 0).count();
    return "no 'normal' room with doors — " + all.size() + " templates loaded, "
            + doorless + " of them doorless, the rest entrance/boss";
}
```

RoomLibrary.describeProblem — the message an operator actually sees.

CHAPTER SUMMARY

- Selection is two-stage: weighted template draw, then door ordering.
- Weight belongs to the *template*. On the pair it would invert the ordering the mapper wrote, and the distortion compounds across a dungeon.
- Draws are without replacement so that backing off terminates.
- Template order affects output; it is alphabetical and stable, and adding a room changes every seed.
- Three pools: normal, branching (no single-door rooms), and the assigned entrance/boss sets.

CHAPTER 11

Backing off, dead doors and the turn bias

`RoomPlacer` is where selection, geometry and collision meet. It answers "may it seat, and which room should be chosen" — and it contains the mechanism that stops a generated dungeon from looking like it was drawn with a ruler.

11.1 fill(): the whole attempt loop

```
public Attempt fill(DungeonLayout layout, OpenDoor door, List<RoomTemplate> pool) {
    List<RoomTemplate> remaining = new ArrayList<>(pool);
    LayoutNode parent = layout.node(door.nodeId());
    int tried = 0;
    String lastReason = "candidate pool was empty";

    RoomTemplate template;
    while ((template = RoomLibrary.drawWeighted(remaining, random)) != null) {
        for (int childDoor : orderDoors(template, door.outward())) {
            tried++;
            PlacedRoom candidate = template.attachTo(childDoor, door.anchor(), door.outward());
            String reason = layout.rejectReason(candidate.bounds());
            if (reason != null) {
                lastReason = template.name() + " door#" + childDoor + ": " + reason;
                continue;
            }
            LayoutNode child = layout.add(candidate, parent.depth() + 1);
            layout.link(parent.id(), door.doorIndex(), child.id(), childDoor);
            return new Attempt(child, childDoor, tried, null);
        }
    }

    layout.markDead(door.nodeId(), door.doorIndex());
    return new Attempt(null, -1, tried, lastReason);
}
```

RoomPlacer.fill. Note that the pool passed in is never mutated — a copy is made.

Three properties of this loop are deliberate:

The caller's pool is copied.

Draws remove elements, so operating on the caller's list would silently shrink the library across calls.

The *first* candidate that fits wins.

There is no scoring pass over all candidates. The randomness is in the draw order, not in a fitness comparison — which keeps generation cheap and reproducible.

A failed door is marked DEAD and never retried.

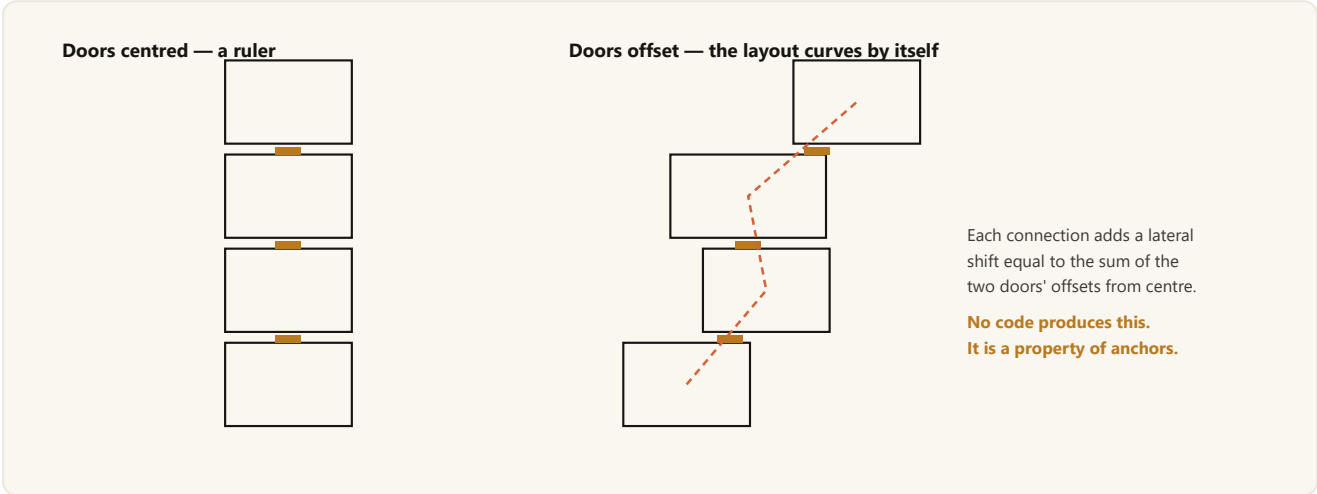
Retrying later would mean the outcome depends on how many rooms were placed in between, which is a different kind of order dependence from the one Chapter 8 eliminated, and just as hard to debug.

Scale is not a concern. The largest dungeon is twenty rooms and the pool is a few dozen templates; the worst case is a few hundred box comparisons for an entire dungeon.

11.2 Two mechanisms break up straight lines

Door offsets — the real engine

If every door sat in the middle of its wall, rooms chained northwards would line up like beads on a wire. They do not, because anchors are arbitrary. If room A's north door is 5 blocks right of centre and room B's south door is 3 blocks left of centre, B seats **8 blocks laterally offset** from A. Those offsets accumulate along a chain and the layout curves on its own, with no code doing anything about it.



Why abandoning the cell grid paid for itself twice: free room sizes, and free meander.

Turn bias — the explicit knob

On top of that, options that continue in the parent's direction are penalised. A door option counts as **straight** when, connecting through it, some *other* door of the room would point along the parent's outward direction — meaning the chain would carry on in the same heading.

```
private List<Integer> orderDoors(RoomTemplate template, Direction parentOutward) {
    int count = template.doorCount();
    List<Integer> order = new ArrayList<>(count);
    for (int i = 0; i < count; i++) {
        order.add(i);
    }
    Collections.shuffle(order, shuffleRandom);

    if (turnBias <= 1.0 || count < 2) {
        return order;
    }
    // Stable partition: turning options in front, straight ones behind; the shuffled order
    // is preserved within each group.
    List<Integer> turning = new ArrayList<>(count);
    List<Integer> straight = new ArrayList<>(count);
    for (int index : order) {
        (continuesStraight(template, index, parentOutward) ? straight : turning).add(index);
    }
    // The penalty is not absolute: the larger turnBias gets, the smaller the chance a
    // straight option jumps back to the front.
    if (!turning.isEmpty() && !straight.isEmpty() && random.nextDouble() < 1.0 / turnBias) {
        turning.addAll(0, straight);
        return turning;
    }
```

```
    turning.addAll(straight);
    return turning;
}
```

RoomPlacer.orderDoors — shuffle, then a stable partition.

The default is `generation.turn-bias: 2.0`, which means a straight option still jumps to the front half the time. `1.0` disables the penalty entirely; larger values make turns dominate.

> **WHY THE BIAS LIVES IN DOOR SELECTION, NOT TEMPLATE SELECTION**
>
> Because touching the template stage would corrupt the meaning of `weight` — the thing Chapter 10 just went to some trouble to protect. Which door the child connects through determines its orientation, and therefore where its remaining doors will point. The bias belongs there.
>
> In a room with two opposite doors — a corridor — *both* options are straight, so the penalty has no effect. That is an honest outcome rather than a bug: that room continues straight no matter what you do to the ordering. The real mechanism is the door offsets above.

11.3 One shuffle source, kept for a reason

```
/**
 * Collections.shuffle wants the legacy java.util.Random. A single instance is kept:
 * creating a new one per call would give different results from the same seed and make
 * generation non-reproducible — which would make it undebuggable.
 */
private final java.util.Random shuffleRandom;
```

A small detail with a large consequence.

11.4 What a DEAD door means downstream

A DEAD door is a door that was tried and for which every candidate either collided or overran the slot. It is not an error. The graph carries on; the door gets sealed by `DoorPlugger` at the end (Chapter 14). From the player's perspective it is indistinguishable from a wall — which is precisely the intent.

The distinction between OPEN and DEAD is preserved in the plug target record only for reporting. An OPEN door is one the side-branch quota never reached; a DEAD one is one that was tried and failed. If a dungeon comes out sparse, the ratio between those two numbers tells you whether the generator ran out of quota or ran out of space.

CHAPTER SUMMARY

- `fill()` draws templates until one of its doors seats; first fit wins.
- A failed door becomes DEAD permanently — retrying later would reintroduce order dependence.
- Door offsets are what actually breaks up straight lines; the turn bias is a secondary, configurable nudge applied at door-selection time.
- The shuffle source is a single instance so that a seed reproduces exactly.

CHAPTER 12

Graph generation: path first, boss last

This is the chapter where the engine stops being geometry and starts being game design. `DungeonGenerator` decides how long the dungeon is, guarantees that length, grows side branches off it, and only then places the boss. The *ordering* of those steps was chosen by measurement, and changing it back reintroduces two specific, measured bugs.

12.1 Size, room count, path length

SIZE	TARGET ROOMS	CRITICAL PATH LENGTH
`small`	3–6	2–4
`medium`	7–12	5–8
`large`	13–20	8–13

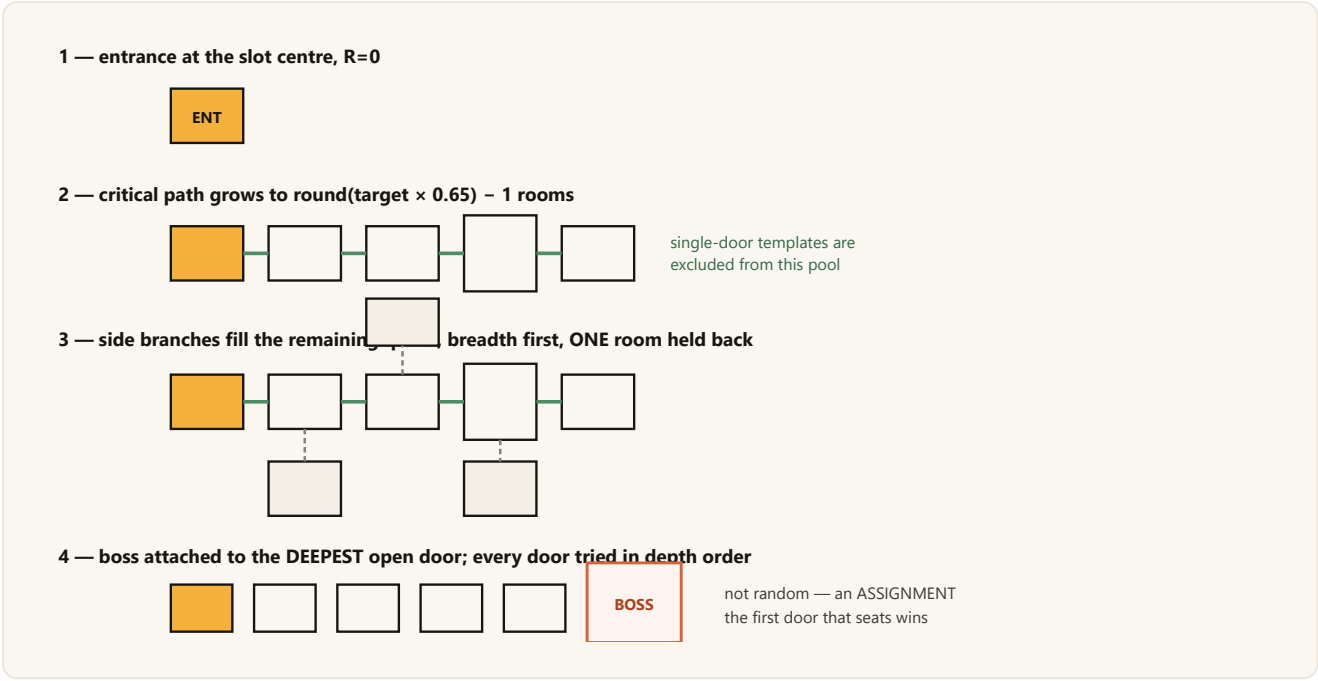
```java
/** Draws a target room count for this size. */
public int pickRoomCount(RandomGenerator random) {
    return minRooms + random.nextInt(maxRooms - minRooms + 1);
}

/**
 * The critical path's target length: round(target × 0.65), minimum 2.
 *
 * The length INCLUDES the entrance and the boss. The minimum of 2 guarantees they are
 * always separate rooms; at 1 the boss would be the entrance itself.
 *
 * The 65% ratio: roughly two thirds of the rooms sit on the mandatory path and one third on
 * side branches. Give side branches too much and the dungeon feels wide but shallow; too
 * little and it becomes a single corridor.
 */
public static int criticalPathLength(int targetRooms) {
    return Math.max(2, Math.round(targetRooms * 0.65f));
}
```

DungeonSize — the count is drawn from a range, the path is a function of it.

The room count is drawn rather than fixed so that two players choosing "medium" do not get dungeons of identical length. The range is kept narrow so the word "medium" keeps meaning something.

12.2 The five phases of generation



Phases 1–4. Phase 5 is the retry: if the path fell short, the entire attempt is discarded.

12.3 Why the path is built first

Scatter rooms at random and then declare the furthest one the boss, and you have no control over how far the boss is from the entrance. Some dungeons would end after two rooms and others after fifteen. Building the path first **puts a floor under playtime**. The slogan for the whole design is one line: *randomness for variety, skeleton for guarantees.*

12.4 The branching-process measurement

Phase 1C used a naive strategy — fill every open door until you reach the target. It does not reach the target. Over 2,000 seeds with a 12-room goal in a 512 slot:

MEASUREMENT	RESULT
Reached the target	**70.8%**
Stalled	584 runs
… of which actually had a DEAD door	only 81
Stopped at exactly **2 rooms**	**204** (10.2%)

So **86% of the stalls were not collisions**. They were the door frontier running dry. This is a branching process going extinct: each placement consumes one door and adds `(doors - 1)`, a net change of `(doors - 2)`. With

the test room set the expected net change is `+0.373` per room — positive, but starting from a single-door root the probability of early extinction is high.

> **THE 10.2% IS NOT A COINCIDENCE**
>
> `test_deadend`'s draw probability is 10.17%. The root, `test_entrance`, has one door; if the first template drawn is a dead end, the frontier is empty immediately and the dungeon ends at two rooms. The number in the measurement *is* the draw probability.

The fix is in the pool, not in the room set

"Require the entrance room to have at least two doors" is the intuitive fix, and the measurement shows it is the wrong diagnosis. Over 3,000 seeds:

ENTRANCE DOORS	NORMAL POOL	REACHED THE TARGET
1	full	70.0%
1	**single-door rooms filtered out**	**97.3%**
2	full	92.1%
2	single-door rooms filtered out	99.8%
4	full	99.2%

The pool filter alone suffices even with a one-door entrance. A two-door entrance helps less and would constrain four biomes' entrance designs — so the map team is **not** given a rule about entrance door count. It stays a gameplay decision.

Hence `RoomLibrary.branchingPool()`: the normal pool minus single-door templates, used only while building the critical path. Dead ends remain allowed on side branches, where ending is exactly what you want. **The same room is a problem in one pool and a feature in the other.**

12.5 Side branches before the boss — the ordering that was measured

In the first implementation the boss attached to the end of the path *before* side branches grew. Two bugs were measured and both disappeared when the order was swapped.

> **BUG 1 — SMALL DUNGEONS COULD NOT PRODUCE 3 ROOMS**
>
> `round(3 × 0.65) = 2`, so the path is entrance + boss. The entrance (`test_entrance`, one door) spends its only door on the boss, and the boss is terminal, so **no open door remains** for a side branch. The third room cannot be placed, generation restarts and lands on a different target. Measured: across 800 seeds a `small` target of 3 *never once* came out — always 4 to 6.

BUG 2 — BOSSLESS DUNGEONS

Attaching first, the boss tried exactly one door. If a 33×33 room collided there, the dungeon came out **with no boss at all** — 4 times in 2,000 medium generations, 7 times in 2,000 large. A bossless dungeon gives the player nothing to aim at.

	OLD ORDER	NEW ORDER
small: path complete	99.0%	**100%**
small: rooms complete	87.5%	**100%**
small: average attempts	3.16	**1.17**
medium: generations with a warning	0.1%	**0%**
bossless out of 2,000 medium	4	**0**

12.6 The boss is assigned, not drawn

```
/**
 * Attaches the boss room to the open door furthest from the entrance.
 *
 * Doors are tried in DESCENDING depth order and the first one that seats wins. The boss
 * therefore goes as far away as possible, and a single door colliding no longer produces a
 * bossless dungeon.
 */
private int attachBoss(DungeonLayout layout, RoomPlacer placer, RandomGenerator random) {
    List<OpenDoor> doors = new ArrayList<>(layout.openDoors());
    doors.sort((a, b) -> Integer.compare(
            layout.node(b.nodeId()).depth(), layout.node(a.nodeId()).depth()));

    for (OpenDoor door : doors) {
        if (layout.node(door.nodeId()).doorState(door.doorIndex()) != DoorState.OPEN) {
            continue;
        }
        RoomPlacer.Attempt placed = placer.fill(layout, door, library.bosses());
        if (placed.success()) {
            return placed.placed().id();
        }
        // When fill() fails it has already marked the door DEAD — it won't be tried again.
    }
    return -1;
}
```

DungeonGenerator.attachBoss — depth-descending, first fit wins.

12.7 Side branches: breadth first, full pool

```
private void growBranches(DungeonLayout layout, RoomPlacer placer,
                          RandomGenerator random, int targetRooms, int bossNodeId) {
    Deque<OpenDoor> queue = new ArrayDeque<>();
    for (LayoutNode node : layout.nodes()) {
```

```
            if (node.id() != bossNodeId) {
                queue.addAll(node.openDoors());
            }
        }

        while (layout.size() < targetRooms && !queue.isEmpty()) {
            OpenDoor door = queue.poll();
            if (layout.node(door.nodeId()).doorState(door.doorIndex()) != DoorState.OPEN) {
                continue;
            }
            RoomPlacer.Attempt placed = placer.fill(layout, door, library.normalPool());
            if (placed.success()) {
                queue.addAll(placed.placed().openDoors());
            }
        }
    }
```

growBranches. The queue is a FIFO, and that choice matters.

Breadth first, because depth first would produce one long tail and the layout would tangle against itself quickly. The quota passed in is `targetRooms - 1`: one room is held back for the boss.

This is also where the maze feeling comes from. When the critical path enters a room, that room's other doors are left unused. Branching starts from those, so a player walks in and sees three exits — one to the boss, two to loot — and does not know which is which.

12.8 Stalling means retrying, and a short dungeon is never silent

```
Result best = null;
for (int attempt = 0; attempt < maxAttempts; attempt++) {
    // The attempt seed is DERIVED from the master seed, so the whole generation stays
    // reproducible from a single seed. If a retry drew a fresh random seed, bringing
    // back "that broken dungeon" would be impossible.
    Result candidate = attemptOnce(bounds, origin, size, seed, attempt);
    if (candidate.perfect()) {
        return candidate;
    }
    if (best == null || isBetter(candidate, best)) {
        best = candidate;
    }
}
return best;

/** A longer path wins; on a tie, more rooms wins. */
private static boolean isBetter(Result a, Result b) {
    if (a.pathLength() != b.pathLength()) {
        return a.pathLength() > b.pathLength();
    }
    return a.rooms() > b.rooms();
}
```

The retry loop. The best attempt is kept, and the result carries a warning.

`generation.max-attempts` defaults to 8. If every attempt stalls, the best one is used and the result is reported **with a warning**. The promise made to the player is that choosing "medium" produces a medium dungeon; silently handing over a small one breaks it without telling anyone.

12.9 Out-of-the-box fallbacks

Three places where the generator degrades rather than failing, all of them because a fresh installation may not have a complete room set:

MISSING	BEHAVIOUR
No multi-door normal rooms	`branchingPool()` is empty, so the path falls back to the full normal pool. The path guarantee weakens; a dungeon still comes out.
No entrance template	An entrance is picked from the normal pool.
No boss template	Generation completes and the result carries the warning "no boss room template — the dungeon was generated without one".

12.10 Measured guarantees

SIZE	PATH COMPLETE	ROOMS COMPLETE	WARNED	AVG. ATTEMPTS
`small`	100%	100%	0%	1.17
`medium`	100%	100%	0%	1.30
`large`	99.9%	100%	0.1%	1.64

3×1000 generations. Phase 1C's naive strategy scored 70% under the same conditions.

12.11 The Result record is a complete description of a dungeon

`DungeonGenerator.Result` carries the layout, the size, the targets, the achieved path length, the boss node id, the attempts used, the seed and any warning. The important consequence is stated in the code itself: **to reproduce a dungeon you need the slot index, the theme, the size and the seed — four values.** Phase 7 writes four columns, not a serialised layout.

CHAPTER SUMMARY

- Path length is `round(rooms × 0.65)`, minimum 2, entrance and boss included.
- Path first, because that is what guarantees playtime. Randomness for variety, skeleton for guarantees.
- Stalls are branching-process extinction, not collisions. Filtering single-door rooms out of the path pool takes 70% to 97%.
- Side branches grow *before* the boss, and that order fixed two measured bugs.
- The boss is assigned to the deepest open door, trying all of them in depth order.
- A stall retries with a derived seed; the best attempt is used and warned about.

CHAPTER 13

Randomness, and a trap that would have been silent

`Seeds` is seventy lines and does almost nothing: it mixes a number and hands it to a generator. It exists as its own class because of a measurement, and because without it every small dungeon in the game would have been the same size — quietly, and forever.

13.1 The measurement

`java.util.Random` is a linear congruential generator whose first output is a direct function of the seed's high bits. Adjacent seeds therefore produce adjacent internal state. This was measured directly:

```
seed = 1..40:
  2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2

distribution over 4000 consecutive seeds:   [0, 0, 1857, 2143]
                                             ^^^^^^
                                            0 and 1 NEVER appear
```

new Random(seed).nextInt(4) over consecutive seeds.

13.2 Why this would have mattered, concretely

A `small` dungeon's room count is drawn from the 3–6 range, and that draw is the *first* call on the generator. If the first draw is constant across consecutive seeds, every small dungeon comes out the same size and the promise of a range is void.

> **AND IT REALLY WOULD HAVE HAPPENED**
>
> In Phase 7 instances get written to a database, and the natural things to seed with are an incrementing id or `System.currentTimeMillis()` — **both consecutive**. The failure would also have been silent: dungeons still generate, they just always come out the same size. Nobody files a bug report for "the variety feels a bit low".

13.3 The fix

```java
/**
 * The splitmix64 final mixing step — spreads a 1-bit input difference across half the output.
 *
 * The constants come from the reference splitmix64 implementation and must not be changed;
 * the quality of the mixing depends on these multipliers.
 */
public static long mix(long seed) {
    long z = seed + 0x9E3779B97F4A7C15L;
    z = (z ^ (z >>> 30)) * 0xBF58476D1CE4E5B9L;
    z = (z ^ (z >>> 27)) * 0x94D049BB133111EBL;
    return z ^ (z >>> 31);
}

/** Builds a randomness source from a seed — consecutive seeds give independent streams. */
public static RandomGenerator from(long seed) {
    return new SplittableRandom(mix(seed));
}

/**
 * The n-th derivative of the same master seed — used for retries.
 *
 * The derivation is deterministic too, so the whole generation stays reproducible from a
 * single seed. If a retry drew a fresh random seed instead, bringing back "that broken
 * dungeon" would be impossible.
 */
public static RandomGenerator derive(long seed, int index) {
    return new SplittableRandom(mix(seed + index * 0x9E3779B97F4A7C15L));
}
```

Seeds — splitmix64 mixing, then SplittableRandom.

The same measurement after mixing gives `[977, 1010, 973, 1040]` — uniform. Two properties are preserved that matter as much as the uniformity:

Reproducibility.

The mixing is deterministic, so the same seed still gives the same dungeon. Procedural generation that cannot be reproduced cannot be debugged: without a seed there is no way to bring back "that broken dungeon".

Independent derivatives.

`derive(seed, n)` gives retry *n* its own stream, still deterministically. Phase 3B uses the same function per room — `Seeds.derive(dungeonSeed, nodeId)` — so a room's contents do not shift because a different room happened to be populated first.

> **WHERE ELSE THIS PATTERN SHOWS UP**
>
> `derive` is the project's general answer to "I need a sub-stream that is stable under reordering". Anything keyed by a stable identifier — retry index, node id, and in future a chest position — can have its own reproducible randomness without threading a single generator through the call graph in a fixed order.

CHAPTER SUMMARY

- `new Random(seed)` is correlated across consecutive seeds; two of four values never appeared in 4,000 draws.
- Phase 7 will seed from an id or a timestamp — both consecutive — so the trap was real, not theoretical.
- splitmix64 mixing plus `SplittableRandom` fixes it while preserving reproducibility.
- `derive(seed, n)` gives stable, independent sub-streams keyed by anything.

CHAPTER 14

Plugging: measuring the opening and its material

When the graph is finished, some doors open into the void. A player who walks through one falls out of the world. The fix could have cost the map team two hundred extra schematics; instead it costs zero files, because the engine *measures* both the hole and the wall around it.

14.1 The rejected alternative, with arithmetic

The intuitive solution is room variants: "give every room a 1-, 2- and 3-door version, and when a door is left over, use a version with one fewer." It was evaluated and eliminated, for a reason that is mathematical rather than aesthetic.

A variant is not defined by the door **count** but by the door **set**. In a three-door room `{N, E, S}` where east is left over, you need `{N, S}` — opposite pair. In another scenario you need `{N, E}` — adjacent pair. Different geometry, different walls. There is no single file called "the two-door version".

SHAPE	EXAMPLE	ROTATIONS IT PRODUCES
Single door	`{N}`	4
Opposite pair	`{N,S}`	2
Adjacent pair	`{N,E}`	4
Three doors	`{N,E,S}`	4
Four doors	`{N,E,S,W}`	1
Total distinct door sets over four walls		**15**

Rotation collapses 15 sets into 5 shapes. But 5 shapes × 40 rooms = **200 schematics**. The rotation decision had already cut the map workload from 120 to 40; variants would have pushed it to 200. The map team is the project's tightest bottleneck.

14.2 What was built instead

`DoorPlugger` seals both OPEN doors (never tried, the side-branch quota ran out) and DEAD doors (tried, everything collided). It does so by scanning.

The opening's size is measured, not declared

There is no "door is 3×3" field anywhere in the metadata. The engine starts at the anchor and flood-fills air blocks *within the room's wall plane*. If a mapper draws a 3×4 door the plug still fits; arched, stepped and asymmetric openings work too.

The material is measured too

The plug block does not come from configuration. The wall blocks at the *edge* of the opening are sampled and the most frequent one wins. A Nether room therefore yields nether brick and an End room end stone, with no per-biome setting at all.

> **THIS DELIVERED MORE THAN THE DESIGN EXPECTED**
>
> The specification listed two implementations: procedural plugging, and hand-drawn plug schematics at four files (one per biome). Because the material is sampled rather than configured, the procedural route already produces the biome-appropriate result. The four-file route is now needed only if somebody wants a *decorative* plug — an ornate frame — rather than a matching wall.

14.3 The scan

```
private static Opening scan(EditSession session, PlugTarget target) {
    Vec3i anchor = target.anchor();
    Aabb bounds = target.roomBounds();
    boolean alongX = target.outward() == Direction.NORTH || target.outward() == Direction.SOUTH;

    List<BlockVector3> cells = new ArrayList<>();
    Set<Long> seen = new HashSet<>();
    Deque<BlockVector3> queue = new ArrayDeque<>();
    // Materials sampled from the edge — the most frequent one wins. Looking at a single
    // neighbour would be misleading: right beside a door there may be a torch, a window or
    // some other decorative block.
    Map<BlockState, Integer> materials = new HashMap<>();

    BlockVector3 start = BlockVector3.at(anchor.x(), anchor.y(), anchor.z());
    queue.add(start);
    seen.add(key(start));

    while (!queue.isEmpty()) {
        BlockVector3 pos = queue.poll();

        if (!contains(bounds, pos)) {
            continue;   // don't leak outside the room
        }
        BlockState state = session.getBlock(pos);
        if (!state.getBlockType().getMaterial().isAir()) {
            materials.merge(state, 1, Integer::sum);   // the edge of the opening is wall
            continue;
        }

        cells.add(pos);
        if (cells.size() > MAX_OPENING_BLOCKS) {
            return null;
        }
        for (BlockVector3 next : neighbours(pos, alongX)) {
            if (seen.add(key(next))) {
                queue.add(next);
            }
        }
    }

    BlockState material = materials.entrySet().stream()
            .max(Map.Entry.comparingByValue())
```

```
            .map(Map.Entry::getKey)
            .orElse(null);
    return new Opening(cells, material);
}
```

DoorPlugger.scan — a bounded flood fill inside the wall plane.

Why each guard exists. Without guard 1 the scan would escape the opening into empty space and fill half the dungeon with wall.

14.4 The 64-block cap is a guard against broken metadata

A 3×3 door is 9 blocks, so 64 leaves plenty of slack. Its real purpose is different: if the anchor sits *inside* the room rather than on a wall — which `RoomTemplateStore` warns about but does not treat as fatal — the scan starts on an empty interior plane and would fill the room's entire cross-section. When the cap is exceeded the door is skipped and a warning is logged, naming the coordinate and asking the obvious question ("is the anchor really on a wall?").

14.5 One EditSession for the whole pass

All plugs go through a single `EditSession`. Opening one per door would break FAWE's batching and become visibly slow on a large dungeon. The pass runs on the same threading decision as pasting: async when FAWE is present, main thread otherwise.

> **PLUGGING HAPPENS AFTER EVERY PASTE, NEVER BEFORE**
>
> Two reasons, and either alone is sufficient. First, a plug written before a neighbouring room's paste would simply be overwritten. Second — and more fundamentally — **whether a door is left open is not known until the graph is complete**. Plugging is not a per-room operation; it is a whole-dungeon one.

14.6 The escape hatch that was left open

The specification deliberately preserves the variant idea in a reduced form: a room may *declare* its own variants in metadata. For places where "this must look perfect" — a boss room, a signature entrance — a

fully hand-drawn variant can be supplied, and the engine will use it when present and fall back to the plug when not. The engine code is the same either way; the map investment is optional.

CHAPTER SUMMARY

- Door-count variants are not a thing: a variant is defined by the door *set*, and that is 5 shapes × 40 rooms = 200 schematics.
- The opening is found by a bounded flood fill in the wall plane; its size is never declared.
- The material is sampled from the opening's edge, most-frequent wins — which gives per-biome plugs for zero files.
- Three guards: the room's box, a 64-block cap against bad anchors, and treating an already sealed door as normal.
- Plug after all pastes, because "is this door open" is a whole-graph question.

CHAPTER 15

The void world and the slot grid

Four small files that set the stage: a world with nothing in it, a set of gamerules that keep it that way, and a grid that hands out squares.

15.1 VoidChunkGenerator: every stage disabled

```
@Override public boolean shouldGenerateNoise()       { return false; }
@Override public boolean shouldGenerateSurface()     { return false; }
@Override public boolean shouldGenerateCaves()       { return false; }
@Override public boolean shouldGenerateDecorations() { return false; }
@Override public boolean shouldGenerateMobs()        { return false; }
@Override public boolean shouldGenerateStructures()  { return false; }
```

VoidChunkGenerator — the whole point is what it returns false for.

No stone, no caves, no structures, no decoration, no bedrock. A dungeon's only content is the schematics pasted into it; leaving vanilla generation on would burn CPU and accumulate unrelated blocks and mobs outside the rooms.

The biome is `THE_VOID`, chosen for two properties: its natural mob spawn table is empty, so nothing leaks in outside the rooms, and there are no visual artefacts such as grass tint or water colour. Two more overrides tidy up the edges — `getBaseHeight` returns the world minimum so that Bukkit's "highest block" queries do not climb to the ceiling, and `getFixedSpawnLocation` hands back a fixed point outside the grid because Bukkit insists on having one even though players only ever arrive by teleport.

15.2 The gamerules

On load, `DungeonWorldManager` applies a set of rules that together make the world inert:

RULE	WHY
`doMobSpawning = false`	Every mob in a dungeon is placed by Phase 3, with known stats. Natural spawns would be outside that system.
`doDaylightCycle = false`	Time is pinned at noon. Rooms are sealed, so there is no sky light — which is also why zombies and skeletons do not burn. A room schematic with an open ceiling changes that.
`doWeatherCycle = false`	No weather in a sealed world.
`doFireTick = false`	Fire must not spread through hand-built rooms.
`mobGriefing = false`	A creeper explosion must not punch a hole in a room. This is why creepers are usable in the default mob set at all.
patrol / trader spawning off	Vanilla event spawns have no place here.
`randomTickSpeed = 0`	Nothing should grow, melt or decay.
`spawnChunkRadius = 0`	No permanently loaded chunks around a spawn nobody uses.

15.3 The slot grid

Covered structurally in Chapter 5; the operational details are here.

```
/** Allocates a new slot: the released pool first, otherwise the next fresh index. */
public synchronized GridSlot allocate() {
    int index = freed.isEmpty() ? nextIndex++ : freed.pollFirst();
    GridSlot slot = slotAt(index);
    allocated.put(index, slot);
    return slot;
}

/** Releases the slot (does not clear its blocks — see the class note). */
public synchronized boolean release(int index) {
    if (allocated.remove(index) == null) {
        return false;
    }
    freed.add(index);
    return true;
}
```

GridSlotManager — allocation and release.

Released indices go into a `TreeSet` and are reused smallest first, so the world does not grow without bound. Every method that touches state is `synchronized`, because allocation happens on whichever thread asked for a dungeon while release can happen from the teardown path.

15.4 GridSlot: the X/Z-only containment test

```
/**
 * Whether a horizontal position falls inside this slot.
```

```
     *
     * Deliberately X/Z only: "which instance is this player in" must answer the same whether they
     * stand on a room's floor or in the void above it — a Y test would lose them the moment they
     * fall through a hole.
     */
    public boolean contains(double x, double z) {
        return x >= originX && x < originX + size && z >= originZ && z < originZ + size;
    }
```

GridSlot.contains — deliberately ignoring Y.

This one method decides the behaviour of three later systems: the teleport block (Chapter 25), the eviction sweep at teardown (Chapter 21), and eventually the party system. A player who falls off a bridge into the void **is still inside that dungeon**, and all three need to agree about that.

The slot's 3D bounds take X and Z from the slot and Y from the world's height limits. There is no vertical slot concept: rooms may stack as tall as the world allows, and the X/Z bound is the only hard requirement, because overflowing it reaches into a neighbouring instance.

15.5 World reset on start

`dungeon-world.reset-on-start` defaults to `true`, and the reasoning is a clean syllogism: this world holds nothing but dungeon instances; no instance survives a shutdown, because they live in memory only; therefore every block left on disk is the debris of a dungeon that no longer exists.

Without the wipe, the slot counter restarts at 0 while the old blocks stay put, and the next dungeon is pasted directly on top of the previous one. `level.dat` is deliberately *not* deleted, so the world's own settings and spawn point survive.

CHAPTER SUMMARY

- Every generation stage is disabled and the biome is `THE_VOID`, whose spawn table is empty.
- The gamerules exist to make the world inert; `mobGriefing = false` is what makes creepers usable.
- Slots are handed out from a reuse pool, smallest index first, under a lock.
- `contains` is X/Z only, so a player in the void below the rooms is still "inside" the dungeon — three later systems depend on that.
- The world resets on start because it is entirely derived data.

CHAPTER 16

Schematics: loading, caching, rotating, pasting

`SchematicService` is the only place in the project that talks to WorldEdit for writing, and it makes three decisions that are easy to get wrong and expensive to get wrong quietly.

16.1 Always load asynchronously, cache the result

Files live under `plugins/TakashiDungeons/schematics/` and are read **always** off the main thread — disk I/O plus NBT parsing will block it. The resulting `Clipboard` is cached in memory: the same room is pasted hundreds of times over a server's life, so the file is read once. `invalidateCache()` is called on reload.

16.2 Async paste is only safe with FAWE

> **THE THREADING RULE**
>
> Plain WorldEdit's `EditSession` is **not thread safe**; writing from another thread corrupts the world. So the paste thread is chosen by what is installed: async when FAWE is present, main thread otherwise. `schematics.force-sync-paste` exists to force the main thread even with FAWE, purely for diagnosis — with it on, generating a large dungeon costs TPS.

The same decision is reused deliberately elsewhere: `RegionCleaner` takes *identical* threading, because a synchronous paste racing an asynchronous wipe in the same slot is exactly the corruption that decision exists to prevent.

16.3 The rotation sign, once more

```
new AffineTransform().rotateY(-degrees)
```

The call, and why the sign is negative.

Negative, so that the engine's rotation matches WorldEdit's own `//rotate` command. If this sign changes, the direction a mapper sees in the editor and the direction the engine produces diverge — and every door in every dungeon stops lining up, silently. Chapter 6 has the measurement.

16.4 Three traps in this file

> **COPYENTITIES(FALSE)**
>
> Entities embedded in a schematic would be duplicated on every paste and would sit *outside* Phase 3's stat system entirely. The map team's rule is therefore "do not embed entities in schematics", and this flag enforces it regardless.

> **CLIPBOARDFORMATS.FINDBYFILE CANNOT BE USED ON THE WRITE PATH**
>
> It detects a format by opening the file, so on a file that does not exist yet it throws `NoSuchFileException`. The write path resolves the format by alias instead (`sponge.3` / `schem` / ...).

> **FAWE DOES NOT LEAVE PASTED CHUNKS LOADED**
>
> This one trap appears in four different places in this book: console block verification (Chapter 2), entity sweeping at teardown (Chapter 21), mob population (Chapter 32), and here. A block read in an unloaded chunk answers *air*. Anything that inspects the world after a paste must load the chunks first.

16.5 TestRoomFactory: placeholder rooms, and why it writes the metadata too

Until Phase 10's hand-built rooms arrive, `/tdungeons gen` generates eight placeholder rooms in code. There are five basic shapes plus three that exist to hold specific properties in place:

ROOM	PURPOSE
`test_corridor`	Two opposite doors — the straight-continuation case
`test_corner`	Two adjacent doors (N+E) — used to verify the rotation sign
`test_cross`	Four doors — the branching case, and the reference for floor/ceiling coverage
`test_deadend`	One door — the branching-extinction case
`test_entrance`	Type `entrance` — the critical path's root
`test_long`	9×25 with an off-centre east door — the Chapter 7 wall-derivation trap
`test_even`	10×16, asymmetric origin — proof the odd-side rule is unnecessary

> **WHY THE FACTORY WRITES THE .YML AS WELL AS THE .SCHEM**
>
> It is the only place that knows where it carved the doors. If the metadata were hand-written, then changing the schematic without changing the metadata would produce rooms whose doors do not line up — and, per Chapter 7, that failure is silent. One producer, one truth.

CHAPTER SUMMARY

- Load async always; cache the clipboard; invalidate on reload.
- Paste async only with FAWE. `RegionCleaner` mirrors the decision on purpose.
- `rotateY(-degrees)` matches WorldEdit's own command; changing the sign breaks every door silently.
- No entities in schematics; no `findByFile` on the write path; load chunks before reading blocks.
- The test room factory writes metadata as well as geometry, because it is the only thing that knows the door positions.

CHAPTER 17

Templates, metadata, themes and bundled rooms

This chapter is the map team's interface, from four directions: what a `.yml` contains, how a theme is declared (it is not declared), how rooms travel inside the jar, and the seven rules a mapper has to follow.

17.1 The metadata file

Every schematic has a `.yml` beside it with the same name. It is deliberately **not** one central file: the map team works in parallel and a central file is a merge conflict generator.

```
type: normal          # entrance | normal | boss

weight: 100           # the TEMPLATE's share in the candidate draw (loot-weight semantics).
                      # INDEPENDENT of door count — a 4-door room and a 1-door room each
                      # count their weight once.

# Door anchors: LOCAL coordinates relative to the origin (the room's reference point).
# [x, y, z] — the base-centre block of the door opening.
# FACING IS NOT WRITTEN HERE. It is derived from the anchor.
doors:
  - [ 0, 1, -8]       # north wall
  - [ 8, 1,  0]       # east wall
  - [ 0, 1,  8]       # south wall
  - [-8, 1,  0]       # west wall
```

schematics/test_cross.yml — the entire schema.

Four fields, and two of them are the same field repeated. Everything else about the room — its size, its bounding box, which wall each door is in, how big the opening is, what the wall is made of, where mobs can stand — is **derived**.

Why `y: 1`?

The opening's base block sits one above the room's floor. In a two-storey room the upper door is `[0, 9, -8]` and nothing about the system changes.

Where are `door1 / door2 / door3`?

The list order *is* the name. It is an address, not a fill order: which door gets filled is decided by geometry, not by the order they were typed. The index is how the engine tracks which door was connected and which one needs plugging.

17.2 The template is a schematic plus a metadata file

```
/**
 * SIZE IS NOT WRITTEN IN THE METADATA — localBox is read from the schematic itself. Like
 * door facing, it is a derived value: a mapper who enlarges a room in the editor and
```

```
 * re-exports the schematic can easily forget to update the .yml, and a wrong box makes the
 * collision test silently wrong. The schematic is the single source.
 */
public record RoomTemplate(String name, RoomType type, int weight,
                           List<DoorAnchor> doors, Aabb localBox) { }
```

RoomTemplate. Note that localBox comes from the schematic, not from the yml.

`RoomTemplateStore` joins the two, asynchronously, and caches the result. Its cache sits *above* `SchematicService`'s clipboard cache, which produces a small trap:

> **TWO CACHES, AND BOTH MUST BE CLEARED**
>
> Clearing only the lower one leaves stale door metadata in memory. `/tdungeons reload` clears both. This matters most during the room-building loop, where a mapper re-exports a room and expects the server to see the new version — otherwise verification passes on stale geometry and says nothing.

17.3 A theme is a folder

A dungeon is built from one theme's room pool. A theme is not a metadata field — it is the directory the room sits in.

```
plugins/TakashiDungeons/schematics/
├── entrance_grand.schem + .yml     -> theme "default"  (the root is a real theme)
├── crypt/
│   └── hall_pillars.schem + .yml   -> theme "crypt"
└── nether/
    └── hall_pillars.schem + .yml   -> theme "nether"   (same name, no collision)
```

The layout on disk. Two rooms may share a name across themes.

Themes are discovered, not declared.

Any folder containing at least one schematic is a theme. There is no theme list in `config.yml`, and therefore nothing that can fall out of sync.

A room cannot land in the wrong theme through a typo.

Moving it requires moving a file. This is Chapter 7's principle applied at the filesystem level.

The root counts as `default`.

Which is why the theme layer needed no migration: every room that existed before themes is now a `default` room, and every old command still works. Room keys are `theme/name`, with `default` rooms keeping their bare name.

The generator itself was **not modified** to support themes. It already took a prepared `RoomLibrary`; the theme layer builds one library per theme. This is entirely a loading-layer concern.

> **THE REPRODUCTION TUPLE IS NOW FOUR VALUES**
>
> **slot + theme + size + seed.** Without the theme, the same seed draws from a different pool and builds a different dungeon. Phase 7 stores four columns.

Two commands make themes visible. `/tdungeons themes` reports each theme's pool composition — room count, entrance/boss/normal split, how many multi-door rooms — and warns if an entrance or boss is missing. `/tdungeons dungeon <theme> [size] [seed]` requires the theme when more than one exists; picking one silently would produce the confused bug report "why did I get crypt rooms".

17.4 Bundled rooms: the out-of-the-box guarantee

Rooms under `src/main/resources/schematics/` travel inside the jar and are extracted to the plugin folder on enable. The intent is that downloading the jar and installing FAWE is enough; a user should not have to hunt for a room pack.

DECISION	REASON
The file list is read from the `JarFile`, but the *content* through `plugin.getResource()`	Paper hands the plugin a remapped copy of its own jar. The resources are identical in both, but taking content through the classloader guarantees what the running plugin actually sees.
The path inside the jar is preserved verbatim	`schematics/crypt/hall.schem` lands directly in the `crypt` theme. Bundling a new theme requires no code change — just a folder.
Extraction runs *before* the WorldEdit check	A server that installs FAWE later must still find the rooms already in place.
An existing file is never overwritten	The schematics folder is also the mapper's workspace — rooms are exported into it from the world. A bundled room with the same name could silently eat a newer export on every restart. Only missing files are written, which also means a new version's new rooms arrive by themselves.

`/tdungeons extract [force]` repeats what enable does, by hand, for the room-building loop: rebuild the jar and get the new room onto the test server without a restart. `force` overwrites — and can therefore eat a newer local export, which is exactly why it is never the default. If any file was written, both caches are cleared.

> **TWO .GITIGNORE LINES, NOT ONE**
>
> `*.schem` is ignored globally. Re-enabling it needs `!src/main/resources/schematics/*.schem` **and** `!src/main/resources/schematics/**/*.schem`. Without the second line, rooms in theme subfolders were silently not committed.
>
> Related: Maven resource filtering is applied to `plugin.yml` only. If filtering is ever opened up to all resources, it will corrupt every `.schem` binary.

> **CURRENT STATE OF THE BUNDLED SET**
>
> The mechanism works and was verified byte-for-byte: with the schematics folder moved aside, enable extracted 6 files, `extract` reported "6 already present", `extract force` rewrote all 6, and the results were byte-identical to the sources. But the jar currently contains **three entrance rooms and no normal or boss rooms**, so a clean installation still cannot generate a dungeon. What is missing is rooms, not code — Phase 10.

17.5 The map team's rules

Seven rules, and the notes on each are the accumulated scar tissue of the first hand-built rooms.

1. The origin is the block you are standing on when you type `//copy`

WorldEdit writes the player's position as the clipboard origin, and every anchor in the `.yml` is the difference vector from it.

> **THIS RULE WAS CORRECTED AFTER THE FIRST HAND-BUILT ROOMS**
>
> It used to say "the origin must be at the room's horizontal centre, at floor level". That is **not producible by hand**: for the origin to be the floor block you would have to stand *inside* the floor and type `//copy`. `TestRoomFactory` can do it because it builds the clipboard in code — its rooms have origin-at-floor and doors at `y: 1`. A human measures at foot level and their doors come out at `y: 0`. **Both work**, because the engine only requires the room to be internally consistent: the box comes from the clipboard and the wall from X/Z. Measure the origin and every door the same way and the offsets take care of themselves.

2. The selection must include the floor below and the ceiling above

> **LEARNED THE HARD WAY**
>
> The block you stand on is the *walking level*, not the floor — the floor is the layer below it. A selection starting at the level you are standing on produces a room with no floor, and in a void world the player falls straight through. The first three hand-built rooms came out with bottom layers that were **57–71% air**. For reference, `test_cross` has 0% air in both its bottom and top layers. Running `//size` before `//copy` and comparing the height against the room's real floor-to-ceiling height catches this in one second.

3. The anchor is the base-centre block of the opening

One block off and the walls interpenetrate or leave a gap. This is the single point where these systems break.

4. Door openings are 3 wide × 3 high

The standard `TestRoomFactory` produces. The plugger measures anyway, so a non-standard opening still seals — but the standard keeps connections looking uniform.

5. Lighting comes from the room's own light sources

There is no sun in the dungeon world and the daylight cycle is off.

6. Do not embed entities

Mobs are spawned by the engine in Phase 3. An embedded entity multiplies on every paste and sits outside the stat system. `copyEntities(false)` enforces it.

7. Do not measure spawn points either

Phase 3B finds them at runtime: a ring search outward from the room's centre finds a seed, then a flood fill spreads across floor that actually exists. Because the fill only reaches walkable space, a sealed alcove receives no mobs and an L-shaped hall fills around the corner rather than through the wall. **Nothing about mob placement goes into the room's `.yml`.**

> **THE RULE THAT WAS DELETED**
>
> **Side lengths no longer have to be odd.** Under the old cell grid they did: on an even side there is no true centre block, so the rotation centre shifts by half a block. Anchor placement removed the need, and `test_even` proved it. The binding rule is not the side length — it is putting the anchor on the right block.

CHAPTER SUMMARY

- A `.yml` has three things: type, weight, and a list of door anchors. Everything else is derived.
- Metadata is per-room so that parallel map work does not conflict.
- A theme is a folder; themes are discovered, and the root is `default`.
- Bundled rooms never overwrite existing files, because the same folder is the mapper's workspace.
- Seven map rules; the two that have actually bitten are the floor layer and the anchor block.

PART TWO

The instance lifecycle

Phase 2. Phase 1 could paste a dungeon into a slot, and nothing owned the result. This part is about ownership: a dungeon that knows who is inside it, counts down, sends everybody home, and deletes itself in an order that is itself the design.

CHAPTER 18

Why a generated dungeon needs an owner

At the end of Phase 1 the plugin could produce a complete, sealed, walkable dungeon. It was also completely unable to get rid of one. This chapter is the short statement of the problem Phase 2 exists to solve, because the shape of the solution follows directly from it.

18.1 The bug that defined the phase

`GridSlotManager.release(index)` returns an index to the pool. It does not touch blocks — it is bookkeeping. So at the end of Phase 1:

1. Party A generates a dungeon into slot #3.
2. Something releases slot #3.
3. Party B is allocated slot #3.
4. **Party B walks into party A's dungeon.**

Nothing in Phase 1 was wrong. There was simply no object whose job it was to say "this dungeon exists, these blocks belong to it, and when it ends, this is what happens". That object is `DungeonInstance`, and the process is `InstanceManager.close`.

18.2 The three questions Phase 2 answers

What exists?

A registry of live instances, each with a state that moves in one direction only. An instance enters the registry *only once it stands* — publishing a half-pasted dungeon would let a player be sent into rooms that do not exist yet.

Who is inside?

A declared membership set, deliberately distinct from "who is standing in that slot right now". Chapter 23 is entirely about why those are two different questions.

When does it end, and what happens then?

A shared once-a-second clock drives expiry, threshold warnings and the boss bar. Teardown runs in a fixed five-step order, and every step is placed where it is because putting it elsewhere breaks something specific.

18.3 Nothing survives a restart, on purpose

Instances live in memory only. Portals live in memory only. The dungeon world is wiped on start. This is not an unfinished feature — it is a coherent position: the dungeon world is **derived data**, and the four values needed to rebuild any dungeon (slot, theme, size, seed) are small enough that Phase 7 will store those rather than a world.

The one thing that *does* need to survive is a player who logged out inside a dungeon. That is handled by a rescue on join (Chapter 21), not by persistence.

CHAPTER SUMMARY

- Releasing a slot never cleared its blocks, so slot reuse handed the next party the previous dungeon.
- Phase 2 adds an owner: one object per live dungeon, plus a fixed teardown order.
- An instance is registered only once it stands.
- Nothing persists across a restart, deliberately; the rescue-on-join net covers the one case that matters.

CHAPTER 19

What an instance is, and what is deliberately not stored

`DungeonInstance` is the object that owns a live dungeon. Reading it is mostly an exercise in noticing what is *absent*: the class is careful about what it keeps in memory and equally careful about what it refuses to.

19.1 Identity is four values

> **STATED IN THE CLASS COMMENT**
>
> "The identity of a dungeon is the quadruple **slot + theme + size + seed** — hand those four back to `DungeonGenerator` and the same rooms come out in the same places. So phase 7 will write four columns, not a layout dump."

The generated `DungeonGenerator.Result` *is* kept in memory, but for two reasons that only apply while the instance is alive: `bounds()` is the exact volume to wipe on close, and Phase 3 needs the room graph to know where to spawn what. Neither survives a restart, and neither needs to.

19.2 The state machine

```
/**
 * BUILDING → ACTIVE → CLOSING → CLOSED — a state never moves backwards, and the transition
 * is enforced rather than assumed. The reason is the same one that made LayoutNode's door
 * states one-way: closing runs across several ticks (evict → clear → unload → release) and a
 * second close landing in the middle of the first would release the slot twice and hand the
 * same square to two parties.
 */
public enum InstanceState {
    /** Rooms are being generated and pasted. Nobody may enter yet. */
    BUILDING,
    /** Standing, walkable, players may be inside. */
    ACTIVE,
    /** Teardown started: players are out, blocks are being wiped. */
    CLOSING,
    /** Gone. The slot has been returned to the pool and may already hold another dungeon. */
    CLOSED
}
```

InstanceState — four states, one direction.

`advanceTo` returns `false` on an illegal transition, and `close()` converts that into a failed future rather than a silent no-op — so the caller sees an error instead of a double free.

19.3 The cleanup volume

An instance's `bounds()` is the **union of its placed rooms**, grown by one block of margin and then clamped to the slot.

```
/**
 * How far the wiped box reaches past the rooms themselves.
 *
 * Doors are plugged inside their own room's box, so in principle the union of the rooms is
 * exactly what was written. One block of slack costs nothing on a volume this size and covers
 * the off-by-one class of mistake — a wipe that misses is far more expensive than one that
 * clears a shell of air.
 */
private static final int CLEANUP_MARGIN = 1;
```

The margin, and the reasoning for it.

The clamp is a second lock on the same door: generation already refuses to place a room that overruns its slot, but if one ever did, an unclamped cleanup would overrun too and eat the neighbouring instance's blocks.

> **THE NUMBER THAT JUSTIFIES ALL OF THIS**
>
> A slot is 512×384×512 ≈ **100 million blocks**. A measured five-room small dungeon occupies a 61×17×56 box, of which **6,551 blocks actually changed**, cleared in 155 ms. Wiping the whole slot instead would turn "close instance" into a visible server stall.

19.4 Two collections, two different questions

```
/**
 * Who is inside, in the order they entered.
 *
 * Membership is DECLARED, not derived from position: a player who logged out inside is still a
 * member, and one who fell through a hole into the void below the rooms has not left. Position
 * answers a different question — instanceAt — and the two are used for different things: this
 * set decides who sees the boss bar and who gets teleported home, position decides who gets
 * swept out of a slot about to be wiped.
 */
private final Set<UUID> players = new LinkedHashSet<>();

/**
 * Where each player came from.
 *
 * Kept per player, not per instance: a party can gather from anywhere, and sending everybody to
 * whoever entered first came from would be a teleport exploit rather than a courtesy.
 */
private final Map<UUID, Location> returnLocations = new LinkedHashMap<>();
```

Membership, and the per-player return location.

The return location is **not overwritten** on re-entry. A player whose connection dropped and who came back should go to where they originally started, not to wherever they happened to be standing the second time.

19.5 The idle timer arms late, on purpose

```
/**
 * When the instance last became empty, or -1.
 *
 * Only armed once somebody has actually been inside. A dungeon an operator generated and never
 * entered has been empty since birth, and killing it on that basis would delete rooms out from
 * under whoever is inspecting them.
 */
private long emptySince = -1;
private boolean everOccupied;
```

Two fields that exist to protect one specific person.

This is a recurring shape in the codebase: a rule that is correct in general gets an explicit exception for the case where following it would delete the work of the person operating the server.

CHAPTER SUMMARY

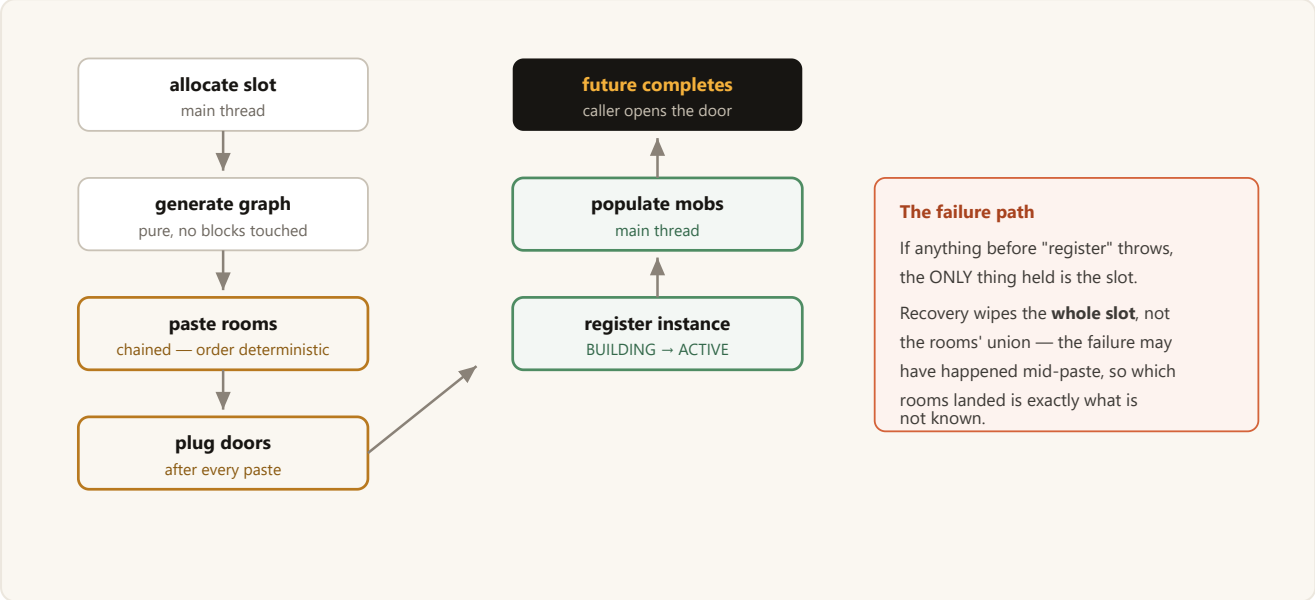
- A dungeon's identity is slot + theme + size + seed. Phase 7 stores four columns.
- State is one-way and enforced; a second close fails rather than double-freeing the slot.
- The wipe volume is the union of the rooms plus one block, clamped to the slot — 6,551 blocks rather than 100 million.
- Membership and position are two different questions with two different answers.
- The empty-timeout only arms after somebody has actually been inside.

CHAPTER 20

Creation: the order of paste, plug, register, populate

`InstanceManager.create` is a chain of futures, and the order of its links is load bearing at three points. This chapter walks the chain and explains each of them.

20.1 The chain



The one path in the system that pays for a full-slot clear — and it is the rare one.

20.2 Pastes are chained, not parallel

```
/** Pastes the rooms one after another — chained so the order stays deterministic. */
private CompletableFuture<Long> paste(SchematicService service, World world,
                                      DungeonGenerator.Result result) {
    CompletableFuture<Long> chain = CompletableFuture.completedFuture(0L);
    for (LayoutNode node : result.layout().nodes()) {
        PlacedRoom room = node.room();
        chain = chain.thenCompose(ignored -> service.load(room.template().name())
                .thenCompose(clip -> {
                    Vec3i o = room.origin();
                    return service.paste(clip, world, o.x(), o.y(), o.z(),
                            room.rotation().degrees(), false);
                }));
    }
    return chain;
}
```

InstanceManager.paste — thenCompose, one room at a time.

Rooms do not overlap, so parallel pasting would be safe in principle — but chaining keeps the sequence deterministic, which matters for reproducing a report and for reasoning about what happened when something goes wrong mid-generation.

20.3 Registration happens only once the dungeon stands

> **WHY NOT REGISTER FIRST AND FILL IN LATER?**
>
> Because publishing a half-pasted dungeon would let a player be sent into rooms that do not exist yet. And on failure there would be a registered instance whose teardown has to undo a teardown that never started. Until the paste is done, the only thing held is the slot, and the failure path returns it.

20.4 Population comes before the future completes

```
/**
 * Fills the rooms with mobs, on the main thread, BEFORE the future completes.
 *
 * The ordering is the point: whoever asked for this dungeon opens the door the moment the
 * future resolves. Populating afterwards would let a player walk into an empty hall and watch
 * mobs blink into existence around them. A tick's pause before the door opens is invisible;
 * a room that fills up behind you is not.
 *
 * A failure here never fails the instance. A dungeon with no mobs is a disappointing dungeon;
 * a dungeon that failed to generate is a held slot and a player standing at a portal being
 * told nothing.
 */
```

InstanceManager.populate — the ordering argument, and the error policy.

Two distinct policies in one method, and both are worth naming. The *ordering* policy trades an invisible cost (one tick) against a visible one (mobs appearing around the player). The *error* policy ranks two failure modes and accepts the cheaper one.

20.5 The close callback seam

```
/**
 * Called once per instance, after it has been torn down.
 *
 * A callback rather than a direct call into the portal layer: instances must not have to know
 * what opened them. Phase 8's DungeonCompleteEvent hangs off the same seam.
 */
private final List<java.util.function.Consumer<DungeonInstance>> closeHandlers = new ArrayList<>();
```

A callback rather than a direct call — and the reason names phase 8.

The portal layer registers here to learn that its dungeon ended. A listener that throws is logged and the teardown continues — by that point the slot has already been released by design, so aborting would leave the system in a worse state than carrying on.

CHAPTER SUMMARY

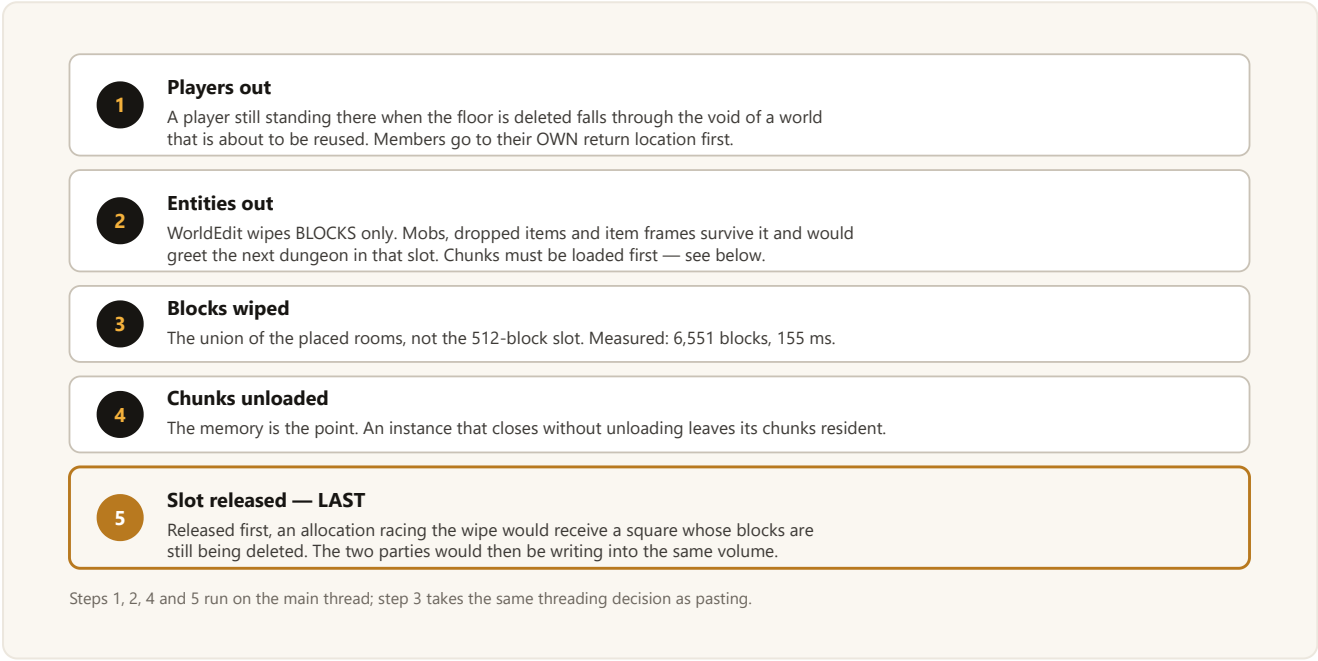
- Allocate → generate → paste → plug → register → populate → complete.
- Pastes are chained for determinism, not for safety.
- Registration happens only once the dungeon stands; before that the only thing held is the slot.
- The failure path wipes the whole slot because which rooms landed is exactly what is unknown.
- Mobs are placed before the future resolves; a population failure never fails the instance.

CHAPTER 21

Teardown: five steps, and why each one is where it is

The class comment for `InstanceManager` says "the teardown order is the whole design", and it is not overstating. Each of the five steps is in its position because moving it breaks something specific and identifiable.

21.1 The order



Move any step and something identifiable breaks. That is what "the order is the design" means.

21.2 The code

```
long start = System.currentTimeMillis();
CompletableFuture<int[]> evicted = onMainThread(() -> {
    // Registered members go to their own return location; the positional sweep that
    // follows is for anyone standing in the slot without being a member — an operator
    // who walked in, or a player mid-fall.
    int home = instance.playerCount();
    sendEveryoneHome(instance);
    return new int[]{home + evictPlayers(world, instance), removeEntities(world, instance)};
});

return evicted
        .thenCompose(counts -> cleaner.clear(world, instance.bounds())
                .thenApply(report -> new int[]{counts[0], counts[1], report.blocks()}))
        .thenCompose(counts -> onMainThread(() -> {
            int chunks = unloadChunks(world, instance);
            plugin.getSlotManager().release(instance.slot().index());
```

```
            unregister(instance);
            instance.advanceTo(InstanceState.CLOSED);
            notifyClosed(instance);
            ...
        }));
```

InstanceManager.close — the shape of the chain mirrors the order above.

Note the two-stage player removal. Members are sent to their *own* return locations first; only then does a positional sweep collect anyone else standing in the slot. If only the sweep ran, everybody would be dropped at world spawn — including the party that entered from a lobby portal fifty blocks away.

21.3 The entity sweep needs its chunks loaded

```
/**
 * THE CHUNKS ARE LOADED FIRST, AND THAT IS THE WHOLE POINT. FAWE does not leave the chunks it
 * wrote resident — the same fact that makes `execute if block` need a `forceload` in manual
 * testing. An entity search only sees loaded chunks, so without this the mobs of a closed
 * dungeon would stay asleep in the region files and wake up inside the next party's dungeon.
 * Loading is not extra work either: the wipe that follows has to load exactly these chunks
 * anyway, and unloadChunks then has something real to release.
 *
 * Players are excluded — they were teleported out a step earlier, and anyone who slipped in
 * since is a bug to notice, not an entity to delete.
 */
private int removeEntities(World world, DungeonInstance instance) {
    Aabb box = instance.bounds();
    for (int cx = box.minX() >> 4; cx <= box.maxX() >> 4; cx++) {
        for (int cz = box.minZ() >> 4; cz <= box.maxZ() >> 4; cz++) {
            world.getChunkAt(cx, cz);
        }
    }
    ...
}
```

removeEntities — and the trap that made this necessary.

> **HOW THIS WAS CAUGHT**
>
> The first test close reported `0 entities removed` where it should have reported `2`. That single wrong number is the whole bug: without the fix, a closed dungeon's mobs sleep in the region files and wake up inside the next party's dungeon. A number being zero is easy to overlook — which is exactly why the close report prints counts at all.

21.4 unloadChunk's return value is deliberately ignored

```
/**
 * THE RETURN VALUE IS DELIBERATELY IGNORED. Paper's chunk system is ticket-based:
 * unloadChunk is a request, it answers false whenever something still holds a ticket —
 * including the one the entity sweep just took — and the chunk is dropped when the last holder
 * lets go. Counting only the calls that returned true therefore reported a flat zero on every
 * close while the chunks did in fact come out of memory. What is counted is what was asked
 * for, which is the part this method controls.
```

```
 *
 * Saved rather than dropped: with reset-on-start off, dropping them would leave the pre-wipe
 * blocks on disk and the deleted dungeon would come back on the next load. The chunks are pure
 * air by this point, so the write is cheap.
 */
```

A case where trusting the API's boolean produced a wrong report.

There is a third detail in the same method: a **forceloaded chunk is skipped**. Forceloading is what an operator does to run block verification (Chapter 2's trap 1), and silently unpinning it would break their diagnosis mid-investigation.

21.5 What close does not do

`onDisable` does not delete blocks.

Shutdown is not a teardown. The world is reset on the next start anyway, so wiping now would only lengthen the stop. The one thing `onDisable` does is move players out of the dungeon world — because their exit location outlives the instance.

`/tdungeons free` is not `close`.

It releases a raw slot and still leaves blocks, deliberately, for the manual `paste` and `connect` commands. Using it on an instance leaves the dungeon standing while the slot gets handed out again.

Instance ids are never reused.

Slot indices are. A slot is a *place* and may be handed out again; an instance is an *event*. Reusing the number would make a log line about "instance#3" ambiguous between two different dungeons.

21.6 The rescue on join

A player who logs out inside an instance has that position written to their player data. The instance does not survive the restart, so on their next join they load into a void world standing on nothing. The same happens, without a restart, to anyone who is offline when their dungeon expires.

```
DungeonInstance standing = instances.instanceAt(loc);
if (standing != null && standing.contains(event.getPlayer().getUniqueId())) {
    // Their dungeon is still up and they are still a member: put them back on the bar
    // and leave them where they were. Logging out is not leaving.
    standing.showBar(event.getPlayer());
    return;
}
instances.teleportInternal(event.getPlayer(), instances.fallbackExit());
```

InstanceListener.onJoin — and the reason it fires only on join.

> **WHY JOIN, AND NOT EVERY WORLD CHANGE**
>
> Entering the dungeon world is a legitimate thing to do — `/tdungeons world` puts an operator there on purpose, and the portal puts players there. A rescue that fired on every world change would drag them straight back out. Logging in, by contrast, is never a deliberate entry: the position is a leftover.

21.7 Verified

Console-verified on 2 September 2026. The floor block at `(256,63,256)`: solid after generation → air after `close` → solid again after regenerating into the same slot, with no overlap. The close report read 6,551 blocks, 25 chunks, 2 entities (a zombie and an armour stand), 155 ms. Six chunk files were deleted at startup. No exceptions in the log.

CHAPTER SUMMARY

- Players out, entities out, blocks wiped, chunks unloaded, slot released — last.
- Members go to their own return locations before the positional sweep, or everyone lands at world spawn.
- The entity sweep must load chunks first; FAWE leaves them unloaded and an entity search only sees loaded ones.
- `unloadChunk`'s boolean is a ticket answer, not a success answer.
- Shutdown is not teardown; `free` is not `close`; instance ids are never reused.

CHAPTER 22

The clock, the boss bar and the warnings

One repeating task drives every live dungeon. This chapter covers why it is one task and not twenty, why the boss bar drains rather than fills, and why threshold warnings compare with `<` rather than `==`.

22.1 One shared task, once a second

```
/**
 * Starts the once-a-second tick that drives expiry, warnings and the boss bar.
 *
 * One shared task rather than a timer per instance: the work per instance is a subtraction
 * and a bar title, and twenty scheduled tasks doing that would cost more in scheduling than
 * in work. A second is also the finest resolution that matters — the bar shows whole seconds.
 */
public void startClock() {
    if (clock != null) {
        return;
    }
    clock = plugin.getServer().getScheduler().runTaskTimer(plugin, this::tick, 20L, 20L);
}
```

InstanceManager.startClock.

```
private void tick() {
    long emptyTimeout = plugin.getConfig().getLong("instance.empty-timeout-seconds", 300) * 1000L;
    for (DungeonInstance instance : all()) {
        if (!instance.isActive()) {
            continue;
        }
        if (instance.isExpired()) {
            expire(instance, "Süre doldu");
            continue;
        }
        // An instance nobody is in is holding a slot for nothing. Only armed once somebody
        // has been inside: a dungeon an operator generated to look at has been empty since
        // birth, and closing it on that basis would delete the rooms they are standing in.
        if (emptyTimeout > 0 && instance.everOccupied() && instance.playerCount() == 0
                && instance.emptyMillis() >= emptyTimeout) {
            expire(instance, "İçeride kimse kalmadı");
            continue;
        }
        updateBossBar(instance);
        warnIfDue(instance);
    }
}
```

The tick body — three decisions per instance.

22.2 Expiry sends people home first

```
private void expire(DungeonInstance instance, String reason) {
    for (UUID uuid : instance.players()) {
        Player player = plugin.getServer().getPlayer(uuid);
        if (player != null) {
            player.sendMessage(Component.text(reason + " — dungeon kapanıyor.",
                    NamedTextColor.YELLOW));
        }
    }
    // Players first, and to THEIR OWN return location — close() only knows the generic way
    // out and would drop everyone at world spawn.
    sendEveryoneHome(instance);
    close(instance).exceptionally(error -> { ... });
}
```

expire — note the comment on why sendEveryoneHome is called before close.

A member who is offline when the dungeon expires is simply deregistered — there is nothing to teleport. The join safety net (Chapter 21) picks them up when they return.

22.3 The boss bar drains, and its colour is information

The bar empties rather than fills, and it changes colour on its own as the fraction falls: **blue → yellow below 25% → red below 10%**.

> **COLOUR IS WHAT A PLAYER READS WITHOUT LOOKING**
>
> The number tells you how much time is left. The colour tells you whether you need to care. Those are two different questions and the second one gets answered peripherally, while the player is fighting something.

The title comes from `instance.boss-bar.title` as MiniMessage, with `<time> <theme> <size> <rooms>` substituted **unparsed**.

> **A BROKEN TAG MUST NOT FLOOD THE CONSOLE**
>
> The title is re-rendered once a second per instance. A malformed MiniMessage tag would therefore throw once a second, forever, in a loop nobody is watching. The render is wrapped in a `catch` that falls back to plain text.
>
> Compare with the HUD, where the same class of problem is solved the other way: there, MiniMessage is validated at *load* time, the bad line is skipped, and the operator is told which one. The rule is the same in both — **never let a formatting error live inside a timer** — and the implementation differs because the HUD has a load step to validate in and the bar does not.

22.4 Warnings fire below the threshold, never on equality

```
private void warnIfDue(DungeonInstance instance) {
    if (instance.playerCount() == 0) {
        return;
```

```
    }
    long remainingSeconds = instance.remainingMillis() / 1000L;
    for (int threshold : plugin.getConfig().getIntegerList("instance.warn-seconds")) {
        if (remainingSeconds > threshold || !instance.markWarned(threshold)) {
            continue;
        }
        // ... announce
    }
}
```

warnIfDue — the comparison, and the once-only guard.

> **WHY NOT REMAININGSECONDS == THRESHOLD?**
>
> Because a per-second tick can be late. On a server that stutters — which is every server, some of the time — an equality test would **silently skip the warning** exactly when the server was struggling. Comparing with `<=` and guarding with `markWarned` makes each threshold fire once, late if necessary, but never not at all.

Warnings only fire while somebody is inside. A dungeon counting down with nobody in it has nobody to warn.

22.5 What is fixed at creation and what is read live

SETTING	WHEN IT TAKES EFFECT
`instance.duration-seconds`	Fixed at `register`. Changing the config does **not** affect open instances — a dungeon's clock must not move under the people inside it.
`instance.empty-timeout-seconds`	Read every tick.
`instance.warn-seconds`	Read every tick, so `reload` applies immediately.
`instance.boss-bar.title`	Read every tick.

> **ONDISABLE MUST REMOVE THE BARS**
>
> A boss bar lives on the client, exactly like the sidebar scoreboard. If it is not removed, then after a `/reload` the player is left with a counter ticking down with no plugin behind it. The same rule applies to the HUD.

CHAPTER SUMMARY

- One shared per-second task; per-instance work is a subtraction and a bar title.
- The empty timeout only arms after someone has been inside.
- The bar drains; colour answers "should I care", the number answers "how long".
- Warnings compare below the threshold and are guarded once-only, because ticks can be late.
- Duration is frozen at creation; everything else is read live.

CHAPTER 23

Membership is declared, not derived from position

This is a small chapter about one distinction, because that distinction quietly determines the correct behaviour of five different features.

23.1 Two questions that look like one

QUESTION	ANSWERED BY	USED FOR
Who is a member?	`instance.players()`	Who sees the boss bar; who gets teleported home when time runs out; when the idle counter starts.
Who is standing there?	`instanceAt(location)`	Who gets swept out of a slot that is about to be wiped.

They diverge in both directions, and both divergences are intentional:

- A player who **logged out inside** is still a member and is not standing there. Logging out is not leaving.
- A player who **fell through a hole** into the void below the rooms is still standing there — because `GridSlot.contains` ignores Y (Chapter 15) — and is still a member.
- An **operator who walked in** with `/tdungeons world` is standing there and is *not* a member. They get swept out at teardown; they never see the boss bar.

23.2 Entering makes you a member; teleporting does not

> **A BEHAVIOUR CHANGE THAT WAS MADE DELIBERATELY**
>
> `/tdungeons dungeon` used to teleport the operator to the slot's coordinates. It now **puts them inside**. Teleporting to a slot makes you someone standing there; entering makes you a *member* — and the counter, the bar and the end-of-time teleport all work through membership.

23.3 Quit and world change deregister; they never teleport

On quit

Deregister only. Teleporting during quit means racing the write of the player's position to disk. The join safety net closes the same case without racing anything.

On world change

Deregister only. The player is already outside; dragging them "home" would be the plugin taking over a movement the player made.

On join, still a member, dungeon still up

Leave them exactly where they are and give the boss bar back.

23.4 The exit that does not exist yet

> **A DELIBERATE GAP IN PHASE 2B**
>
> An ordinary player has **no way to leave voluntarily**. `/tdungeons leave` requires admin permission and teleporting is blocked. What gets you out is the timer, or dying. The real door is the portal (Chapter 25) — and that was 2C's job. This is recorded in the project's own notes as a known, intentional gap rather than an oversight.

CHAPTER SUMMARY

- Membership is declared; position is measured. They are used for different things.
- Logged out inside = still a member. Fell into the void = still both.
- Entering makes you a member; teleporting to the coordinates does not.
- Quit and world change deregister without teleporting, to avoid racing player-data writes.

CHAPTER 24

Blocking teleports without blocking the plugin

"`/tp` and `/tpa` do not work inside a dungeon, admins excepted" is one line in the design charter and about forty lines of code, most of which exist to solve one problem: the plugin's own teleports look exactly like the ones being blocked.

24.1 Why the rule exists at all

Without it the whole instancing design leaks. A player teleports a friend past the boss, or teleports out with the loot and back in on a fresh timer. At that point the instance stops being an instance.

24.2 Both ends are tested

```
/**
 * Both ends are tested. Blocking only the destination would still let a player teleport out of
 * a dungeon with the loot; blocking only the origin would let a friend drop in past the boss.
 * Teleports between two slots are blocked by the same test, which is right — that is one party
 * walking into another's instance.
 */
@EventHandler(priority = EventPriority.HIGH, ignoreCancelled = true)
public void onTeleport(PlayerTeleportEvent event) {
    if (!BLOCKED_CAUSES.contains(event.getCause())) {
        return;
    }
    if (!isDungeonWorld(event.getFrom()) && !isDungeonWorld(event.getTo())) {
        return;
    }
    Player player = event.getPlayer();
    if (player.hasPermission(BYPASS_PERMISSION)) {
        return;
    }
    InstanceManager instances = plugin.getInstanceManager();
    // Our own moves have cause PLUGIN too. Rather than let the rule guess whose teleport it
    // is looking at, the manager announces its own for the length of the call.
    if (instances != null && instances.isInternalTeleport(player.getUniqueId())) {
        return;
    }
    event.setCancelled(true);
    player.sendMessage(Component.text("Dungeon içinde ışınlanma kapalı.", NamedTextColor.RED));
}
```

InstanceListener.onTeleport.

24.3 Which causes, and which are deliberately allowed

```
/**
 * The teleport causes that count as "being teleported" rather than playing.
 *
```

```
 * Ender pearls and chorus fruit are deliberately absent: they are items with a cost, used
 * inside the room the player is standing in, and blocking them would be a nerf to gameplay
 * rather than a closing of the exploit. COMMAND is vanilla /tp; PLUGIN covers every /tpa,
 * /home and /warp plugin there is.
 */
private static final List<PlayerTeleportEvent.TeleportCause> BLOCKED_CAUSES = List.of(
        PlayerTeleportEvent.TeleportCause.COMMAND,
        PlayerTeleportEvent.TeleportCause.PLUGIN,
        PlayerTeleportEvent.TeleportCause.SPECTATE);
```

BLOCKED_CAUSES — three entries, and a documented omission.

The distinction being drawn is not "magic versus commands". It is **whether the movement has a cost and stays local**. An ender pearl is thrown across the room you are in. A `/home` is not.

24.4 The plugin marks its own teleports

Every teleport the plugin performs has cause `PLUGIN` — the same cause the rule blocks. Rather than have the rule try to guess whose teleport it is looking at, `InstanceManager.teleportInternal` marks the player for the duration of the call:

```
teleportInternal(player, destination):
        mark player as "internal"
        player.teleport(destination)          <- the event fires SYNCHRONOUSLY, inside here
        unmark player

  no timer, no window: the event cannot leak into a later tick
```

The flow, and why no time window is needed.

> **A TIME WINDOW WOULD HAVE BEEN THE OBVIOUS IMPLEMENTATION AND THE WRONG ONE**
>
> "Mark the player for 500 ms" is easy to write and creates a real hole: for half a second, that player's *own* `/tp` would also pass. Because Bukkit fires the teleport event synchronously inside `teleport()`, the mark can be exactly as wide as the call and no wider.
>
> The corollary is a rule for the rest of the codebase: **every teleport the plugin performs must go through `teleportInternal`**. One that does not will be blocked by the plugin's own rule.

24.5 The admin exception

`takashidungeons.teleport.bypass`, default `op`. The charter originally said "admins included"; that was revised on 2 September 2026. The reason is in `plugin.yml`'s own comment: without it, the rule would lock staff out of the very instances they need to reach in order to moderate.

CHAPTER SUMMARY

- Both origin and destination are tested; slot-to-slot teleports are blocked by the same test.
- `COMMAND`, `PLUGIN` and `SPECTATE` are blocked; ender pearls and chorus fruit are deliberately not.
- The plugin marks its own teleports for exactly the length of the synchronous call — no time window, therefore no hole.
- Every plugin teleport must go through `teleportInternal`.

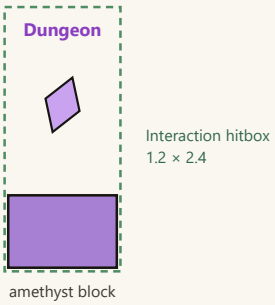
CHAPTER 25

The entrance portal: four pieces and two deaths

Up to Phase 2B the only way into a dungeon was an admin command, which is not a game — the player has to *find* the way in. The portal is that way in. It is also the smallest system in the project with the highest ratio of "why is it built like that" per line.

25.1 The visual is a placeholder, and it is four objects

An amethyst block, a slowly turning amethyst shard 1.25 blocks above it, the word "Dungeon" floating at 1.9, and an invisible box around the whole thing. All four are configurable (`portal.block`, `portal.item`, `portal.label`), so replacing the look later is an edit rather than a rewrite.



Block
The thing a right-click can actually hit. Protected, but never placed by force.

ItemDisplay — the shard
No hitbox, no gravity, no collision, no AI to tick. FIXED billboard, because a billboarded item turns to face every player and never looks like it is spinning.

TextDisplay — the label
CENTER billboard, so it always faces you, like a player's nametag.

Interaction — the reason there are four pieces
Displays cannot be clicked. A player aiming at the shard would click straight through it. This gives the visual the hitbox the eye already assumes is there.

Each entity type was chosen for a property. The fourth exists because of a mismatch between what the player sees and what the server can be clicked on.

> **THE USABILITY ARGUMENT FOR THE HITBOX**
>
> Clicking something that cannot be clicked does not read as "I missed". It reads as **"this portal is broken"**. That is the whole justification for a fourth entity.

25.2 The spin costs one update per second

```
e.setTransformation(rotation(0f));
// Interpolation is what turns one update a second into a smooth spin: the client
// eases between transformations instead of snapping.
e.setInterpolationDuration(20);
e.setInterpolationDelay(0);
e.setViewRange(0.6f);
```

Interpolation does the animation, not the server.

45° per second — a full turn every eight seconds — updated once a second and smoothed client-side. There is no per-tick task animating anything.

25.3 Two ways to click, and the off-hand trap

A player can right-click the block (`PlayerInteractEvent`) or the hitbox (`PlayerInteractEntityEvent`). Both are handled deliberately. Both check `EquipmentSlot.HAND`.

> **OFF-HAND FIRES A SECOND EVENT**
>
> Without the hand check, one right-click produces two events and therefore **two dungeons** — two slots consumed, one of them orphaned.

There is a second guard for a related case: a `generating` set holds portals whose dungeon is mid-generation. A second click during that window does not start another one.

25.4 Right-clicking an occupied portal joins, it does not open

If the portal's bound dungeon is still standing, a right-click puts the player *into that dungeon*. This is what lets a party's second player follow the first through the same door, and it is the seam Phase 5 will build on.

25.5 Two kinds, two different deaths

```
/**
 * Where a portal came from — which decides what happens to it once its dungeon ends.
 *
 * The difference is not cosmetic: a wild portal is a FIND, and a find that respawned on the
 * spot would stop being one. A lobby portal is FURNITURE, and furniture that vanished after
 * one use would leave the lobby with a hole in it.
 */
public enum PortalKind {
    /** Spawned out in the world by the plugin. Consumed when its dungeon ends. */
    WILD("doğa"),
    /** Placed by an operator at a fixed point. Comes back at the next refresh hour. */
    LOBBY("lobby");
}
```

PortalKind — the roadmap's "wild disappears, lobby resets" made concrete.

The state machine differs from `InstanceState` in one important way: it is **not** one-way. A lobby portal cycles `READY → OCCUPIED → COOLDOWN → READY` for as long as the server runs. What must not happen is two dungeons hanging off one portal, and `OCCUPIED` is what prevents it.

25.6 Refresh is an hour of the day, not a duration

```
/**
 * When a used lobby portal opens again — the next configured hour of the day.
```

```
     *
     * An hour of the day rather than a duration: a lobby is a shared place, and "the dungeons
     * refresh at noon and midnight" is something a server can announce and players can plan
     * around. A rolling timer would have every portal come back at a different, invisible moment.
     * With the list empty there is no waiting at all.
     */
```

nextRefreshTime — and the reasoning behind the choice.

Default `portal.lobby.refresh-hours: [0, 12]`. This is a design decision about *shared knowledge*, not about balance: the value of a fixed hour is that it can be announced.

25.7 Wild portals spawn around a player, not around spawn

SETTING	BEHAVIOUR
`portal.wild.enabled`	Master switch for spontaneous spawning.
`portal.wild.worlds`	Empty means any world.
`portal.wild.interval-minutes`	How often an attempt is made; clamped to at least one minute.
`portal.wild.max`	How many wild portals may stand at once.
`portal.wild.min-distance` / `max-distance`	The ring around the chosen player.
`portal.wild.spacing`	Minimum distance between two portals.

AROUND A RANDOM ONLINE PLAYER

A dungeon that appears where nobody is, is a dungeon nobody finds. If nobody is online, nothing spawns at all.

CANDIDATE POSITIONS ARE TESTED WITHOUT LOADING CHUNKS

Loading a chunk per attempt would turn a background dice roll into main-thread world generation. An unloaded candidate is skipped rather than loaded.

25.8 The block is protected, and only ever restored

Breaking, piston push *and* pull, explosions and liquid flow are all refused, and the `Interaction` entity is made immune to damage — otherwise a left-click could delete it and the portal would become block-only.

The manager's registration is keyed on the block's location, so if the block disappeared the display entities would be left hanging in mid-air with nothing tracking them. Refusing every way it could disappear is cheaper than detecting it afterwards.

Conversely, `portal create` only restores the previous block **if it placed the block itself**. If an operator changed it in the meantime, they did so on purpose.

25.9 Crash debris

Portals do not survive a restart — like the instances they open. A portal pointing at a dungeon that no longer exists is a trap for the first player who clicks it. Persistence arrives in Phase 7 and will persist the **lobby** portals; `portal.lobby.points` is today's permanent address for them.

The cost of that choice is debris after a hard kill: a block and three display entities with nothing tracking them. Both halves are handled.

The block is restored on a clean shutdown.

`onDisable` must remove portals; otherwise a clickable block is left behind with no plugin behind it.

The entities are tagged and swept.

Display entities carry a PDC tag. Orphans are removed on enable *and* on every chunk load. The enable sweep only sees loaded chunks; the chunk-load hook catches the rest as the world is explored — otherwise there would be shards spinning over nothing in a corner of the map nobody visits.

25.10 Commands, and one asymmetry

```
/tdungeons portal list
/tdungeons portal create [size] [theme]
/tdungeons portal remove <id|all>
/tdungeons portal tp <id>
```

The portal commands.

`create` places a **lobby** portal only. Placing a "random discovery" on request is a contradiction in terms.

25.11 Verified

Console-verified 2 September 2026: a lobby portal is built from config, the block is amethyst, all three entities are in place; `portal remove` restores the previous block and deletes the entities; after a **hard kill**, the next startup cleaned up 3 orphaned pieces and left exactly one intact portal.

In-game verified 5 September 2026, locally with one account and on a hosted server with two: the portal's appearance, right-clicking both the block and the shard, the boss bar, the threshold warnings, returning to the point of entry when time expires, wild spawning, the wild portal being consumed, and the lobby cooldown. With two accounts: the `/tp` block, a second player *joining* through the same portal, leaving by dying, and the empty timeout. Two gaps were found and closed in the same session — instance open/close logging, and announcing the reason when `enter()` refuses.

CHAPTER SUMMARY

- Four objects: a block to click, two Display entities with no hitbox, and an Interaction entity to supply the hitbox the eye expects.
- The spin is client-side interpolation — one update per second.
- Both click paths check the hand, or one click opens two dungeons.
- An occupied portal is joined, not reopened — the seam for parties.
- Wild is consumed, lobby resets at an announced hour.
- The block is protected against every way it could vanish; orphan entities are swept on enable and on chunk load.

PART THREE

The mob system

Phase 3. The plugin never creates a mob type — it registers mobs that already exist and decides where they stand, which class they come from and how hard they hit. 3A and 3B are complete; 3C is the current work.

CHAPTER 26

The provider abstraction, and availability versus existence

The design charter has a rule: adding a new mob source must mean "implement `MobProvider`", not "edit the spawner". This chapter is that rule turned into an interface, plus the two decisions that make it work when the plugin it names is not installed.

26.1 The plugin does not create mobs

```
MobProvider (interface)
├── VanillaMobProvider     -> spawn by EntityType. Always available. The fallback.
├── MythicMobsProvider     -> spawn through MythicMobs, via reflection. Softdepend.
└── [future]               -> EliteMobs, etc.
```

Three providers, one of which does not exist yet.

The spawner in Phase 3B knows nothing about MythicMobs. It holds a `MobDefinition` and asks whoever owns that definition's provider id to produce an entity.

> **THIS INTERFACE IS PUBLIC API SURFACE**
>
> Phase 8 exposes it so addons can register their own source, and the charter forbids breaking changes to it. New capabilities therefore arrive as `default` methods — `defaultStatOverride()` is already one.

26.2 Availability is asked, never assumed

```java
/** Whether this provider can spawn anything right now. */
boolean isAvailable();

/**
 * Whether this provider recognises the key.
 *
 * Checked at load time so a typo is reported when mobs.yml is read, not the first time a
 * player walks into the room that would have contained the mob.
 */
boolean supports(String mobKey);

/**
 * Spawns the mob at the location, on the main thread.
 *
 * @return the spawned entity, or null if the provider refused — a key it does not know, or a
 *         plugin that went away between the load and now. Never throws for a missing mob; a
 *         dungeon with one absent mob type must still be enterable.
 */
@Nullable LivingEntity spawn(String mobKey, Location location);
```

MobProvider — the two methods that carry the design.

WHY MYTHICMOBSPROVIDER IS REGISTERED EVEN WHEN MYTHICMOBS IS ABSENT

Registering conditionally would push the same question into every call site, and — more importantly — the *reason* a definition is disabled would be lost. A conditional registration makes `mythicmobs:Foo` report "unknown provider": a correct symptom with the wrong diagnosis. Today's answer is "MythicMobs is not installed", which is what the operator actually needs to read.

26.3 MythicMobs through reflection, and the cost of that

There is no Maven dependency on MythicMobs. Adding one would put Lumine's repository into `pom.xml`, and the project would then fail to build for anyone who cannot reach that host — disproportionate for two method calls.

	CONSEQUENCE
The cost	If the MythicMobs API changes, the failure is a runtime error rather than a compile error.
The mitigation	Any reflection failure disables the provider **entirely** and logs it **once** (a `broken` flag). It never throws inside the spawn loop, and it never silently falls back to vanilla.

26.4 statOverride: whose stats win

```
/**
 * Whether definitions from this provider have their stats overridden unless told otherwise.
 *
 * true for vanilla — a vanilla zombie has no stat design to protect. false for external
 * sources: a MythicMobs mob arrives with the stats its author wrote, and overwriting them by
 * default would delete the reason that plugin is installed. An explicit statOverride in
 * mobs.yml always wins.
 */
default boolean defaultStatOverride() {
    return true;
}
```

The default is chosen by the provider, and an explicit value always wins.

`/tdungeons mob info` shows the *resolved* value **and where it came from** — which is the operator's actual question when a mob is not behaving as expected.

CHAPTER SUMMARY

- The plugin registers existing mobs; it never defines a mob type.
- Providers are always registered; availability is a separate question, so the failure message names the real cause.
- MythicMobs is reached by reflection to avoid a build-time repository dependency; any reflection failure disables that provider once, loudly, with no silent fallback.
- `statOverride`'s default comes from the provider — vanilla true, external false. An explicit value always wins.

CHAPTER 27

The registry: mobs.yml, addressing, and classes as pool tags

Seventeen mob definitions ship in the jar and all of them are vanilla. This chapter covers why they live in their own file, how a definition names its mob, and the decision that keeps a configured number and an in-game number the same number.

27.1 mobs.yml is a separate file, and that is a Phase 9 decision

> **COMMENTS DO NOT SURVIVE A PROGRAM REWRITING A FILE**
>
> In Phase 9 a GUI editor will **write** this file. A file rewritten by a program loses its comments. If the mob set lived in `config.yml`, every explanatory comment in that file would be deleted the first time somebody saved from the GUI. So the mob set has its own file, and `config.yml`'s documentation survives.

27.2 Addressing is `provider:key`

```
mob: vanilla:ZOMBIE              # vanilla entity registry — always works
mob: mythicmobs:SkeletalKnight   # MythicMobs — THIS ENTRY is disabled if not installed
```

Two forms of address. The second disables itself when MythicMobs is absent.

If the provider is missing, or the provider does not recognise the key, that entry is **disabled with its reason**: one line in the console and a red block in `/tdungeons mob list`.

> **IT NEVER SILENTLY FALLS BACK TO VANILLA**
>
> A silent fallback produces the bug "why is the boss a normal zombie", and that bug comes back weeks later disguised as a balance complaint. Meanwhile the out-of-the-box guarantee is met a different way: **the bundled set is entirely vanilla**, so all seventeen definitions work on a server with no mob plugin at all.

27.3 A class is a pool tag, not a multiplier

```java
/**
 * A class does NOT scale anything. It answers one question — "which pool may this mob be drawn
 * from" — and the stats come from the definition's own ranges.
 *
 * The alternative, giving each class a stat multiplier, was rejected for the same reason
 * generation.md §5.4 keeps room weight on the template: an operator who writes
 * health: [40, 50] and sees 120 in game has no way to find out where the number came from.
 * One number, one place.
 */
public enum MobClass {
    WEAK, NORMAL, STRONG, SUPER_STRONG,
    /**
     * Drawn only for the boss room, and never by the ordinary room spawner — a boss that can
     * turn up in a corridor is not a boss.
     */
    BOSS
}
```

MobClass — the decision, and its parallel elsewhere in the codebase.

This is the same decision as Chapter 10's "weight belongs to the template", applied to a different system. Both are instances of one principle: **a number the operator writes must be the number that happens**, unless something they can also see changes it.

Difficulty is the one thing in this system that genuinely *is* a multiplier — and it is visible, global, and named in the same file.

27.4 A definition

```yaml
vindicator_enforcer:
  mob: vanilla:VINDICATOR
  class: strong
  weight: 90
  health: [38, 46]
  damage: [7, 9]
  name: "<gray>Balta Ustası</gray>"

ravager_bull:
  mob: vanilla:RAVAGER
  class: super_strong
  weight: 60
  health: [90, 110]
  damage: [9, 12]
  speed: 0.28
  name: "<red>Yıkıcı</red>"

bone_lord:
  mob: vanilla:WITHER_SKELETON
  class: boss
  weight: 100
  health: [220, 260]
  damage: [12, 15]
  speed: 0.26
  name: "<dark_red><bold>Kemik Lordu</bold></dark_red>"
```

Three of the seventeen bundled definitions.

`weight` means exactly what it means for room templates and what it will mean for loot: a share within the same pool. 200 comes up twice as often as 100. Stats may be a single number or a `[min, max]` range. An unwritten stat leaves the entity's natural value alone.

27.5 What the dungeon world implies for mob choice

The bundled set is shaped by three properties of the world it spawns into, and these are worth knowing before adding to it:

`Environment.NORMAL`
: Piglins and piglin brutes **zombify** here, so neither is in the default set.

`mobGriefing` is off
: A creeper explosion does not break blocks, so creepers cannot punch a hole in a room. This is what makes them usable at all.

The daylight cycle is off and rooms are sealed
: There is no sky light, so zombies and skeletons do not burn. A room schematic with an open ceiling changes that.

27.6 Commands and the enable log

```
/tdungeons mob list [class]
/tdungeons mob info <id>
/tdungeons mob spawn <id> [difficulty]
/tdungeons mob providers
/tdungeons reload

enable log:  17 mob (weak=3, normal=5, strong=4, super_strong=2, boss=3)
```

The mob commands, and what a healthy startup looks like.

`/tdungeons reload` re-reads the mob registry, and it does so **before** the WorldEdit check — a server with no WorldEdit must still be able to fix a typo in `mobs.yml` without a restart.

CHAPTER SUMMARY

- `mobs.yml` is separate because Phase 9's GUI will rewrite it and comments do not survive that.
- Addressing is `provider:key`; a bad address disables that entry with a reason and never falls back silently.
- A class is a pool tag. Only difficulty multiplies anything.
- The bundled set is entirely vanilla, which is how the out-of-the-box guarantee is met.

CHAPTER 28

Stats, difficulty and the health ordering bug everyone hits

`MobService` turns a definition into a mob standing in the world. It is a short class with a strict order of operations, and one line in it is the difference between a working boss and a boss that dies in three hits.

28.1 The order: spawn, stats, equipment, name

```
LivingEntity entity = provider.spawn(definition.mobKey(), location);
if (entity == null) {
    return null;
}

if (definition.baby() && entity instanceof Ageable ageable) {
    ageable.setBaby();
}

if (definition.resolveStatOverride(provider)) {
    DifficultyScaling scaling = registry.scaling(difficulty);
    applyStats(entity, definition, scaling, random);
    applyArmor(entity, scaling, random);
}
applyName(entity, definition);
applyDungeonBehaviour(entity);
return entity;
```

MobService.spawn — the whole flow.

> **MAX HEALTH AND CURRENT HEALTH ARE TWO DIFFERENT VALUES**
>
> Raising `MAX_HEALTH` does **not** raise current health. A mob given 200 max health walks out of the spawner on 20 and dies in three hits. This is the single most common way a stat system looks broken, and the fix is one line — heal the entity after setting the attribute.

```
private void applyStats(LivingEntity entity, MobDefinition definition,
                        DifficultyScaling scaling, RandomGenerator random) {
    if (definition.damage() != null) {
        set(entity, Attribute.ATTACK_DAMAGE,
                definition.damage().roll(random) * scaling.damage());
    }
    if (definition.speed() != null) {
        set(entity, Attribute.MOVEMENT_SPEED,
                definition.speed().roll(random) * scaling.speed());
    }
    if (definition.health() != null) {
        double max = definition.health().roll(random) * scaling.health();
        if (set(entity, Attribute.MAX_HEALTH, max)) {
            // Max health and current health are separate values. Without this line every mob
            // arrives at whatever health its species starts with, and a 200-hp boss dies like
            // a zombie.
            entity.setHealth(Math.min(max, entity.getAttribute(Attribute.MAX_HEALTH).getValue()));
        }
    }
}
```

applyStats — health is written last among the stats, then the entity is healed.

The attribute writer returns `false` when the species has no such attribute — a slime has no attack damage attribute — and refusing quietly is correct there. The definition is not wrong; the attribute simply does not exist on that species.

28.2 statOverride is the whole contract

> **FROM THE CLASS COMMENT**
>
> "When it is off, this class touches nothing but the name — no attributes, no armour. That is not a limitation to work around; it is the promise made to every mob plugin the server has installed. **Difficulty scaling rides on the same flag, because a multiplier applied to somebody else's carefully written boss is still an override.**"

28.3 Three multipliers, not one

```
/**
 * Scaling speed the way health is scaled turns a hard dungeon into a track meet: at 2.2×
 * movement speed nothing can be kited, fled from, or fought with a bow, and the fight stops
 * being a fight. Health tolerates a large multiplier, damage a moderate one, speed almost none
 * — so they are three separate numbers and the defaults reflect that.
 */
public record DifficultyScaling(double health, double damage, double speed,
                                ArmorTier armor, double armorChance) { }
```

DifficultyScaling — why speed is not scaled like health.

DIFFICULTY	HEALTH	DAMAGE	SPEED	ARMOUR	CHANCE
`easy`	1.0	1.0	1.0	none	—
`medium`	1.5	1.25	1.05	leather	0.35
`hard`	2.2	1.6	—	—	—

The shape is what matters: health takes the big multiplier, damage a moderate one, speed barely moves.

The numbers are not hard-coded in the enum. `Difficulty` is just a name; the values live in `mobs.yml`. An enum constant with a literal 2.5 in it is a number nobody can tune without recompiling, and "everything is configurable" is a project rule.

28.4 Armour is the visible half of difficulty

```
/**
 * A hard dungeon whose only difference is a bigger health number is a longer fight, not a
 * harder one — and the player cannot see it. Armour is the half of difficulty that is VISIBLE
 * BEFORE THE FIRST HIT: a mob in diamond reads as dangerous across a room. That readability is
 * the point; the damage reduction is a side effect.
 */
```

ArmorTier — readability first, damage reduction second.

`armor-chance` is deliberately not 1.0. At 100% every skeleton in the dungeon is dressed identically and the armour reads as a *uniform*. At 35% it reads as *danger*.

> **DROP CHANCE IS ZERO, AND THAT IS A LOOT-SYSTEM DECISION**
>
> Loot is Phase 4's system and comes from a table the operator writes. Armour that falls off a mob would be a second, invisible loot source that nobody configured and that **scales with difficulty** — exactly the leak that makes a hard dungeon the most profitable place to farm iron.

28.5 Ranges, rolled per spawn

```
/**
 * Two zombies in the same room with the same health bar read as copies of one object. A range
 * costs nothing and makes a group of mobs look like a group of individuals. The range is also
 * the unit phase 9's GUI will edit — two sliders, not a text field.
 *
 * A single number in YAML is accepted and becomes a zero-width range, so an operator who does
 * not want variance never has to write [20, 20].
 */
```

StatRange — why a range rather than a number.

Parsing **throws** on a malformed value rather than falling back to a default, and the error message names the file and the field. A mistyped stat produces a mob that is wrong in a way nobody notices until a player complains it hits too hard.

28.6 Dungeon behaviour

Mobs do not despawn.

`setRemoveWhenFarAway(false)`. A room that empties itself reads as a generation bug.

Mobs do not pick up items.

A mob that collects loot deletes the player's reward.

Spawning is main-thread only, and the check is here.

`MobService.spawn` asserts `isPrimaryThread()` itself rather than trusting callers, because Phase 3B populates rooms downstream of a FAWE chain — a mistake that is easy to make once and hard to see afterwards.

CHAPTER SUMMARY

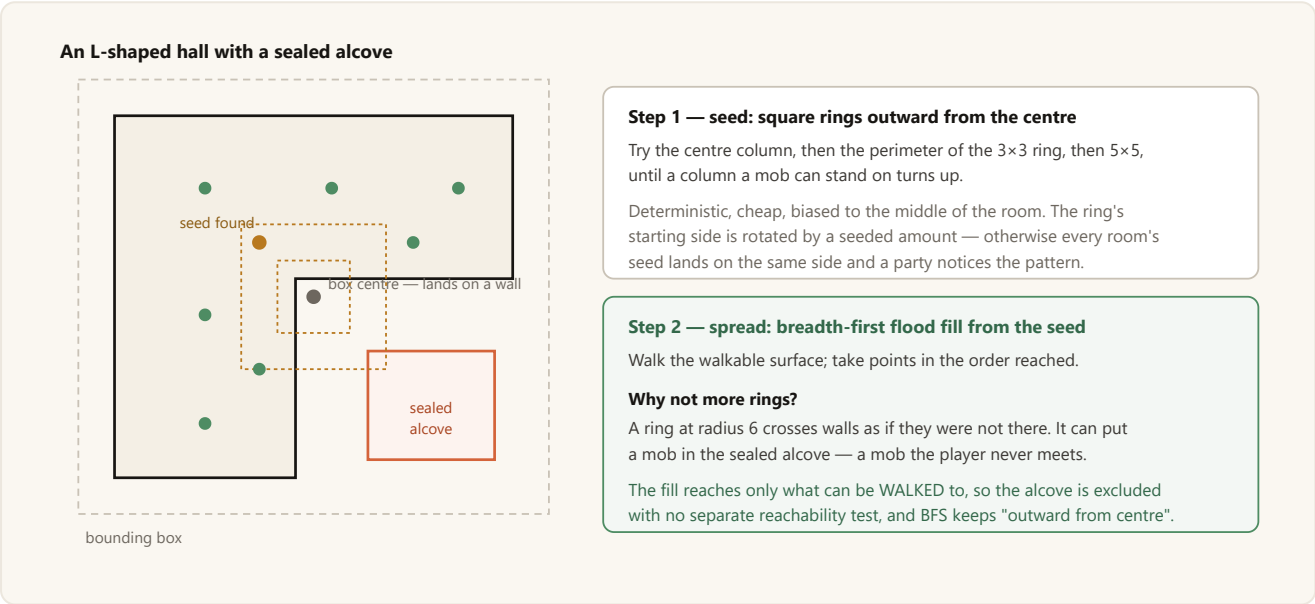
- Provider spawns, then stats, then equipment, then name. Health is set last among the stats and the entity is healed afterwards.
- `statOverride: false` means *nothing* is touched but the name — difficulty included.
- Three multipliers because speed cannot be scaled like health without ending the fight.
- Armour is difficulty you can see; its drop chance is zero so it does not become an unconfigured loot source.
- Stats are ranges rolled per spawn; parsing throws rather than defaulting.

CHAPTER 29

Finding somewhere to stand: ring seed and flood fill

Nothing about mob placement is written into a room's `.yml`. The engine finds standing positions at runtime, and the two-step search it uses is the reason a sealed alcove never receives a mob the player cannot reach.

29.1 Two steps, two different jobs



Rings assume the walkable area is a blob centred on the box. Rooms are not required to be rectangles.

29.2 Three constants, each with a stated reason

```
/**
 * How much clear space a mob needs above the block it stands on.
 *
 * Two blocks is the vanilla mob envelope. A ravager is taller, but requiring three would rule
 * out every low-ceilinged corridor, and a mob that spawns slightly clipped settles itself; one
 * that never spawns leaves an empty room.
 */
static final int HEADROOM = 2;

/**
 * Minimum distance between two chosen points, in blocks.
 *
 * Without it the fill hands the second mob the cell next to the first, and the group reads as
 * one mob with a rendering fault rather than as a group. Three blocks is the smallest gap that
 * still looks deliberate in a nine-wide room.
 */
static final int MIN_SPACING = 3;
```

```
/**
 * How far a neighbouring column's floor may sit above or below this one and still count as
 * connected.
 *
 * One block: stairs, slabs and a raised podium stay walkable, while the floor above stays a
 * separate storey. Allowing two would merge a mezzanine into the ground floor and let a mob
 * spawn on a balcony the fill had no business reaching.
 */
static final int STEP_TOLERANCE = 1;
```

RoomSpawnFinder's tuning constants.

There is a fourth rule that is not a constant: **no diagonal steps**. Two blocks meeting at a corner would let the fill squeeze between them and cross a wall the player cannot.

29.3 survey() and spread() are separate on purpose

```
/**
 * Every column of the room that can be walked to from the centre, in breadth-first order.
 *
 * Separate from spread because the SIZE of this list is itself an answer: phase 3B sizes a
 * room's mob count by how much floor it actually has, and a cross-shaped room has far less
 * walkable floor than its bounding box suggests. Surveying once and using the result twice is
 * also cheaper than measuring the room and then searching it.
 */
public List<Vec3i> survey(Aabb box, RandomGenerator random) { ... }

/**
 * Takes up to count points from a surveyed surface, keeping them apart.
 *
 * A short list is information rather than failure: a room whose floor is mostly furniture
 * genuinely has fewer places to put a mob, and reporting three where five were asked for is
 * more useful than quietly stacking two of them on one block.
 */
public List<Vec3i> spread(List<Vec3i> surface, int count) { ... }
```

The split, and why the surface list's SIZE is itself an answer.

> **BREADTH-FIRST IS NOT AN IMPLEMENTATION DETAIL**
>
> It is what makes a *prefix* of the result mean "fill outward from the middle". A depth-first walk would run down one corridor and put every mob in it while the rest of the hall stayed empty.

29.4 The seed starts from the box centre, not the origin

Because anchor-based placement dropped the odd-edge rule (Chapter 5), a room may sit asymmetrically around its origin — and then "origin" is off to one side rather than in the middle. The seed search uses the *bounding box* centre.

The ring's starting side is rotated by a seeded amount. With a fixed start, every room's seed lands on the same side of centre whenever the centre itself is blocked, and a party walking through twenty rooms notices the pattern. The rotation removes the tell while keeping the dungeon reproducible.

29.5 ColumnProbe: the purity boundary again

```java
public interface ColumnProbe {

    /** Whether a mob can stand ON this block. */
    boolean isFloor(int x, int y, int z);

    /**
     * Whether this block is empty enough for a mob to occupy.
     *
     * Separate from !isFloor: a block can be neither floor nor free space. Water is not solid,
     * so it is not floor, but a mob standing in it has not been placed on land — and lava is
     * the same test with a worse outcome.
     */
    boolean isClear(int x, int y, int z);
}
```

The entire interface. Two methods, and one of them exists for a subtle reason.

> **THE JUSTIFICATION, IN THE CODE'S OWN WORDS**
>
> "Geometry that is wrong only on unusual room shapes is exactly the kind a server test does not catch: **every room still gets its mobs, they are simply in the wrong places.**"

`WorldColumnProbe` is the one-line Bukkit implementation. `scripts/geo-probe/SpawnProbe.java` implements the same two methods over hand-drawn room shapes and asserts **22 checks**: an L-shaped hall, a narrow corridor, a room with a closed centre, a two-storey room, a sealed alcove the fill must not reach, minimum spacing, same seed producing the same result, and a floorless room.

> **CHAPTER SUMMARY**
>
> - Ring search finds a seed; flood fill spreads from it. Rings alone would place mobs behind walls.
> - Headroom 2, spacing 3, step tolerance 1, no diagonals — each with a stated failure it prevents.
> - `survey` and `spread` are separate because the surface's size is what sizes the room's mob count.
> - Breadth-first ordering is what makes "take the first N" mean "fill from the middle".
> - `ColumnProbe` keeps the whole search testable without a server.

CHAPTER 30

How many, which class: SpawnRules

Two independent questions, answered by two independent inputs. How many mobs a room gets comes from its *floor area*; which class they are drawn from comes from its *relative depth*. Keeping them separate is what makes both of them tunable.

30.1 Count comes from walkable columns, not from depth

```java
/**
 * COUNT COMES FROM FLOOR AREA, NOT FROM DEPTH
 * The number of mobs is the room's walkable column count divided by density. Room size is the
 * thing the player can see: a great hall standing nearly empty and a corridor packed shoulder
 * to shoulder both read as bugs, and neither is fixed by knowing how deep the room is. Depth
 * changes WHAT is in the room, not how many.
 *
 * Walkable columns, not bounding-box area: a cross-shaped room's box is mostly wall, and
 * sizing by the box would put a hall's worth of mobs in the four arms of a cross.
 *
 * CLASS COMES FROM RELATIVE DEPTH
 * depth / maxDepth, not the raw depth. On absolute thresholds a four-room small dungeon would
 * be weak mobs from end to end — "small" means short, not harmless. Relative bands give every
 * size the same shape of difficulty curve over its own length.
 */
public record SpawnRules(int density, int minPerRoom, int maxPerRoom, boolean entranceMobs,
                         List<Band> bands) { }
```

SpawnRules — the record and the reasoning behind each half.

```java
/**
 * How many mobs a room with this much walkable floor gets.
 *
 * Clamped at both ends, and the floor only applies to a room that has SOMEWHERE to put a mob:
 * a sealed or floorless room gets nothing rather than one mob wedged into it.
 */
public int countFor(int walkableColumns) {
    if (walkableColumns <= 0) {
        return 0;
    }
    return Math.clamp(walkableColumns / density, minPerRoom, maxPerRoom);
}
```

countFor — clamped at both ends, with one asymmetry.

The early return is the asymmetry worth noticing. `min-per-room` means "a room that has floor gets at least this many", not "every room gets at least this many". A room the search could not find anywhere to stand in gets zero.

30.2 The depth bands

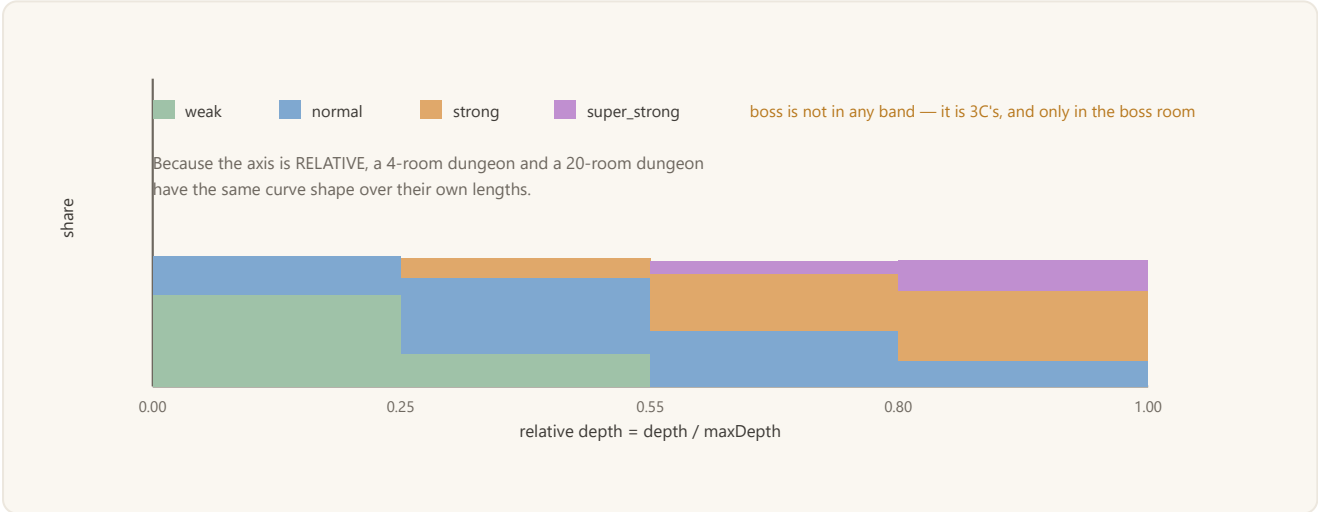
```yaml
spawn:
  # How many walkable blocks per mob. Raising it thins the rooms out.
  density: 45

  # Per-room floor and ceiling.
  min-per-room: 1
  max-per-room: 8

  # Whether the entrance room is populated. Default OFF: the player is teleported in and
  # should not land in the middle of a fight, and a party gathers there.
  entrance: false

  # Depth bands. until = upper bound of depth/maxDepth (inclusive);
  # classes = the class mix drawn inside that band.
  bands:
    - until: 0.25
      classes: { weak: 70, normal: 30 }
    - until: 0.55
      classes: { weak: 25, normal: 60, strong: 15 }
    - until: 0.80
      classes: { normal: 45, strong: 45, super_strong: 10 }
    - until: 1.00
      classes: { normal: 20, strong: 55, super_strong: 25 }
```

The spawn: block of mobs.yml, with defaults.



The difficulty curve. Weak mobs disappear by the middle; super_strong only appears in the last 45% of the way in.

Band weights use the same convention as everything else in the project: 70/30 means 70 to 30.

30.3 Two levels of graceful degradation

```java
/**
 * Reads the spawn: block. A malformed band is skipped with a message rather than taking the
 * whole file down — the rest of the mob set is still usable.
 */
public static SpawnRules parse(ConfigurationSection section) {
    if (section == null) {
```

```
            return DEFAULT;
        }
        ...
        bands.sort((a, b) -> Double.compare(a.until(), b.until()));
        return new SpawnRules(
                section.getInt("density", DEFAULT.density()),
                ...
                bands.isEmpty() ? DEFAULT.bands() : bands);
    }
```

Parsing: a bad band is skipped, not fatal.

Note that the bands are **sorted** after parsing. `bandFor` returns the first band whose `until` covers the ratio, so an operator who writes them out of order still gets the behaviour they meant.

> **THE UPGRADE CASE, AND A GOTCHA WORTH KNOWING**
>
> `SpawnRules.DEFAULT` exists for servers that upgrade the plugin while keeping a `mobs.yml` from Phase 3A, which has no `spawn:` block at all. The values are *identical* to the ones in the bundled file, so behaviour is correct — but the operator cannot edit a section that is not in their file. The plugin never overwrites an existing `mobs.yml`. The fix is to delete the file and run `/tdungeons reload`.

CHAPTER SUMMARY

- Count = walkable columns / density, clamped; a room with no floor gets nothing.
- Walkable columns rather than box area, because a cross-shaped room's box is mostly wall.
- Class comes from *relative* depth so every dungeon size has the same curve shape.
- A malformed band is skipped; bands are sorted after parsing; a missing `spawn:` block falls back to identical defaults.

CHAPTER 31

MobPopulator, and what 3C still has to do

The class that puts everything in this part together: for each room, survey the floor, size the group, draw a class, draw a mob, spawn it. Plus the three operational details that make it work in a live server, and an honest account of what is not finished.

31.1 What decides what

QUESTION	INPUT	MECHANISM
How many	the room's walkable floor	`survey().size() / density`, clamped
Which class	the room's relative depth	`bandFor(depth/maxDepth)`, then a weighted pick inside the band
Which mob	the class	weighted draw within that class's pool
How hard	the instance's difficulty	`MobService` applies the three multipliers and the armour
Where	the room's geometry	`spread()` over the surveyed surface

31.2 The per-room loop

```
RandomGenerator random = Seeds.derive(result.seed(), node.id());
List<Vec3i> surface = finder.survey(box, random);
int wanted = rules.countFor(surface.size());
List<Vec3i> points = finder.spread(surface, wanted);
if (points.size() < wanted) {
    shortOfSpace++;
}

double ratio = maxDepth == 0 ? 1.0 : (double) node.depth() / maxDepth;
SpawnRules.Band band = rules.bandFor(ratio);
...
for (Vec3i point : points) {
    MobClass mobClass = band.pick(random);
    MobDefinition definition = mobClass == null ? null : registry.pick(mobClass, random);
    if (definition == null) {
        // An empty class pool is a configuration gap, not an error: the operator may
        // simply have no super_strong mobs yet. The room ends up lighter, and the
        // report says so.
        refused++;
        continue;
    }
    Location where = new Location(world, point.x() + 0.5, point.y() + 1, point.z() + 0.5);
    LivingEntity entity = service.spawn(definition, where, difficulty, random);
    ...
}
```

MobPopulator — survey once, use the result twice.

Two details in the coordinate: `+0.5` on X and Z centres the mob in its block, and `+1` on Y puts it *on top of* the floor block the search returned — the search returns floor blocks, not standing positions.

31.3 Two rooms are skipped

```
/**
 * The entrance and the boss room are left alone; so is a room whose type says entrance even
 * when the graph did not make it the root, because a mapper marked it as a place players
 * arrive.
 */
private boolean isSkipped(LayoutNode node, DungeonGenerator.Result result, SpawnRules rules) {
    if (node.id() == result.bossNodeId()) {
        return true;
    }
    boolean entrance = node.id() == 0 || node.template().type() == RoomType.ENTRANCE;
    return entrance && !rules.entranceMobs();
}
```

isSkipped — and a case that is easy to miss.

The entrance

The player is teleported into it and should not arrive in the middle of a fight; a party also gathers there. Configurable via `spawn.entrance`.

The boss room

Phase 3C's job. A boss drawn by the ordinary room spawner would be a boss standing among four zombies — which is not a boss encounter, it is a room with a big zombie in it.

31.4 Reproducibility per room

```
RandomGenerator random = Seeds.derive(result.seed(), node.id());
```

Each room draws from its own derived stream.

The same seed therefore gives the same dungeon *and* the same mobs. More subtly: a room's contents do not shift because a different room happened to be populated first. This is exactly the property Chapter 13 built `derive` for.

31.5 The FAWE chunk trap, for the fourth time

```
/**
 * Loads every chunk the room's box touches.
 *
 * Synchronous on purpose. The alternative — populating asynchronously as chunks arrive — would
 * spawn mobs into a dungeon a player may already be walking through, and a room that fills up
 * behind you is worse than a two-tick pause before the door opens. The dungeon world is
```

```
 * void-generated, so a chunk that is not already resident costs almost nothing to make.
 */
```

loadChunks — synchronous, and the alternative is worse.

> **WITHOUT THIS, EVERY ROOM COMES BACK EMPTY**
>
> A block read in an unloaded chunk answers *air*. The floor survey would find no floor anywhere, `countFor(0)` would return 0, and the dungeon would be silently empty — with no error and no warning. The same fact appears in Chapter 2 (console verification), Chapter 16 (pasting), and Chapter 21 (the entity sweep).

31.6 The report, and what each number tells you

```
public record Report(int roomsPopulated, int roomsSkipped, int spawned, int refused,
                     int roomsShortOfSpace) { }

// Mob yerleştirildi: instance#4 — 23 mob, 8 oda, 2 reddedildi, 1 odada yer yetmedi
```

The Report record and the log line it produces.

NUMBER	WHAT IT MEANS WHEN IT IS NOT WHAT YOU EXPECTED
`refused`	A class pool was empty, or a provider declined. Configuration gap, not an error.
`roomsShortOfSpace`	The density asked for more points than the room's floor could hold apart. This is the number that explains a thin dungeon.
`roomsSkipped`	Entrance and boss, plus any room where nothing could be placed at all.

31.7 Performance, and where to look if it ever matters

`survey()` walks the whole room with no early exit. A 23×23 room is roughly 500 columns; a twelve-room dungeon is a few milliseconds, once, at creation. The class comment names this as the place to measure if room counts ever rise substantially — which is the right way to leave a known-cheap linear scan in place.

31.8 What 3C still has to do

ITEM	NOTES
Boss room spawn logic	The boss node id is already carried in `DungeonGenerator.Result` and the boss class pool already exists. What is missing is the encounter: one boss, placed deliberately, not drawn by the room spawner.
Instance ownership via PDC tag	Every spawned mob should carry its instance id in persistent data, so teardown can clean up precisely rather than by bounding box — and so that a mob which somehow escapes the box is still identifiable.
A kill signal	The seam Phase 4's loot and Phase 8's `MobKillEvent` both hang off.

31.9 Verified

In-game on 5 September 2026: dungeon generation, room occupancy, and the depth curve. Headlessly: `SpawnProbe`'s 22 assertions. Phase 3A was verified in game the same day — the enable log read `17 mob (weak=3, normal=5, strong=4, super_strong=2, boss=3)` with no disabled entries, and real spawns applied stats, armour and `statOverride` correctly.

> **THE ONE UNTESTED PATH**
>
> **The MythicMobs provider has never run.** The test server does not have MythicMobs installed, so that branch will be exercised for the first time on a server that does. This is recorded rather than glossed over — it is the single largest known gap in Phase 3.

CHAPTER SUMMARY

- Survey once; the size sizes the group and the list supplies the positions.
- Entrance and boss rooms are skipped; the entrance check also honours a mapper's `type: entrance` on a non-root room.
- Each room draws from `Seeds.derive(dungeonSeed, nodeId)`, so contents are stable under reordering.
- Chunks must be loaded first or every room comes back empty, silently.
- 3C remains: boss encounter, PDC ownership tags, and the kill signal.

PART FOUR

Patterns and reference

The principles that recur across all three phases, the traps that have actually cost time, and the command and configuration reference.

CHAPTER 32

Nine recurring design principles

Reading the three phases together, the same nine ideas keep reappearing in different clothes. If you are extending this codebase, these are the rules that will make your addition look like it belongs — and, more importantly, behave like the rest of it under failure.

32.1 Derive rather than declare

> THE ONE-LINE VERSION
>
> **Every field that can be written is a field that can be written wrong.**

VALUE	DERIVED FROM
A door's facing	the anchor vector, normalized against half-extents
A room's bounding box	the schematic itself
The size of a door opening	a bounded flood fill in the wall plane
The plug material	sampling the wall at the opening's edge
Mob spawn positions	a ring search plus a flood fill at runtime
A room's theme	the folder it sits in
The required rotation	`(d_p + 2 − d_c) mod 4`
A slot's position	its index

The map team's remaining job is three fields per room: type, weight, and one line per door. Forty rooms × five spawn points would have been two hundred numbers that could each be quietly wrong.

32.2 Measure before you commit, especially when the failure would be silent

The rotation sign was measured on a bare JVM *before* the placement code existed, because a wrong sign would have broken every door in every dungeon without throwing anything. The `Random` seed correlation was measured because it would have collapsed a range to a point without producing an error. The weight-ownership question was settled by computing both distributions on the actual room set rather than by argument. The boss ordering was settled by running 2,000 generations twice.

> **THE PATTERN TO COPY**
>
> When a decision has a plausible-sounding wrong answer *and* a failure mode that produces no exception, spend the twenty minutes to measure it. That is the only class of bug that survives to production in a system with this much self-checking.

32.3 A number the operator writes must be the number that happens

Room `weight` belongs to the template, not the `(template × door)` pair, because door count is not visible in the configuration file. `MobClass` is a pool tag, not a multiplier, because an operator who writes `health: [40, 50]` must not see 120. Difficulty is the one real multiplier in the mob system, and it is global, named, and in the same file.

The corollary: when you *do* want a factor, give it its own explicit, switchable knob. `generation.turn-bias` is that pattern — a separate coefficient in candidate ordering rather than something folded into the weight.

32.4 One-way state, enforced by throwing

STATE	TRANSITIONS	WHAT AN ILLEGAL ONE WOULD HAVE CAUSED
`DoorState`	OPEN → CONNECTED / DEAD	Two rooms attached to one door; the second silently lost
`InstanceState`	BUILDING → ACTIVE → CLOSING → CLOSED	A double close, releasing the slot twice and handing one square to two parties

`PortalState` is the deliberate exception — a lobby portal must cycle. What it guards instead is that two dungeons never hang off one portal.

32.5 Ordering is design, not implementation detail

Four places where the sequence *is* the correctness argument:

- **Plug after every paste** — earlier, the next room overwrites it; and whether a door is open is a whole-graph question.
- **Side branches before the boss** — measured; the other order produced 3-room small dungeons that never appeared and bossless mediums.
- **Populate before the future completes** — otherwise the player watches mobs appear around them.
- **Slot released last in teardown** — otherwise an allocation races the wipe.

32.6 Keep a pure core behind a two-method interface

`generation` knows nothing of Bukkit or WorldEdit. `RoomSpawnFinder` sees the world only through `ColumnProbe.isFloor` and `isClear`. `DungeonLayout` receives its slot bounds as an `Aabb` rather than asking a

`World`. In all three cases the payoff is the same: the logic that is *wrong in ways a server test cannot see* becomes testable in under a second.

32.7 Degrade with a stated reason; never fall back silently

SITUATION	RESPONSE
No WorldEdit installed	Plugin enables; generation reports itself disabled by name
No boss template	Dungeon generates, with a warning attached to the result
Path cannot reach its target	Best attempt used, warning reported — a short dungeon is never handed over silently
MythicMobs absent	That entry is disabled *with its reason*; no vanilla substitution
Reflection failure	Provider disabled entirely, logged once, never inside the spawn loop
Empty mob class pool	That spawn point is skipped and counted as `refused`
Malformed depth band	Skipped with a message; the rest of the file still loads
Broken MiniMessage	Caught at load (HUD) or fallen back to plain text (boss bar) — never thrown in a timer

32.8 Protect the person operating the server from your own rules

The idle timeout only arms after someone has actually been inside, so an operator inspecting a freshly generated dungeon does not have the rooms deleted underneath them. A forceloaded chunk is skipped during unload, because forceloading is what an operator does while diagnosing something. Bundled rooms never overwrite an existing file, because the same folder is the mapper's workspace. The admin teleport bypass exists so the "no teleporting" rule does not lock staff out of the instances they need to moderate.

32.9 Reproducibility is a debugging feature, not a nicety

> "Procedural generation that cannot be debugged, will not be debugged. Without a seed, bringing back *that broken dungeon* is impossible."

Which is why the retry uses a *derived* seed rather than a fresh random one, why the shuffle source is a single instance, why each room draws from `derive(seed, nodeId)` so contents are stable under reordering, and why `DungeonGenerator.Result` carries the seed it was built from.

32.10 Say what broke, not that something broke

`DungeonLayout.validate()` never runs on the generation path; it exists so that a failure can name the invariant and the two rooms involved. `rejectReason` returns a sentence rather than a boolean, so a DEAD door can explain whether the dungeon ran out of space or ran into itself. `addRoot`'s exception names the

room, its size and the slot size, because the realistic cause is a configuration mistake. `describeProblem()` tells a fresh installation exactly which pool is empty and why.

CHAPTER SUMMARY

- Derive rather than declare; measure when failure would be silent; keep configured numbers honest.
- One-way state that throws; ordering as an explicit correctness argument.
- A pure core behind a tiny interface; degrade loudly with a reason; never a silent fallback.
- Protect the operator from your own rules; make everything reproducible; make errors name the invariant.

CHAPTER 33

The trap catalogue

Everything in this chapter has actually cost time on this project. They are collected here because they share one property: **none of them throws an exception.** Each produces a wrong answer that looks like a right answer.

33.1 FAWE does not leave pasted chunks loaded

This is the most-repeated trap in the codebase, appearing in four systems. A block read in an unloaded chunk answers *air*.

WHERE	SYMPTOM IF YOU FORGET
Console verification	`execute if block` returns nothing; a test "passes" having checked nothing
Teardown entity sweep	0 entities removed; the mobs wake up in the next party's dungeon
Mob population	No floor found anywhere; every room is silently empty
Anything else reading blocks after a paste	Air

33.2 `new Random(seed)` on consecutive seeds

Over 4,000 consecutive seeds, `nextInt(4)` returned only two of the four possible values. Phase 7 will naturally want to seed from an incrementing id or a timestamp. Use `Seeds.from` / `Seeds.derive`.

33.3 Max health is not current health

Set `MAX_HEALTH` to 200 and the mob still has 20. Heal it afterwards. This is described in the code as "the single most common way a stat system looks broken".

33.4 The naive wall rule is correct in square rooms

`|dx| > |dz|` works in every test room until the first rectangular room arrives — possibly months later, built by somebody else. `test_long` exists to fail loudly the moment the formula regresses.

33.5 Off-hand fires a second interact event

Without an `EquipmentSlot.HAND` check, one right-click opens two dungeons and consumes two slots.

33.6 Client-side UI must be removed in `onDisable`

Boss bars and sidebar scoreboards live on the client. After a `/reload`, an unremoved bar keeps counting down with no plugin behind it.

33.7 Formatting errors must never live inside a timer

A malformed MiniMessage tag in a per-second boss bar title would throw once a second, per instance, forever. Validate at load (the HUD's approach) or catch and fall back to plain text (the bar's).

33.8 `unloadChunk`'s boolean is a ticket answer

Paper's chunk system is ticket-based. `false` means "somebody still holds a ticket", not "it failed". Counting only `true` returns reported a flat zero on every close while the chunks genuinely came out of memory.

33.9 Template order changes generation

The cumulative weight scan means the same seed with a different template order builds a different dungeon. Order comes from a sorted file listing, so it is stable — but adding a room changes every seed's output. This is the answer to "why did the same seed give me a different dungeon".

33.10 `ClipboardFormats.findByFile` on the write path

It opens the file to detect the format, so it throws `NoSuchFileException` on a file that does not exist yet. Resolve by alias when writing.

33.11 Two caches, one reload

`RoomTemplateStore`'s cache sits above `SchematicService`'s. Clearing only the lower one leaves stale door metadata in memory and verification proceeds on geometry that no longer exists on disk.

33.12 A time window where a synchronous marker would do

Marking the player as "internally teleporting" for 500 ms would open a 500 ms hole in the teleport block. Bukkit fires the teleport event synchronously inside `teleport()`, so the marker can be exactly as wide as the call.

33.13 Two .gitignore negations, not one

`!path/*.schem` does not cover `path/sub/*.schem`. Theme subfolder rooms were silently not committed until `!path/**/*.schem` was added. And Maven resource filtering must stay restricted to `plugin.yml` — applied

to all resources it corrupts every `.schem` binary.

33.14 Console-driving traps

The old `latest.log` still contains `Done (`, so a readiness check fires early. PowerShell 5.1 writes a UTF-8 BOM on the first stdin write and corrupts the first command. The test world must be deleted between runs or the slot index restarts at zero over stale blocks.

33.15 Environment facts that shape the mob set

The dungeon world is `Environment.NORMAL`, so **piglins zombify**. Rooms are sealed with the daylight cycle off, so undead do not burn — unless a room schematic has an open ceiling. `mobGriefing` is off, which is the only reason creepers are usable.

CHAPTER SUMMARY

- All fifteen produce wrong answers rather than exceptions.
- The FAWE chunk trap alone appears in four separate systems.
- Most of them were found by a number being *zero* when it should not have been — which is why the reports print counts.

CHAPTER 34

Command reference

Every subcommand of `/tdungeons` (aliases `/td`, `/takashidungeons`; permission `takashidungeons.admin`, default op), grouped by what it is for.

34.1 Status and inspection

COMMAND	WHAT IT DOES
`/tdungeons version`	Version and author. One of the three places attribution travels.
`/tdungeons status`	The soft-integration table: WorldEdit, FastAsyncWorldEdit, MythicMobs, Vault. Detected once at enable — a plugin loaded later via PlugMan will not appear until reload.
`/tdungeons world`	Teleport to the dungeon world. A legitimate entry, which is why the join rescue does not fire on world change.
`/tdungeons slots`	Allocated slots.

34.2 Rooms and themes

COMMAND	WHAT IT DOES
`/tdungeons list`	Schematic files on disk.
`/tdungeons rooms`	Loaded templates with type, weight, size and door count.
`/tdungeons room <name>`	One template in detail: each door's anchor and derived wall.
`/tdungeons themes`	Per theme: room count, entrance/boss/normal split, multi-door count, and a warning if an entrance or boss is missing.
`/tdungeons weights [pool]`	The weighted draw distribution as percentages. Matches `GenProbe`'s measured numbers exactly.
`/tdungeons gen`	Generate the eight placeholder test rooms, `.schem` and `.yml` both.
`/tdungeons extract [force]`	Extract bundled rooms from the jar. Never overwrites without `force`. Clears both caches if anything was written.
`/tdungeons reload`	Re-read templates, mob registry and HUD from disk. Runs the non-WorldEdit parts first, so a server without WorldEdit can still reload them.

34.3 Manual geometry testing

COMMAND	WHAT IT DOES
`/tdungeons paste <room> [rot]`	Paste a single room into a raw slot.
`/tdungeons connect <a> <b> [doorA] [doorB]`	Attach two rooms by their doors and report the computed R, origin and box — the values that were hand-checked against the specification.
`/tdungeons free <index>`	Release a raw slot. **Leaves blocks behind, deliberately.** Not a substitute for `close`.

34.4 Dungeons and instances

COMMAND	WHAT IT DOES
`/tdungeons dungeon <theme> [size] [seed]`	Generate a full dungeon and register it as an instance, then **put the caller inside** (not teleport them to the coordinates — see Chapter 23). The theme is implied when only one exists. Same seed, same dungeon.
`/tdungeons instances`	Id, theme/size, room count, slot, state, age, remaining time, players inside.
`/tdungeons enter <id>`	Join an instance as a member.
`/tdungeons leave`	Leave. Admin-permission only today; the real exit is the portal.
`/tdungeons close <id|all>`	Full teardown, reporting blocks, chunks, entities, players and milliseconds.

34.5 Portals

COMMAND	WHAT IT DOES
`/tdungeons portal list`	Id, kind, state, bound instance, position.
`/tdungeons portal create [size] [theme]`	Place a **lobby** portal where you stand. Wild portals are never placed on request — that would be a contradiction.
`/tdungeons portal remove <id|all>`	Remove, restoring the previous block if the plugin placed it.
`/tdungeons portal tp <id>`	Teleport to a portal.

34.6 Mobs

COMMAND	WHAT IT DOES
`/tdungeons mob list [class]`	Definitions, with disabled entries marked in red *and their reason*.
`/tdungeons mob info <id>`	One definition, including the **resolved** `statOverride` and where that value came from.
`/tdungeons mob spawn <id> [difficulty]`	Spawn one where you are looking, with stats applied.
`/tdungeons mob providers`	Each provider's availability and known key count.
`/tdungeons mob reload`	Re-read `mobs.yml`.

34.7 HUD

COMMAND	WHAT IT DOES
`/hud [on|off]`	Player-facing. Permission `takashidungeons.hud`, **default true**. Aliases `/tdhud`, `/dhud`. The preference is session-only — persisting it would mean player data in YAML.
`/tdungeons hud`	Show current HUD settings.
`/tdungeons hud name <text>`	Write the server name into `config.yml` and repaint every open sidebar. Comments in the file are preserved.
`/tdungeons hud ip <text>`	Same for the IP line.

34.8 Permissions

NODE	DEFAULT	GRANTS
`takashidungeons.admin`	op	All of `/tdungeons`
`takashidungeons.hud`	**true**	Toggling your own sidebar
`takashidungeons.teleport.bypass`	op	Teleporting into and out of dungeons — the admin exception to the no-teleport rule

CHAPTER 35

Configuration reference

Two files. `config.yml` holds everything except the mob set; `mobs.yml` holds the mob set, separately, because Phase 9's GUI will rewrite it and a rewritten file loses its comments.

35.1 config.yml

dungeon-world

KEY	DEFAULT	NOTES
`name`	`takashi_dungeons`	Keep it unique so it cannot collide with another world.
`slot-size`	512	Must exceed the largest possible dungeon's footprint. Changing it after the world has content shifts all slot geometry.
`columns`	32	512 × 32 = 16,384 blocks along X, well inside the world border.
`base-y`	64	The Y rooms are seated at.
`reset-on-start`	true	Wipes chunk files at startup. Turn off only if you deliberately build something permanent here. `level.dat` is never touched.

schematics

KEY	DEFAULT	NOTES
`extract-bundled`	true	Off means zero rooms on a clean install. Only servers managing their own room set should disable it.
`force-sync-paste`	false	Diagnostic only. On, generating a large dungeon costs TPS.

generation

KEY	DEFAULT	NOTES
`turn-bias`	2.0	1.0 = no penalty; 2.0 = a straight option still jumps ahead half the time; higher = turns dominate. Applied at **door** selection, never template selection.
`max-attempts`	8	Retries with a derived seed when the path falls short. When they run out, the best attempt is used *and warned about*.
`plug-open-doors`	true	Off, players walk out of rooms into the void. Useful while debugging to see where the graph choked.

instance

KEY	DEFAULT / SHIPPED	NOTES
`duration-seconds`	1800 (300 in the current file — a test value)	**Fixed at creation.** Changing it does not affect open instances.
`empty-timeout-seconds`	300 (60 currently)	0 disables. Only arms after somebody has been inside.
`warn-seconds`	`[300, 60, 10]` ([120, 60, 10] currently)	Each fires once, only while somebody is inside, and fires *below* the threshold rather than at it.
`boss-bar.enabled`	true	Off, the timer still works; it is just invisible.
`boss-bar.title`	MiniMessage	Placeholders `<time> <theme> <size> <rooms>`, all unparsed. Read every tick.

The shipped file carries deliberately short test values, marked as such in its own comments and scheduled to be restored before release.

portal

KEY	DEFAULT	NOTES
`block` / `item` / `label`	amethyst block / amethyst shard / "Dungeon"	Placeholder visual, fully configurable.
`size`	`medium`	Size of the dungeon a right-click opens.
`theme`	empty	Empty means: use the only theme if there is one; refuse with a reason if there are several.
`lobby.refresh-hours`	`[0, 12]`	Hours of the day, not a duration — so a server can announce them. Empty means no wait.
`lobby.points`	`[]`	Fixed portals built at startup. Today's permanent address for lobby portals, since portals do not persist until Phase 7.
`wild.enabled`	true	Wild portals are single-use: consumed with the dungeon they opened.
`wild.worlds`	`[world]`	Empty means any world.
`wild.interval-minutes`	10	Clamped to at least 1.
`wild.max`	3	Concurrent wild portals.
`wild.min-distance` / `max-distance`	64 / 256	The ring around a random **online player**. Nobody online, nothing spawns.
`wild.spacing`	128	Minimum distance between two portals.

hud

KEY	DEFAULT	NOTES
`enabled`	true	Off, nobody sees it and `/hud` stops working.
`show-by-default`	true	The per-player toggle is session-only until Phase 7's database.
`refresh-ticks`	20	20 ticks = 1 second. Matters once coin and XP return real values.
`server-name` / `server-ip`	MiniMessage / plain	Also writable from `/tdungeons hud name|ip`.
`lines`	6 lines	Placeholders: `<player> <coin> <xp> <rank>` unparsed, `<server> <ip>` parsed. **Maximum 16 lines** — one per colour code. Sidebar width follows the longest line.

35.2 mobs.yml

SECTION	CONTENTS
`default-difficulty`	Used by dungeons generated without one specified.
`spawn.density`	Walkable blocks per mob. Higher thins rooms out.
`spawn.min-per-room` / `max-per-room`	Clamps. The minimum only applies to rooms that have floor.
`spawn.entrance`	Whether to populate the entrance. Default off.
`spawn.bands`	`until` = upper bound of `depth/maxDepth`; `classes` = weighted mix. Sorted after parsing.
`difficulty.<level>`	`health`, `damage`, `speed`, `armor`, `armor-chance`. Three separate multipliers on purpose.
`mobs.<id>`	`mob: provider:key`, `class`, `weight`, optional `health` / `damage` / `speed` (number or `[min,max]`), `name`, `baby`, `statOverride`.

> **A CONFIGURATION HAZARD WORTH REPEATING**
>
> The plugin **never overwrites an existing** `mobs.yml`. A file carried over from Phase 3A has no `spawn:` section, so `SpawnRules.DEFAULT` takes over. The values are identical to the shipped ones, so behaviour is correct — but the operator cannot tune a section that is not in their file. Delete it and reload.

CHAPTER 36

Glossary, and what comes next

36.1 Glossary

Anchor

The base-centre block of a door opening, stored as a local coordinate relative to the room's origin. Facing is derived from it, never written.

Back to back

The placement convention in which a child room's door anchor sits exactly one block outside the parent's, so the two walls face each other and no room writes another's blocks.

Branching pool

The normal pool minus single-door templates. Used only while building the critical path, where a dead end kills the branch.

Critical path

The mandatory route from entrance to boss. Its length is `round(rooms × 0.65)`, minimum 2, and it is built first so that playtime has a floor.

DEAD door

A door that was tried and for which every candidate collided or overran the slot. Sealed by the plugger; indistinguishable from a wall in game.

Instance

One live dungeon: a slot, a generated layout, a membership set, a countdown and a teardown. Identity is slot + theme + size + seed.

Plug

A procedurally built wall sealing a door that opens into the void. Its size and material are both measured, never configured.

Slot

A 512-block square of the void world reserved for one instance. Position is a pure function of the index.

statOverride

Per-definition flag deciding whether the plugin writes a mob's attributes. Default comes from the provider — vanilla true, external false. Difficulty scaling rides on the same flag.

Theme

The folder a room sits in. Discovered, never declared; the root folder is `default`.

Turn bias

A coefficient in door ordering that pushes straight continuations back. The *real* mechanism against straight lines is door offsets.

36.2 What comes next

PHASE	SUBJECT	THE INTERESTING PROBLEM IN IT
3C	Boss and ownership	The boss encounter; PDC tags binding mobs to their instance; the kill signal that Phase 4 and Phase 8 both hang off.
4	Loot	Weighted rarity on a 1000 base, and a difficulty multiplier applied *only* to rare and above, with the increase subtracted from common so the total stays 1000. Applying it to every class would leave the ratios unchanged after normalization — that is the trap.
5	Parties	Hangs off the portal's existing "an occupied portal is joined, not reopened" behaviour.
6	Supply mob	Cancel `PlayerInteractEntityEvent` and open a custom GUI. Vanilla villager trading cannot accept custom currency.
7	Database	SQLite (relocated to `com.takashi.dungeons.libs.sqlite`) and MySQL, async, with a schema version field. Persists lobby portals and the HUD preference. Stores four columns per dungeon, not a layout.
8	Public API	`MobProvider` is already shaped for it. Events: `DungeonEnterEvent`, `MobKillEvent`, `DungeonCompleteEvent`. No breaking changes, ever.
9	GUI editors	Over the YAML that already works. This is why `mobs.yml` is a separate file.
10	Map building	4 biomes × 10 room types = 40 schematics, in parallel. **This is the current bottleneck**: the engine is finished and the jar has three entrance rooms.
11–12	Ranks, Market	Separate jars, separate repos, over the Phase 8 API. The HUD's `coin`, `xp` and `rank` rows are already waiting.
13	Release	Multi-version testing, TPS under many instances, documentation, GitHub Releases, SpigotMC / Modrinth / Hangar, and promotional material — which is the project's actual return.

36.3 Where the engine stands

Phase 1 is complete and verified: 135 headless assertions across four probes, plus block-level verification on a live server, plus a measured 100% / 100% / 99.9% critical path guarantee across 3,000 generations. Phase 2 is complete and verified both from the console and in game with two accounts. Phase 3 is at 3C, with 3A and 3B verified in game.

What is missing to make this a product somebody can download and use is not code. It is **rooms** — and that is Phase 10.

CLOSING NOTE

Almost every rule in this book exists because a specific thing went wrong, or would have gone wrong silently, and somebody measured it. The most useful habit to take from the codebase is the one that produced those measurements: when a decision has a plausible wrong answer *and* a failure mode with no exception attached, do not reason about it — run it.